

Vectorizing Complex Programs for Modern Architectures

by

Yishen Chen

B.S., University of Illinois (2018)

S.M., Massachusetts Institute of Technology (2021)

Submitted to the Department of Electrical Engineering and Computer Science

in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

May 2026

©2026 Yishen Chen. All rights reserved. The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by: Yishen Chen

Department of Electrical Engineering and Computer Science

February 27, 2026

Certified by: Saman Amarasinghe

Professor of Electrical Engineering and Computer Science

Thesis Supervisor

Accepted by: Leslie A. Kolodziejski

Professor of Electrical Engineering and Computer Science

Chair, Department Committee on Graduate Students

Vectorizing Complex Programs for Modern Architectures

by

Yishen Chen

Submitted to the Department of Electrical Engineering and Computer Science
on February 27, 2026, in partial fulfillment of the
requirements for the degree of
DOCTOR OF PHILOSOPHY

Abstract

Vector instructions are ubiquitous in modern processors, enabling efficient fine-grained parallelism across a wide range of architectures. Despite decades of research, auto-vectorization remains difficult in practice, and production compilers often fail to vectorize real-world programs due to limitations in handling control flow, ambiguous dependencies, and instruction complexity.

This thesis revisits Superword-Level Parallelism (SLP) as a general framework for vectorization and presents three key contributions that address these core challenges. First, I introduce SuperVectorization, a generalized SLP vectorizer that uses a new intermediate representation, Predicated SSA, to vectorize across basic blocks and loop nests. Second, I describe a fine-grained program versioning framework that handles ambiguous dependencies with runtime checks, allowing vectorization of programs that static techniques cannot analyze precisely. Finally, I present VeGen, a system that supports increasingly common vector instructions that fall outside the traditional SIMD model, using only their documented semantics.

These techniques collectively extend the capability auto-vectorization and enable more effective use of vector hardware. My prototype achieves significant speedups across a variety of benchmarks, demonstrating the viability of these techniques as a basis for the next generation of auto-vectorizers.

Thesis Supervisor: Saman Amarasinghe

Title: Professor of Electrical Engineering and Computer Science

Acknowledgments

I would like to thank my Saman Amarasinghe for advising and mentoring me over the past seven years, and for believing in me when I could not believe in myself.

I would like to thank Charith Mendis and Michael Carbin, who helped me immensely with my first few papers and for giving me research guidance in my first few years at MIT.

I would also like to thank everyone in the COMMIT group for their friendship, support, and insightful research discussions. In particular, I am grateful for the many discussions with Ajay Brahmakshatriya, Caleb Kim, Catherine Mohr, Changwan Hong, Charith Mendis, Daniel Donenfeld, Filippo Carloni, Francesco Peverelli, Fred Kjolstad, Jaeyeon Won, Jonathan Zhou, Manya Bansal, Teodoro Collin, Willow Ahrens, and Yunming Zhang.

Finally, I would like to thank Mary McDavitt for her constant encouragement—and candies—over the years.

Contents

1	Introduction	12
2	Background	19
3	SuperVectorization: Control-flow vectorization for SLP	24
3.1	Introduction	24
3.2	Background and Motivation	25
3.2.1	Vector Instruction Sets	28
3.2.2	Definitions	29
3.3	SuperVectorization	30
3.3.1	Predicated SSA	31
3.3.2	Vector Packing	35
3.3.3	Loop Fusion and Co-iteration	37
3.3.4	Generating Vector Instructions from Packs	42
3.3.5	Lowering to IR with control flow	44
3.4	Implementation	45
3.4.1	Loop Unrolling	45
3.4.2	Dependence Analysis	45
3.4.3	Cost Model	46
3.5	Evaluation	47
3.5.1	TSVC	48
3.5.2	PolyBench	48
3.5.3	ISPC	49

3.6	Related Work	50
3.7	Summary	51
4	Fine-grained program versioning for SLP	58
4.1	Introduction	58
4.2	Running Example	61
4.2.1	Dependence Analysis	61
4.2.2	Motivating Example	63
4.3	Approach	64
4.3.1	Dependence Condition	65
4.3.2	Versioning by Example	69
4.3.3	Inferring Versioning Plans	74
4.3.4	Materializing Versioning Plans	76
4.4	Implementation	79
4.4.1	Optimizing Dependence Conditions	81
4.4.2	Aliasing Annotation	82
4.5	Evaluation	82
4.5.1	Versioning for SLP Vectorization	83
4.5.2	Versioning for redundant load elimination	89
4.6	Related Work	90
4.7	Summary	92
5	VeGen: vectorization beyond SIMD instructions	93
5.1	Introduction	93
5.2	Teaser: Using a neural network instruction	96
5.3	Lane Level Parallelism	97
5.4	VEGEN's Workflow	98
5.5	An end-to-end example of VEGEN using a non-SIMD instruction . .	100
5.5.1	Vector Instruction Description Language	102
5.5.2	Generating Pattern Matchers	103
5.5.3	Pattern Matching	104

5.5.4	Vectorization	104
5.5.5	Code Generation	108
5.6	Vector Pack Selection	108
5.6.1	Pack Selection Using the SLP heuristic	110
5.6.2	Improve the SLP heuristic with Deinterleaving and Transposition	112
5.6.3	Fine-tuning with Vector Combination	113
5.7	Implementation	115
5.7.1	Supporting x86 Vector Instructions	115
5.7.2	Target Instruction Specification	115
5.7.3	Cost Model	118
5.7.4	Supporting Arm Neon Instructions	119
5.8	Experimental Results	120
5.8.1	Synthetic Benchmarks	121
5.8.2	Optimizing Image and Signal Processing Kernels	122
5.8.3	Discussion	124
5.8.4	Optimizing OpenCV's Dot-Product Kernels	125
5.8.5	Optimizing Complex Multiplication	126
5.9	Related Work	128
5.9.1	Auto-vectorization	128
5.9.2	Instruction Selection	129
5.9.3	Superoptimization	130
5.10	Summary	130
6	Conclusion and Future Work	132

List of Figures

1-1	An example program not vectorizable by traditional techniques due to complex control flow.	13
1-2	A example program not vectorizable due to pointer aliasing	14
1-3	An example program not vectorizable due to unanalyzable dependence	14
2-1	Example of a loop nest that benefits from outer-loop vectorization . .	20
2-2	An example showing the workflow of SLP vectorization.	21
2-3	An example of vectorizing a loop with interleaved accesses	23
3-1	Motivating example containing a C code with complex control flow not vectorizable by existing techniques	26
3-2	An example program with control flow but for which if-conversion is not necessary for vectorization	27
3-3	Workflow of SuperVectorization	30
3-4	Definition of Predicated SSA	31
3-5	Example program represented inr Predicated SSA	32
3-6	Definition of Control Predicates.	33
3-7	An example of <i>control-flow equivalent</i> basic blocks	35
3-8	Algorithm for computing control predicates	36
3-9	Algorithm to co-iterate multiple loops	39
3-10	The two loops in Figure 3-1 after co-iteration.	40

3-11	Result of packing and vectorizing the loops in Figure 3-1. <code>vmu</code> denotes vector μ -node. For vector load/store instructions that require masking, we use their original control predicates (e.g., <code>active</code> and <code>active2</code> in Figure 3-10) to compute the masks.	42
3-12	Algorithm for lowering Predicated SSA to a traditional IR with CFG (e.g., LLVM IR)	53
3-13	Result of running SuperVectorization TSVC	54
3-14	Result of running SuperVectorization on PolyBench	54
3-15	Result of running SuperVectorization and Polly on PolyBench	54
3-16	An example benchmark from TSVC where our vectorizer gains a $2.91\times$ speedup over LLVM's vectorizers	55
3-17	Code snippets from Volume Renderer, which SuperVectorizer yields a $3.28\times$ speedup over both Clang and ICC	56
3-18	Speedup over LLVM vectorization (Loop + SLP) on the ISPC benchmarks	57
3-19	Reduction in dynamic instruction counts in the ISPC benchmarks . .	57
4-1	Running example for fine-grained versioning	63
4-2	Running example transformed so that the stores become independent	63
4-3	Definition of predicated SSA.	64
4-4	The running example from Figure 4-1 represented in predicated SSA .	64
4-5	Definition of dependence condition.	65
4-6	Algorithm for determining the dependence condition	66
4-7	Conditional dependence graph of the running example	68
4-8	Flow graph constructed for the running example	70
4-9	A primary cut to separate the two stores in the dependence graph . .	71
4-10	A secondary cut to make the comparison (c) independent of the stores.	72
4-11	Updated primary cut from Figure 4-9	73
4-12	The final nested versioning plan derived from the cuts shown in Figure 4-10 and Figure 4-11	73
4-13	Algorithm for versioning plan inference	77

4-14	Algorithm for materializing a set of versioning plans	78
4-15	The running example with the versioning plan (from Figure 4-12) materialized	80
4-16	Results of fine-grained versioning + SuperVectorization on PolyBench	84
4-17	Snippet of floyd-warshall, which is vectorizable with versioning + Su- perVectorization but not by traditional techniques	86
4-18	Sketch of the code generated by our versioning algorithm for floyd-warshall	87
4-19	Result of versioning + SuperVectorization on TSVC	88
4-20	TSVC's s281, made vectorizable by fine-grained versioning	88
4-21	TSVC's s342, also made vectorizable by versioning	88
4-22	s258, made vectorizable by versioning	89
4-23	Result of running redundant load elimination (with versioning) on the SPEC 2017 Floating Point benchmark suite.	90
5-1	Examples of SIMD and non-SIMD instruction in the AVX2 instruction set	94
5-2	Result of vectorizing a dot-product kernel used by TVM's 2D convolu- tion layer	96
5-3	VEGEN's workflow	99
5-4	Semantics of <code>pmaddwd</code> formalized in VEGEN's vector instruction de- scription language	101
5-5	Pattern matcher generated for the vector instruction <code>pmaddwd</code>	101
5-6	the instruction DAG corresponding to the example scalar program. . .	102
5-7	Generated vector code	103
5-8	Syntax of the Vector Instruction Description Language (VIDL)	103
5-9	Semantics of <code>vpmuldq</code>	106
5-10	Recurrence used by the SLP heuristic to decide whether to produce a vector operand v directly via a vector pack or by vector insertions . .	109
5-11	Improved recurrence for choosing a vector producer for a given vector operand	114

5-12 Results on some instruction selection tests ported from LLVM's x86 backend	121
5-13 Results on kernels selected from x265 and FFmpeg	123
5-14 Result of vectorizing idct4 using VEGEN	124
5-15 OpenCV's dot-product kernels specialized for AVX2 and AVX-512 (VNNI) and different kernel sizes.	126
5-16 Vector code that VEGEN generated for the <i>int32</i> $\times 8$ dot-product kernel in OpenCV	127
5-17 Result of vectorizing comoplex multiplication	127

List of Tables

5.1	Summary of the non-SIMD Neon instructions that VEGEN supports .	120
5.2	Evaluation Benchmarks	122

Chapter 1

Introduction

Vector parallelism, along with instruction-level parallelism (ILP), is one of the two mainstream techniques for supporting fine-grained parallelism in modern architectures. Compared to ILP, vector parallelism can be supported with much a simpler hardware structure and with less overhead [Rajovic et al., 2011]. With vector instructions, the savings appear both in instruction fetch and decode—thanks to the more compact encoding—and in the energy and area otherwise spent by the hardware detecting and managing parallelism dynamically [Kozyrakis and Patterson, 2003, Barredo et al., 2020]. Not only is ILP challenging to support efficiently in hardware, but real-world programs also tend to exhibit only limited ILP: substantial parallelism often appears only in instruction windows larger than 10,000, which are impractical for real hardware to exploit [Postiff et al., 1999]. Due to these advantages, vector instructions are ubiquitous in modern processors: be it embedded processors, mobile devices, desktops, or high-end servers [Apple, 2025, Qualcomm, 2024a,b]. However, unlike instruction-level parallelism, where the programmer can rely on the hardware to exploit the parallelism automatically, taking advantage of vector-level parallelism requires upfront effort to rewrite the program—either manually or via compiler auto-vectorization—to use the vector instructions. Automatic vectorization is therefore critical to making the performance benefits of vector instructions accessible to non-expert programmers. Yet after decades of research, compilers still struggle to vectorize many programs in the

real world [Adit and Sampson, 2022, Maleki et al., 2011, Pohl et al., 2016], and many programmers still resort to hand-vectorization when performance matters.

To understand why auto-vectorization is falling short, it is important to understand the two sources of complexity that a vectorizing compiler faces on the language level: complex, data-dependent control flow and dependence analysis. High-level sequential languages have features that complicate both, making auto-vectorization challenging [Pharr and Mark, 2012].

```
int haystack[];
int needles[2];
int needle_idx[2];

for (int i = 0; i < n; i++)
    if (haystack[i] != needles[0]) {
        needle_idx[0] = i;
        break;
    }

for (int i = 0; i < n; i++)
    if (haystack[i] != needles[1]) {
        needle_idx[1] = i;
        break;
    }
```

Figure 1-1: An example program not vectorizable by traditional techniques due to complex control flow.

High-level programs often contain non-trivial, data-dependent branches. Mapping such control flow to vector execution typically requires a heroic level of transformation—assuming the compiler can even identify the available parallelism in the first place. Figure 1-1 shows one such program. The example contains two loops searching for two different keys within the same array. The two loops are independent and perform the same type work so are amenable for vectorization. Vectorizing this program however requires much more than simply *packing* the scalar instructions into vector ones. Before any vector instructions are generated, the vectorizer needs to generate a single loop that properly simulates the executions of the two originally disjoint loops; this task is furthermore complicated by the fact that the two loops may have different

run-time trip counts and therefore not fusible by traditional techniques.

Dependence analysis is another fundamental source of complexity that makes auto-vectorization difficult. To vectorize a program safely, the compiler must first prove that certain computations are independent. This analysis is often complicated by pointer aliasing. Figure 1-2 shows an example program that is only vectorizable if the compiler can prove that the arrays `A` and `B` do not alias. Alas, precise alias analysis is undecidable in general and NP-hard even in more restrictive cases [Ramalingam, 1994, Horwitz, 1997]. As a result, production compilers such as LLVM in practice employ rudimentary alias analyses that trade precision for compile-time efficiency, often missing feasible vectorization opportunities. Aliasing is also not the only source of imprecise dependence analysis: many programs contain parallelism that can only be detected at run time. Figure 1-3 shows such an example, where the program is not vectorizable if the variable `inc` is in the range of $[-1, -VL]$ (where VL is the native vector length). In many cases such information is only obtainable at run time but not statically.

```
for (int i = 0; i < N; i++)
    A[i] += B[i];
```

Figure 1-2: A simple program that is only vectorizable if the compiler can show the arrays `A` and `B` do not alias.

```
for (int i = 0; i < N; i++)
    A[i] += A[i + inc];
```

Figure 1-3: A program that's vectorizable if the variable `inc` is outside the range of $[-1, VL]$, where VL is the native vector length.

Another source of complexity that a vectorizing compiler faces comes from the hardware. Modern vector ISAs expose a vast number of instructions. As an example, we can consider AVX512, which is itself a family of extensions ranging from floating-point arithmetic (AVX512F) to machine learning acceleration (AVX512-VNNI). There are 625 distinct mnemonics in AVX512, of which 574 are for vector compute and data-movement, with the rest of the 51 mnemonics used for vector-mask manipulation [Intel

Corporation, 2025]. These are however only mnemonics. When we consider the different vector widths (e.g., 128-bit vs 256-bit) that are supported, the number of distinct instruction encodings balloons to more than 2000. Although many of these instructions also perform operations that are much more complicated than traditional instructions. In AVX512, for integer additions alone, there are 8 different mnemonics to perform different variants of saturating add.

Further complications arise from the fact that vector instructions are no longer necessarily SIMD (single-instruction, multiple-data). Some instructions do not simply apply single operations independently across the different vector lanes. The SSE3 instruction `addsubps`, for example, performs vector additions and subtractions on alternating vector lanes. Most instructions designed to accelerate neural network workload are also non-SIMD because they use operations that consume values coming from different input vector lanes. The VNNI instruction `VPMADDUBS` computes a vector of partial dot-products by taking in two vectors of 8-bit integers as input, zero-extends the first input vector, sign-extends the second, multiplies the extended intermediate values, and sums up every pairs of adjacent multiplications before the result is finally saturated to 16-bit values. Most of these vector instructions are not supported by production vectorizers because all existing vectorization techniques are only designed to target SIMD parallelism. In the rare cases where compiler developer put in dedicated effort to support such instructions, the support comes with considerable human effort. For example, to support the aforementioned `addsubps` instruction, LLVM’s developers made 63 distinct commits in the course of 15 years, with the changes spread between its SLP vectorizer and the x86 backend [LLVM, 2025].

The goal of this thesis is to develop a vectorization framework can reliably auto-vectorize real world programs despite these complexities. The hope is that, in the future, vectorizing compilers can utilize vector instructions as effectively as today’s compilers optimize scalar code. My research addresses these complexities with three approaches.

Control-flow Vectorization. To vectorize control-flow systematically, I first developed a new intermediate representation (IR) called Predicated SSA. To simplify

control-flow code motion, *Predicated SSA* represents the control dependencies of individual instructions and loops explicitly with *control predicates* that indicate whether the instructions (or loops) should be executed at run time. Because the instructions and loops have their own predicates, reordering instructions in this new IR is as simple as intra-basic block code motion in traditional compiler IR. Based on this new IR, a more general and aggressive SLP vectorizer [Larsen and Amarasinghe, 2000] that can now pack instructions across different basic blocks and loops. This new vectorizer can vectorize many programs with complex data-dependent control flow that stumps even the best production vectorizers.

Fine-grained Program Versioning. To handle program dependencies that cannot be analyzed statically, I introduce a general framework for *fine-grained* program versioning, which allows for the first time targeted, on-demand versioning at the granularity of groups of instructions as well as loops. Given a set of instructions whose independence are desired but cannot be proven by the vectorizer, the framework first identifies a minimal set of conditions under which the pack is provably independent and then rewrites the program to dynamically check those conditions. This approach allows us to vectorize many programs with complex dependencies that are not vectorizable with traditional techniques by *speculating* on both data and control dependencies that otherwise block vectorization.

In addition to vectorization, I also demonstrate the generality of this framework by applying it to redundant load elimination, speculatively removing redundant loads that cannot be proven statically due to memory aliasing or complex control-flow. With the fine-grained versioning framework, I was able to implement this transformation with relatively light effort.

Vectorization with Non-SIMD Instructions. To address the increasing complexity of modern vector ISAs, I introduce *Lane-Level Parallelism* (LLP), a model of short-vector parallelism allowing the vector instructions to perform different operations on different lanes as well cross-lane communications. This model captures the semantics of both traditional SIMD and the newer class of non-SIMD vector instructions.

To exploit this new model of parallelism, I introduce *VeGen*, a system for automatically generating target-specific vectorizer given a formal description of the target instructions semantics (e.g., from Intel’s Intrinsics Guide). With this approach, we can automatically use many irregular, complex vector instructions that existing compilers fail to utilize.

Contributions

This thesis makes the following contributions:

- **Predicated SSA**, a new intermediate representation that explicitly encodes control dependencies as symbolic predicates, simplifying control-flow code motion and enabling more general vectorization.
- **SuperVectorization**, a control-flow-aware vectorization technique based on Predicated SSA that extracts superword-level parallelism across basic blocks and loop nests.
- **Fine-grained program versioning**, including the concept of *conditional dependencies*, an inference algorithm to identify key dependencies limiting optimizations, and a code generation algorithm that selectively rules out these dependencies at run time.
- **Lane-level parallelism**, a generalization of SLP that captures opportunities exposed by both SIMD and complex, non-SIMD instructions.
- **VeGen**, a vectorizer generator that leverages lane-level parallelism to automatically select and combine SIMD and non-SIMD instructions solely from their documented semantics.
- **Empirical evaluations**, showing that:
 - **SuperVectorization** achieves an average 75% speedup (up to 10×) on the ISPC benchmark suite.

- **Fine-grained versioning** combined with SuperVectorization yields speedups of 17% on TSVC and 51% on PolyBench.
- **VeGen** can effectively use non-SIMD instructions, for example achieving $3\times$ speedup on x265’s `idct4`.

Outline

The rest of this thesis is organized as follows. Chapter 2 gives background on SLP vectorization, on which my vectorizer is based on. Chapter 3 describe SuperVectorization and the design of Predicated SSA, which is used both for control-flow vectorization and fine-grained versioning (Chapter 4). Chapter 4 introduces fine-grained versioning, its implementation, and its applications. Chapter 5 introduces lane-level parallelism and VeGen, detailing the high-level idea of VeGen, its workflow, the implementation, and several new SLP packing heuristics that I designed to support non-SIMD instructions.

Chapter 2

Background

Loop vectorization. Allen and Kennedy [1987] pioneered loop vectorization with their seminal work in the 1980s. The basic idea of loop vectorization is to widen instructions in the loop body from scalar to vector instructions. This technique was originally motivated by the long vector architectures that were common at the time (e.g., the Cray machines). While traditional loop vectorization focuses on inner loops by executing the inner-loop iterations in parallel, outer-loop vectorization instead transforms the program to execute the outer-loop iterations in parallel [Nuzman and Zaks, 2008]. Depending on the access pattern of the original scalar code, outer-loop vectorization can be preferable to inner-loop vectorization because it improves data-locality, reduces vector data-movement overhead, or because there is more data parallelism in the outer loop. Figure 2-1 shows one such example, where the j -loop of the matrix multiplication kernel is preferably vectorized for multiple reasons because, firstly, the output matrices $\mathbf{B}$ and $\mathbf{C}$ both have their innermost dimension indexed by j and secondly vectorizing the innermost loop (the i -loop) will require us to vectorize along the reduction dimension (which reduces parallelism).

Traditionally outer-loop vectorization works by interchanging an outer-loop with the inner-most loop and then vectorizing the inner loop. Nuzman and Zaks [2008] proposed an alternative approach that uses unroll-and-jam instead. While one can use SLP vectorization to target loop-level parallelism by unrolling a given loop beforehand,

```

for (int i = 0; i < N; i++)
  for (int j = 0; j < M; j++)
    for (int k = 0; k < K; k++)
      C[i][j] += A[i][k] * B[k][j];

```

Figure 2-1: Example of a loop nest that benefits from outer-loop vectorization. It is more beneficial to vectorize the j-loop than the i-loop.

this approach fails at vectorizing outer-level loops. One can unroll an outer loop to expose the loop-level parallelism as superword-level parallelism that cannot be exploited by existing SLP vectorizers because the independent instructions are now scattered across different disjoint inner loops.

SLP vectorization. In the early 2000s, Larsen and Amarasinghe [2000] developed superword-level parallelism (SLP) vectorization to target the multimedia extensions that were emerging. SLP vectorization works by searching for independent, isomorphic instructions in straight-line code and replacing them with vector instructions. Because SLP vectorization targets straight-line code, it requires much simpler dependence analyses (as opposed to the more complex loop-dependence analyses required by loop vectorization).

SLP models a restricted kind of short-vector parallelism, in which instructions execute multiple, identical scalar operations in parallel with the inputs and outputs of these operations residing in packed storage locations (e.g., vector registers). SLP vectorization works in three steps. First, the vectorizer runs an SLP packing heuristic to select groups of independent, isomorphic instructions to pack together. Second, the vectorizer reorders the instructions so that the dependences of any group of packed instructions appear before the group. Finally, the vectorizer replaces each group of packed instructions with an equivalent vector instruction; During code generation, the vectorizer also needs to take care to insert the necessary vector data-movement instructions when there is a mismatch between the vector (and scalar) values that are produced and the vector (and scalar) values that are being used as instruction operands. For example, if an instruction is packed (together with some other instructions) but has a scalar user that is not packed, then the code generator needs to emit additional

code to extract the scalar values from the appropriate vector lane.

Figure 2-2 shows this process in action. Figure 2-2a shows the example C program to be optimized by SLP vectorization. For this example, we assume the packing heuristic selects two packs, one consists of the values `a0` and `a1` and another of the values `b0` and `b1`. Notice that the value `b0` has another scalar user (the one defining the value `t`), which is left unpacked. Figure 2-2b shows the instructions reordered according to the packing aforementioned decisions. As we can see, instructions belonging to the same packs are now grouped together. It is crucial that such reordering preserves the original program semantics by respecting all possible data dependencies. Figure 2-2c shows the final result of vectorization. All packed instructions are replaced with the equivalent vector instructions. The code generator additionally emits an `extract` instruction to extract the value `b0` which is now produced is now placed at the first lane of the vector `vb`.

<code>a0 = A[0];</code>	<code>a0 = A[0];</code>	<code>va = A[0:2];</code>
<code>b0 = a0 + 1;</code>	<code>a1 = A[1];</code>	<code>vb = a + {1, 1};</code>
<code>t = b0 * 2;</code>	<code>b0 = a0 + 1;</code>	<code>b0 = extract vb, 0;</code>
<code>a1 = A[1];</code>	<code>b1 = a1 + 1;</code>	<code>t = b0 * 2;</code>
<code>b1 = a1 + 1;</code>	<code>t = b0 * 2;</code>	
(a) Scalar program to be vectorized.	(b) Instructions reordered after pack selection.	(c) Final vector program after code generation.

Figure 2-2: An example showing the workflow of SLP vectorization.

Compared to loop vectorization, SLP vectorization is both simpler and more flexible. SLP vectorization is simpler because it does not target parallelism that spans different loop iterations, thus not requiring loop dependence analysis to reason with loop-carried dependence. SLP vectorization can nonetheless exploit inner-loop parallelism using loop unrolling [Larsen and Amarasinghe, 2000, Rocha et al., 2020].

SLP vectorization is also more flexible at exploiting the type of short-vector parallelism implemented by existing multimedia architecture extensions. The SLP framework defines a search space that specifies which instructions can be packed together—subject to dependences and the capability of the target architecture, leaving

the SLP heuristic free to vectorize however it sees fit [Rosen et al., 2007, Liu et al., 2012, Porpodas et al., 2019, Porpodas and Jones, 2015, Porpodas et al., 2015, 2018, Mendis and Amarasinghe, 2018]. In contrast, loop vectorization mechanically transforms each loop instruction to a wider vector instruction. Adapting loop vectorization to handle cases that deviate from this assumption requires research in itself.

Consider the loop in Figure 2-3a as an example of SLP vectorization’s flexibility. A naive loop vectorizer would emit instructions similar to those in Figure 2-3b, vectorizing the even and odd accesses separately, losing performance because the memory accesses are not contiguous and are less efficient. To address this inefficiency, Nuzman et al. [2006] extended loop vectorization to support interleaving and generate instructions similar to the ones in Figure 2-3c. In contrast, an SLP vectorizer (with unrolling) vectorizes such loops effectively without any extensions because SLP is loop agnostic.

This flexibility offers many interesting opportunities. For example, Porpodas et al. [2019] exploited the commutative instructions to increase the available parallelism. Another example is my work (discussed in Chapter 5) to extend SLP to include more complex vector instructions such as the new neural network instructions [Chen et al., 2021]. Another important benefit is that a SLP vectorizer can support partial-vectorization natively, while a loop vectorizer either vectorize the whole loop or not at all. In all of these examples, the overall framework of SLP remains unchanged: the vectorizer uncovers data parallelism from straight-line code by packing independent instructions into vector instructions.

```

for (i = 0; i < n; i+=2) {
    a[i]    = b[i] + c[i];
    a[i+1] = b[i+1] + c[i+1];
}

```

(a) An example loop with interleaved accesses

```

for (i = 0; i < n; i+=8) {
    t  = b[i:i+8];
    t2 = c[i:i+8];
    b_even = shfl-even t;
    c_even = shfl-even t2;
    e = b_even + c_even;
    b_odd  = shfl-odd t;
    c_odd  = shfl-odd t2;
    o = b_odd + c_odd;
    a[i:i+8] =
        interleave e, o;
}

```

(b) Result of applying loop vectorization naively

```

for (i = 0; i < n; i+=4) {
    /*
       After unrolling,
       before SLP vectorization:
       a[i]    = b[i] + c[i];
       a[i+1] = b[i+1] + c[i+1];
       a[i+2] = b[i+2] + c[i+2];
       a[i+3] = b[i+3] + c[i+3];

       After SLP vectorization:
    */
    a[i:i+4] = b[i:i+4]
               + c[i:i+4];
}

```

(c) Result of interleave-aware loop vectorization. SLP vectorization can discover the same vectorization scheme natively.

Figure 2-3: An example of vectorizing a loop with interleaved accesses. Nuzman et al. [2006] proposed an extension to loop vectorization to vectorize such an example. SLP vectorization can discover the equivalent vectorization scheme.

Chapter 3

SuperVectorization: Control-flow vectorization for SLP

3.1 Introduction

I introduce SuperVectorization, a new vectorization framework that generalizes SLP vectorization to pack independent instructions across different basic blocks and loop nests—such loops can be imperfectly nested and have different trip counts or multiple side exits. Because our framework can pack instructions across control-flow regions such as basic blocks and loops, it simultaneously subsumes the roles of inner-loop, outer-loop, and straight-line vectorizers while retaining the flexibility of SLP vectorization (e.g., partial vectorization). For example, outer-loop vectorization in our framework involves unrolling an outer loop and then packing the instructions from the inner loops duplicated by the unroller. Our evaluation shows that a single instance of our vectorizer is competitive with and, in many cases, significantly better than LLVM’s vectorization pipeline, which includes both loop and SLP vectorizers.

Central to our framework is *Predicated SSA*, a new intermediate representation

(IR) that we developed to simplify the pervasive inter-basic block code motion required to target SLP that spans different basic blocks or loops. Rather than using a control-flow graph (CFG), this IR represents the input program as a flat list of instructions and loops and tracks the control dependence of each instruction explicitly with a boolean formula that expresses whether the instruction should execute. We refer to such formulas as *control predicates*.¹ Using Predicated SSA, inter-basic block code motion is straightforward: We just move an instruction (or loop) together with its control predicate, and no information is lost because we can recover control flow from equivalent control predicates.

With the capability to pack instructions across control-flow regions like basic blocks and loops, our framework simultaneously takes the roles of inner-loop, outer-loop, and straight-line vectorizer while being able handle (divergent) control-flow natively. For example, outer-loop vectorization in our framework is simply unrolling an outer loop and then packing the instructions from the inner loops duplicated by the unroller.

3.2 Background and Motivation

Our goal is to develop a vectorization strategy that retains the simplicity and flexibility of SLP vectorization while generalizing it to systematically handle control flow. To this end, we review the background on SLP vectorization and motivate the technical challenge of extending SLP vectorization to handle control flow.

Handling Control Flow in SLP Vectorization. SLP vectorization has so far been limited to target parallelism within individual basic blocks, despite SLP captures a more general form of parallelism. Consider, for example, the sequential program in Figure 3-1, which performs two independent linear searches over the same array for two different elements. No existing vectorizers can exploit the available (superword-level) parallelism among the loops.

Code Motion. The key challenge to targeting SLP that spans arbitrary control-flow

¹Predicated SSA does not perform predicated execution and executes instructions conditionally similar to a traditional IR with CFG.

```

int haystack[];
int needles[2];
int needle_idx[2];

for (int i = 0; i < n; i++)
    if (haystack[i] != needles[0]) {
        needle_idx[0] = i;
        break;
    }

for (int i = 0; i < n; i++)
    if (haystack[i] != needles[1]) {
        needle_idx[1] = i;
        break;
    }

```

Figure 3-1: Running example of a sequential C code that we will vectorize. The program performs two linear searches over the same array twice to find the indices of two different elements (`needles[0]` and `needles[1]`). The two loops are independent, and we will exploit the parallelism between the two loops by packing their instructions together.

regions is inter-basic block code motion. Exploiting the SLP available in Figure 3-1, for example, requires moving instructions from one loop to another. Code motion in a traditional IR with CFG is challenging because it requires restructuring the CFG and the complexity of doing so increases with the complexity of control dependences. Consider the task of hoisting a store to an earlier location in the CFG as an example. Because relaxing the condition under which the store executes may lead to segfault, we need to preserve the condition of the store. Consequently, hoisting a store in general entails 1) finding a basic block that is *control-flow equivalent* to the original basic block—and, if such a basic block does not exist, restructuring the CFG to create the block—and 2) recursively hoisting the dependences of the store to preserve data dependences, which may require further changes to the CFG.

If-conversion. If-conversion [Allen et al., 1983] converts control dependences to data dependences by transforming the program to unconditionally execute both sides of every conditional branch and replaces control-flow joins (i.e., the ϕ -nodes in SSA) with

```

if (c)
    x = a[i];
else
    x = b[i];

```

```

if (c)
    y = a[i+1];
else
    y = b[i+1];

```

(a)

```

va = vload a
vb = vload b
x_y = vselect {c,c}, va, vb

```

(b) Result of vectorizing the code from Figure 3-2a by aggressive if-conversions and then SLP vectorization

```

if (c)
    x_y = vload a
else
    x_y = vload b

```

(c) A more efficient way to vectorize the code from Figure 3-2a.

Figure 3-2: An example where if-conversion is not necessary for vectorizing control flow. Shin et al. [2005]’s approach generates code in Figure 3-2b and requires a post-vectorization pass to recover control flow. Our approach can directly reason with control flow and emits code in Figure 3-2c without if-conversion.

select instructions (which chooses two alternative values based on a boolean condition, similar to the ternary operator in C/C++). Because if-conversion unconditionally executes instructions that may not otherwise execute, naively applying if-conversion can decrease performance [Shin et al., 2005, Moll and Hack, 2018]. Nonetheless, if-conversion is necessary for vectorizing divergent control flow (i.e., branches with different conditions).

Prior Work. Shin et al. [2005] proposed handling control flow in SLP vectorization with if-conversion. After applying if-conversion to eliminate all (forward) control flow, they use SLP vectorization as a black box to vectorize instructions within the relatively few large basic blocks. Their approach has two drawbacks. First, without directly addressing the code motion problem, their approach cannot target parallelism that spans different loops nests (e.g., the program in Figure 3-1). A secondary drawback is that because their approach unconditionally eliminates all control flow, it can pessimize performance when if-conversion is not necessary, or when vectorization is

simply not profitable. Consider the scalar C-code in Figure 3-2a. Shin et al.’s approach will first generate the code in Figure 3-2b. The same C code can instead be vectorized more efficiently as in Figure 3-2c, where only one vector load is executed at run time (versus two loads in the previous example). To address the second drawback, Shin et al. proposed using a post-vectorization pass that tries to recover some control flow. They proposed a post-vectorization pass that tries to recover some control flow from the if-converted instructions. In contrast, our proposed framework can directly generate the code in Figure 3-2c in a single pass. And being able to exploit parallelism that spans across different loops is interesting to us because when combined with an unroller, this capability leads to a simple way to support outer-loop vectorization.

Our Approach. The key to our approach is Predicated SSA, an IR that we developed to simplify the pervasive code motion required to target SLP that spans different basic blocks (and loops). Instead of using a CFG, which complicates code motion, Predicated SSA represents the input program as a flat list of instructions and loops while tracking the control dependences of each instruction separately with a symbolic boolean formula that expresses whether the instruction should execute. Code motion in Predicated SSA only entails reordering an instruction and its dependences as one would for intra-basic block code motion; such reordering in Predicated SSA is safe because we always reorder an instruction together with its control predicate, and no information is lost. Predicated SSA does not commit to any code layout decision (e.g., assigning an instruction to a particular basic block). Only after vectorization and deciding an overall execution schedule for the instructions, do we revert back to a traditional IR with CFG.

3.2.1 Vector Instruction Sets

Our approach targets vector instruction sets with fixed vector width. This covers Intel’s vector extensions (SSE, AVX2, and AVX-512) and Arm’s Neon instruction set. In the rest of this section, we discuss the hardware features that our approach either relies on or benefits from.

Selection/Blending. Depending on their control dependences, our approach may independent control-flow joins into data-flow joins in the form of *vector selects* (*blending*). Intel’s vector extensions support this operation with their **blend** family of instructions (e.g., **blendvps** for blending two packed vectors of single-precision floats). Arm’s Neon extension supports this operation with the **bsl** instruction (bitwise select).

Gather/Scatter. Traditional vector instruction sets only support loading/storing contiguous memory locations, and vector gather/scatter allows loading/storing a vector of arbitrary pointers. Although not a required feature for our technique, vector gathers improve performance for applications with irregular memory access patterns. When the target hardware does not support vector gather/scatter, our approach emits equivalent (but less efficient) scalar loads/stores and vector insertion instructions.

Predication. While vectorizing conditional load/store instructions (with distinct conditions), we require the ability to dynamically suppress the effect of vector lanes whose conditions evaluate to false. If a target architecture does not support predicated vector load/store instructions, our approach will emit sequential scalar stores with branches instead. AVX-512 supports predicated (masked) loads and stores.

3.2.2 Definitions

In this section, we review the compiler terminologies that we will use in Chapter 4.3.

Let b_1 and b_2 be two basic blocks in a given CFG. b_1 *dominates* b_2 if every path from the entry basic block to b_2 must go through b_1 . Relatedly, b_1 *post-dominates* b_2 if every path from b_1 to the exit node goes through b_2 .

The *post-dominance frontier* of a basic block b is the set of all basic blocks b' such that b' is not post-dominated by b but has a predecessor post-dominated by b .

Let b be a basic block. The set of basic blocks that b is *control-dependent* on is exactly the *post-dominance frontier* of b [Cytron et al., 1991].

Gated SSA. Our vectorizer uses Predicated SSA, an IR that borrows many ingredients from gated SSA [Ottensstein et al., 1990]. Gated SSA extends SSA [Cytron et al., 1991] by differentiating its ϕ -nodes into two categories.

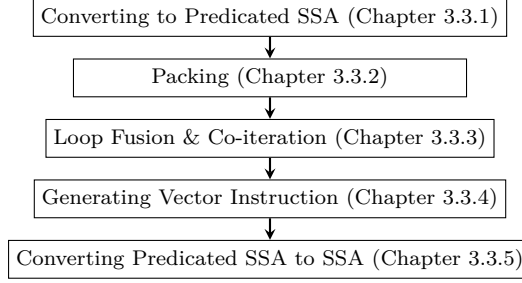


Figure 3-3: Vectorization workflow in our framework

- For ϕ -nodes placed at loop headers, gated SSA renames them as μ -nodes (which have the same semantics as the loop-header ϕ -nodes in SSA).
- For each n -ary ϕ -nodes used for forward control-flow join, gated SSA replaces it with $n - 1$ gating operators called γ -nodes. A γ node selects two alternative incoming values (of the original ϕ -node) depending on a branch condition (similar to C's ternary operator).

3.3 SuperVectorization

SuperVectorization is our generalization of SLP vectorization to pack arbitrary, independent instructions from different basic blocks and loops. Such loops can be imperfectly nested, have different trip counts, or have multiple side exits.

The workflow. Figure 3-3 shows our workflow. We begin by transforming an input sequential program from SSA to Predicated SSA, an IR that we developed to simplify the pervasive code motion that is required for vectorizing instructions across basic blocks and loops. Once the input is in Predicated SSA, we find profitable vector packs as one does in traditional SLP vectorization (Chapter 3.3.2). The precise algorithm for finding the vector packs is orthogonal to our work; in our implementation, we use the bottom-up SLP algorithm [Rosen et al., 2007]. Once we have a set of vector packs, we identify loops that share common vector packs and transform the loops so that all instructions from any given pack reside in a single new loop (Chapter 3.3.3). Having ensured that all instructions from the same packs will be in the same loops, we replace

$$\begin{aligned}
fn &::= item_1 : p_1, \dots, item_n : p_n \\
p &::= \mathbf{control\ predicate} \\
item &::= \mathbf{instruction} \mid loop \\
loop &::= \mathbf{with} \ v_1 = \mu_1, \dots, v_m = \mu_m \ \mathbf{do} \\
&\quad item_1 : p_1, \dots, item_n : p_n \\
&\quad \mathbf{while} \ p_{cont} \\
\mu &::= \mathbf{mu}(v_{init}, v_{rec}) \\
\phi &::= \mathbf{phi}(v_1 : p_1, \dots, v_n : p_n) \\
v &::= \mu \mid \mathbf{instructions} \mid \mathbf{constant} \mid \mathbf{argument}
\end{aligned}$$

Figure 3-4: Definition of Predicated SSA. The root of the grammar is fn , representing a given input function. Similar to gated SSA, Predicated SSA represents IR values defined recursively in loops with μ -nodes, where v_{init} represents the value flowing from the loop pre-header, and v_{rec} represents the value flowing from the back edge.

the packed instructions with equivalent vector instructions (Chapter 3.3.4). So far, we have done everything in Predicated SSA (i.e., packing, fusion, and vectorization). Once we replace scalar instructions with vector instructions, we finish by lowering Predicated SSA back to a traditional IR with CFG (Chapter 3.3.5).

We will use the program in Figure 3-1 as a running example. This example contains two disjoint, independent loops. We will pack instructions from the loops together into vector instructions.

3.3.1 Predicated SSA

Figure 4-3 shows the syntax of Predicated SSA, and Figure 3-5 shows the first loop in the running example (Figure 3-1) translated to Predicated SSA.

Control predicate. Predicated SSA forgoes the use of CFG entirely and instead represents the input program as a flat list of instructions and loops while tracking the control dependence of individual instructions and loops with first-class constructs that we refer to as *control predicates*. Figure 3-6 shows the definition of control predicates. A control predicate is a symbolic boolean expression that indicates whether an instruction (or loop) should execute. Predicated SSA represents each function as a list of *items*. Each item is either an instruction or a loop, and each loop also contains a list of

```

with i = mu(0, i') do
  t      = load haystack[i] : true
  needle = load needles[0]  : true
  found  = cmp eq, t, needle : true
  i'     = add i, 1         : true
  not_found = not found    : true
  lt_n     = cmp lt i', n   : true
while not_found and lt_n    : true
store i, needle_idx[0]     : found

```

Figure 3-5: Example program represented inr Predicated SSA

The first loop in Figure 3-1 translated to Predicated SSA. The *true* control predicates indicate that the items execute unconditionally. For items inside the loop, the predicate indicates that they should execute as long as the loop is not terminated. Observant readers may notice that the load of `needles[0]` is loop-invariant and can be hoisted; we do not hoist the load here because loop-invariant code motion is not part of the IR conversion process.

items. Each item has its own *control predicate* that indicates whether that item should execute. With this representation, code motion is straightforward: We can reorder an instruction (or loop) together with its control predicate without losing information.

Although our use of control predicates is inspired by predicated execution [Park and Schlansker, 1991], Predicated SSA does not perform predicated execution. In Predicated SSA, instructions and loops execute conditionally, depending on their control predicates. In contrast, in predicated executions, predicated instructions execute unconditionally but with their side-effects suppressed conditionally.

Similar to gated SSA, Predicated SSA uses μ -nodes to represent control-flow joins placed at loop headers. For forward control-flow joins, Predicated SSA uses *gated* ϕ -nodes (rather than the γ -nodes from gated SSA); our gated ϕ -nodes are similar to the ϕ nodes in classical SSA, except that the incoming basic block labels are replaced with *control predicates*. The gated ϕ -nodes provide us with a simple way to vectorize ϕ -nodes. While packing multiple ϕ -nodes, if the incoming values of the ϕ -nodes have the same predicates, then we retain control flow and replace them with a single vector ϕ ; if the incoming values of the ϕ -nodes have different predicates, then we lower them into vector selects, emitting the select conditions according to the control predicates.

$$\begin{aligned}
c &::= \mathbf{branch-conditions} \text{ (IR values used for branching)} \\
p &::= \mathbf{true} \mid c \mid \bar{c} \mid p_1 \wedge p_2 \mid p_1 \vee p_2
\end{aligned}$$

Figure 3-6: Definition of Control Predicates.

Converting from SSA to Predicated SSA. We begin by converting the input IR into a canonical form. We assume that the CFG of the input program is reducible and that we can transform each loop into a canonical form, with each loop having:

- a single incoming edge
- a single back edge
- a dedicated loop header
- a dedicated loop pre-header (i.e., the unique predecessor of the loop pre-header)
- a dedicated loop latch (i.e., the unique source of the back edge)

We also normalize each loop in a rotated form where all loops execute at least once (i.e., similar to do-while loops in C).

Once we transform the input into this canonical form, we convert it into Predicated SSA by visiting the input loops recursively. We convert the forward ϕ -nodes into gated ϕ -nodes by replacing the incoming basic block labels with the control predicates of the basic blocks. (We will discuss how to compute control predicates next.) Because all loops are in the canonical form that we just discussed, we translate the ϕ -nodes in the loop headers into μ -nodes by labeling the values flowing from the loop pre-headers as the initial values (v_{init} in Figure 4-3) and the values flowing from the loop latches as the recursive values (v_{rec}).

Computing Control Predicates. One tempting approach to computing the control predicates would be to perform symbolic execution over the branch instructions of the input program, namely, emitting conjunction when taking a branch and emitting disjunction when encountering a control-flow join. However, this approach can introduce unnecessarily complicated predicates that require further simplification. Consider the

example in Figure 3-7 where there is a branch on the condition c . When we join both sides of the branch, we would get the predicate $c \vee \bar{c}$, but we want the more concise predicate *true*. To achieve this aim, we use an algorithm that uses control dependence to directly produce simplified predicates.

Figure 3-8 shows the algorithm for computing the control predicate of a given basic block. The control predicate of an instruction is simply the predicate of its basic block. The idea of the algorithm follows from the observation that whether a basic block b executes depends exactly on two conditions: (1) whether any of its control-dependent basic blocks b' executes, and (2) whether b' takes the branch that leads to b . For condition (1), we simply compute the post-dominance frontier of b , which is equivalent to the control-dependent blocks of b [Cytron et al., 1991]. For condition (2), we observe that because b' is in the post-dominance frontier of b , the branch that leads from b' to b must be the branch $b' \rightarrow b''$, such that b'' is a successor of b' and b post-dominates b'' (i.e., any control flow leaving b'' must end at b); such successor b'' must exist uniquely because b' (the predecessor of b'') is in the post-dominance frontier of b . For the example program in Figure 3-7, our algorithm directly computes a *true* predicate for the `join` block because it post-dominates all preceding blocks and, therefore, has no control dependences.

When computing control predicates, we use the concept of control-flow equivalence. *Control-flow equivalence* is a relation over basic blocks that holds when the execution of one basic block implies that of the other and vice versa. This relation can be difficult to establish for arbitrary pairs of basic blocks. Instead, we will use an incomplete heuristic and say two basic blocks b_1 and b_2 are control-flow equivalent if b_1 dominates b_2 and b_2 post-dominates b_1 . Figure 3-7 shows an example of control-flow equivalent basic blocks.

We ignore backward control flows when we compute the control predicates, and we assign a *true* predicate to basic blocks that are known to be *control-flow equivalent* to the loop header. Consequently, if a predicate is used within a loop, the truthness of the predicate is conditioned on whether the execution reaches a given loop iteration. We track the continue predicate (i.e., whether the back edge is taken) as a special case

```

entry:
    br c, if_true, if_false
if_true:
    br join
if_false:
    br join
join:
    ...

```

Figure 3-7: An example of *control-flow equivalent* basic blocks. The blocks `entry` and `join` are control-flow equivalent because `entry` dominates `join` and because `join` post-dominates `entry`. Consequently, the control predicate of the `join` block is the same as `entry`’s predicate (*true*).

(p_{cont} in Figure 4-3) in Predicated SSA. We compute the *continue predicate* of a loop as the conjunction of the control predicate of the loop latch (i.e., whether we reach the latch) and the branch condition of the back edge.

We compute the control predicate of a loop exit—a loop may have multiple exits—with the intuition that the execution only reaches the exit all of the following is true: (1) if we enter the loop in the first place, (2) if we reach the predecessor of the exit, and finally (3) if the conditional branch preceding the exit is taken. Therefore, the control predicate of a loop exit is the conjunction of (1) the control predicate of the pre-header of the loop from which it is exiting, (2) the control predicate of the predecessor of the basic block (which is inside the loop), and (3) the branch condition of the edge leading to the loop exit.

3.3.2 Vector Packing

Once we convert the input program into Predicated SSA, we run a packing heuristic to decide which instructions should be packed together. Deciding which instructions to pack is orthogonal to our work. SuperVectorization is compatible with arbitrary packing heuristics as long as program dependence is not violated [Rosen et al., 2007, Liu et al., 2012, Porpodas et al., 2019, Porpodas and Jones, 2015, Porpodas et al., 2015, 2018, Mendis and Amarasinghe, 2018]. For this paper, we adapt the bottom-up SLP heuristic [Rosen et al., 2007] for its simplicity. Both GCC and Clang use variants

$$\begin{aligned}
cp_{\text{block}}(b) &= \begin{cases} \text{true} & \text{If } ctrl\text{-}flow\text{-}equivalent(b, header(b)) \\ \bigvee_{b' \in PDF(b)} cp_{\text{edge}}(b', succ\text{-}pdom(b', b)) & \text{Otherwise} \end{cases} \\
cp_{\text{edge}}(b_1, b_2) &= \begin{cases} \text{true} & \text{If } b_1 \rightarrow b_2 \text{ is back edge} \\ cp_{\text{block}}(b_1) & \text{If } preds(b_2) = \{b_1\} \\ cp_{\text{block}}(preheader(b_1)) & \text{If } b_1 \rightarrow b_2 \text{ loop-exiting} \\ \wedge cp_{\text{block}}(b_1) \\ \wedge cond(b_1, b_2) & \\ cp_{\text{block}}(b_1) & \text{Otherwise} \\ \wedge cond(b_1, b_2) & \end{cases}
\end{aligned}$$

Figure 3-8: Algorithm for computing the control predicate of a basic block b , where $PDF(b)$ is the post-dominance frontier of b ; $succ\text{-}pdom(b', b)$ is the successor of b' that is post-dominated by b (there uniquely exists such a basic block because b' is in the post-dominance frontier of b); and $cond(b_1, b_2)$ denotes the branch condition for when the control-flow edge $b_1 \rightarrow b_2$ is taken.

of the same heuristic.

The bottom-up SLP heuristic works as follows. First, it identifies groups of instructions that are known to be vectorizable, which are termed *seed instructions*; common seeds are stores to contiguous memory locations and reductions. Next, the algorithm attempts to find more vectorizable instructions by traversing the use-def chains of the seed instructions packing the operands of the instructions that it encounters along the way. This process stops when it encounters operands that cannot be directly produced by vector instructions (e.g., a group of instructions with different opcodes). At this point, to ensure that the final vectorization decision does not violate program dependence, the heuristic checks that the set of selected vector packs do not have any circular dependences. Finally, the algorithm uses a cost model to estimate the cost benefit of vectorizing the candidate instructions, with the benefit coming from replacing scalar instructions with more efficient vector instructions, and the cost coming from using vector data-movement instructions (e.g., vector shuffle) to produce the operands that are not vectorized.

Once we have decided which instructions to pack, we lower the selected packs into vector instructions as follows. First, if we are packing instructions from different loops,

we transform the loops so that the instructions of any given vector pack belong to the same loops. Then, we lower the packed instructions into vector instructions (e.g., a pack of additions becomes a single vector addition). At this point, we have done everything in Predicated SSA. Finally, we lower the IR back to a traditional IR with CFG (e.g., LLVM IR).

A note on Pattern Matching. Many SLP vectorizers use pattern matching. For example, LLVM’s SLP vectorizer employs pattern matching to (among other use cases) identify and vectorize reductions. Doing pattern matching in Predicated SSA is the same as that in a traditional SSA-based IR (e.g., LLVM) because the former maintains the same use-def information that SSA tracks.

3.3.3 Loop Fusion and Co-iteration

Our framework supports packing instructions from different loop nests. This capability allows, for example, vectorizing the two independent search loops in the running example from Figure 3-1. More importantly, when coupled with a vectorization-aware loop unroller to unroll outer loops, packing instructions across loops is equivalent to outer-loop vectorization. This approach is similar to how Nuzman and Zaks [2008] applied unroll-and-jam to achieve outer-loop vectorization. One major difference here is that in our framework, a packing heuristic need not be aware of the loop structures.

While packing instructions from different loops, we must generate a new loop so that the instructions execute in the same loop. To accomplish this, we either fuse or *co-iterate* the loops, depending on whether the loops are control-flow equivalent (i.e., have the same control predicates) and whether they have the same trip counts.

Loop Fusion. We apply loop fusion if we can prove that loops are independent,² they have the same iterations, they execute under the same condition (i.e., they have identical control predicates), and their parent loops can also be fused. Fusing loops in

²While this is not necessary for fusion in general, it simplifies our implementation.

Predicated SSA requires only creating a new loop whose μ -nodes and loop items are the concatenations of those of the original loops.

Predicated SSA also increases the applicability of loop fusion, despite the latter being a well-studied optimization. Traditionally, compilers only fuse loops that are adjacent [Blainey et al., 2002, Kennedy and McKinley, 1993]. Predicated SSA removes this restriction by making it straightforward to hoist (or sink) any intervening instructions and control flow.

Loop Co-iteration. When we cannot fuse some loops because they execute under different control predicates or because they have different trip counts, we apply a transformation that we call *loop co-iteration*. The intuition behind co-iteration is to interleave the iterations of multiple loops in a single new loop while preserving the execution condition of the original instructions. It is safe to co-iterate when the loops are independent and when their parent loops are also safe to co-iterate. Although we use co-iteration to enable packing instructions from different loops, co-iteration (similar to fusion) is a standalone scalar optimization that is applicable independent of vectorization.

```

1 def coiterate(loops: the loop to coiterate):
2     if identical(loops):
3         return
4
5     parents = {l.parent for l in loops}
6     parent = coiterate(parents)
7     mus = concat(l.mus for l in loops)
8     items = concat(l.items for l in loops)
9     # A map of IR values that tells
10    # if a given loop is active
11    active_mus = {}
12    # Create a boolean Mu node to indicate
13    # whether a co-iterating loop is active
14    for l in loops:
15        active = Mu()
16        mus.append(active)
17        active_mus[l] = active
18
19        pred = parent.control_pred_of(l)
20        active_init = GatedPhi(
21            {pred: true, negate(pred): false})
22        active_next = GatedPhi({
23            And(active, l.cont_pred): true,
24            negate(active): false,
25            negate(l.cont_pred): false})
26        active.set_init(active_init)
27        active.set_rec(active_next)
28
29        items.append(active_next)
30        parent.items.append(active_init)
31        parent.remove_loop(l)
32
33    # We continue iterating
34    # as long as there is an active loop
35    cont_pred = Or(
36        [And(active_mus[l], l.cont_pred) for l in loops]
37    )
38    new_loop = Loop(
39        mus=mus, items=items,
40        cont_pred=Or(cont_pred, parent=parent)
41    )
42    # Strengthen the predicates so that
43    # an item only executes when the parent is active
44    for l in loops:
45        for item in l.items:
46            new_loop.set_control_pred(item,
47                And(active_mus[l],
48                    l.control_pred_of(item)))
49
50    # Introduce mu nodes to guard loop live-outs
51    for l in loops:
52        active = active_mus[l]
53        for x in l.live_outs():
54            x_out = Mu()
55            x_pred = l.control_pred_of(x)
56            x_out_next = GatedPhi({
57                And(active, x_pred): x,
58                negate(active): x_out,
59                negate(x_pred): x_out
60            })
61            x_out.set_init(undef)
62            x_out.set_rec(x_out_next)
63            for user in x.users():
64                if user not in new_loop.items:
65                    user.replaceUsesOfWith(x, x_out)
66
67    return new_loop

```

Figure 3-9: Algorithm to co-iterate multiple loops

```

with i = mu(0, i'),
    i2 = mu(0, i2'),
    i_out = mu(undef, i_out'),
    i2_out = mu(undef, i2_out'),
    found_out = mu(undef, found_out'),
    found2_out = mu(undef, found2_out'),
    active = mu(true, active'),
    active2 = mu(true, active2')
do
    t          = load haystack[i]          : active
    needle     = load needles[0]           : active
    found      = cmp eq, t, needle         : active
    not_found  = not found                  : active
    i'         = add i, 1                   : active
    lt_n       = cmp lt, i', n              : active

    t2         = load haystack[i2]         : active2
    needle2    = load needles[1]           : active2
    found2     = cmp eq, t2, needle2       : active2
    not_found2 = not found2                 : active2
    i2'        = add i2, 1                  : active2
    lt_n2      = cmp lt i2', n              : active2

    active'    = phi(
        active and not_found and lt_n      : true,
        _                                     : false)      : true
    active2'   = phi(
        active2 and not_found2 and lt_n2    : true,
        _                                     : false)      : true
    found_out' = phi(active: found, _: found_out) : true
    found2_out' = phi(active2: found2, _: found2_out) : true
    i_out'     = phi(active: i', _: i_out)       : true
    i2_out'    = phi(active2: i2', _: i2_out)    : true

    cont = or active', active2' : true
while cont : true
store i_out, needle_idx[0] : found_out
store i2_out, needle_idx[1] : found2_out

```

Figure 3-10: The two loops in Figure 3-1 after co-iteration.

Figure 3-9 shows the algorithm for co-iterating multiple loops. Similar to loop fusion, co-iteration entails moving the μ -nodes and the loop items of the original loops into a new loop and the following additional steps:

- For each loop, introduce a boolean μ -node to track whether the loop still has remaining iterations (lines 19-27).
- For each loop item, strengthen its control predicate so that the item only executes when its original loop is active and its original predicate evaluates to true (lines 44-48).
- For each loop live-out (i.e., an instruction with users outside of the loop), introduce a new μ -node to keep the value alive across the iterations even when the loop is no longer active (lines 51-65).
- Set the continue predicate of the new loop to be the disjunction of the active conditions of the co-iterating loops.

Figure 3-10 shows the result of co-iterating the loops in the running example. In the example, we use the booleans `active` and `active2` to indicate whether the original loops are still active. We also insert the new μ -nodes `i_out` and `i2_out` to keep the values `i` and `i2` alive across the iterations where their original loops would have exited already.

```

with i = vmu({0, 0}, i'),
    i_out = vmu({undef, undef}, i_out'),
    found_out = vmu({undef, undef}, found_out'),
    active = vmu({true, true}, active'),
do
  addrs      = vadd haystack, i                : true
  t          = vgather addrs with mask=active : true
  needle     = vload needles with mask=active : true
  found      = vcmp eq, t, needle              : true
  not_found  = vnot found                    : true
  i'         = vadd i, {1, 1}                 : true
  lt_n       = vcmp lt, i', {n, n}            : true
  active'    = vand active, not_found, lt_n    : true
  i_out'     = vselect active, i, i_out        : true
  found_out' = vselect active,
                    found,
                    found_out                  : true
  cont       = vreduce or, active'            : true
while cont                                       : true
vstore needle_idx, i_out with mask=found_out : true

```

Figure 3-11: Result of packing and vectorizing the loops in Figure 3-1. `vmu` denotes vector μ -node. For vector load/store instructions that require masking, we use their original control predicates (e.g., `active` and `active2` in Figure 3-10) to compute the masks.

3.3.4 Generating Vector Instructions from Packs

After selecting a profitable set of vector packs, we generate an optimized vector program, according to the packing decision, as follows:

1. Schedule (reorder) the instructions and loops to satisfy the dependences of the packed instructions.
2. Replace vector packs with vector instructions.
3. Assign control predicates to the vector instructions.

Scheduling. We first schedule the instructions and loops so that for any given vector pack, the dependences of the packed instructions precede the packed instructions (while also satisfying the dependences of the other scalar instructions). We find such

schedules by doing topological sort on the dependence graph formed by the instructions and loops.

Scheduling in our framework is relatively straightforward because we are using Predicated SSA, which enables us to freely reorder instructions and loops. Scheduling instructions and loops with arbitrary control dependences directly on an IR with CFG requires multiple coordinated changes to the CFG and is more complicated.

Emitting Vector Instructions. After scheduling, we replace the packed instructions with their corresponding vector instructions. For most packs, lowering into vector instructions is straightforward. For example, a pack of additions becomes a single vector addition. We need to take special care of packs of ϕ -nodes and packs of loads or stores. Figure 3-11 shows the result of packing the instructions in Figure 3-10 and translating them into vector instructions.

Within a pack of ϕ -nodes, if the control predicate of i 'th ϕ operand is different from the control predicate of i 'th operand of a different ϕ -node, we lower the pack of ϕ -nodes into a vector select; otherwise, we lower the pack ϕ -nodes into a single vector ϕ -node. To lower a pack of n -ary ϕ -nodes into vector select, we emit a chain of $n - 1$ selects and emit the select conditions according to the incoming control predicates of the ϕ -nodes. If we pack loads (or stores) that have different control predicates, we emit masked vector loads (instead of ordinary vector loads or stores).

Assigning Control Predicate. Because we support packing instructions with different control predicates, we need a mechanism to assign a new control predicate for each new vector instruction. For a pack of instructions with identical control predicates, we simply assign the same control predicate. For a pack of instructions that have different control predicates, we set the new control predicate to the strongest control predicate which is also the necessary condition of (i.e., implied by) the original predicates. For example, if a pack contains two instructions with the predicates $c \wedge c_2$ and $c \wedge c_3$, we set the new control predicate to c .

3.3.5 Lowering to IR with control flow

Figure 3-12 shows the algorithm for converting Predicated SSA to a lower-level IR with control flow. We first convert the gated ϕ -nodes into move instructions, whereby we temporarily destroy the single-definition invariant of SSA (which we will restore later). Once we remove all ϕ -nodes, we proceed to reconstruct the CFG level by level, from the top-level function to the deepest nested loop. For each item, we first create (or reuse) a basic block whose control predicate is equivalent to that of the item's (line 16). If the item is a loop, we lower the loop recursively and use the basic block as the loop's pre-header (line 18). If the item is an instruction, we simply move the instruction to the basic block (line 20).

Finally, we generate the back edge and exits of the loop as follows. We first allocate a special variable that indicates whether the loop should continue (line 35). We initialize this variable to *false* in the loop header (i.e., the loop exits by default). Next, we create a basic block based on the control predicate of the loop continue condition (p_{cont} in Figure 4-3), and in this basic block, we set the variable to *true*. Finally, in the loop latch, we create a conditional branch that goes to the header if the variable is true and otherwise to the loop exit (line 45).

We use a utility data structure that we call *block builder* (line 6) to encapsulate the operation of recreating basic blocks from control predicates. While lowering the predicates into basic blocks, the builder maintains a hash table that maps the source predicate of every block to the block itself. We refer to this hash table as the predicate table. For each conjunction of the form $p \wedge c$, the builder queries the predicate table for an existing block equivalent to the predicate p (and recursively creates such a block if it does not exist) and creates a conditional branch (based on the condition c) to the new basic block. For each disjunction, the builder similarly queries the predicate table for basic blocks equivalent to the sub-terms of the disjunctions and joins those basic blocks.

3.4 Implementation

We implemented our vectorization framework within the LLVM compiler infrastructure [Lattner and Adve, 2004].³ We implemented our prototype with 7,831 lines of C++. All components of the prototype (i.e., the new IR infrastructure, dependence analysis, and transformation) are implemented from scratch, independent of LLVM’s existing vectorization infrastructure.

3.4.1 Loop Unrolling

Our vectorizer exposes loop-level parallelism with unrolling. Because SuperVectorization can pack instructions from different loop nests, it achieves outer-loop vectorization by unrolling outer loops and then packing the instructions from the duplicated inner loops. We unroll loops following an approach similar to Rocha et al. [2020]’s. We traverse a given loop nest top-down (parents before children), unrolling each loop virtually and running an SLP packing algorithm (without actually lowering the packs into vector instructions) to determine whether unrolling exposes more parallelism.

3.4.2 Dependence Analysis

Unlike a loop vectorizer, our framework does not need to reason with loop-carried dependence because it does not pack an instruction with a different instance of itself (from a different iteration). Similar to a traditional SLP vectorizer, we only need to check for loop-independent dependence. Nonetheless, our framework requires testing whether any two given loops are independent. To prove that two loops are independent, we first check that there are no use-def dependences (through registers). We then prove that there are no read-write memory dependences between the loops.

For any two memory instructions (with at least one being a write) from the two loops, we overapproximate the ranges of memory locations that could be accessed by the instructions and verify that the ranges do not overlap (using LLVM’s ScalarEvolution analysis). In the cases where the memory accesses are unpredictable and, therefore,

³LLVM 12.0.0.

have compile-time uncomputable bounds, LLVM’s alias analysis framework also allows checking whether two memory regions with unknown sizes overlap. Finally, to improve analysis precision, we have special-case support to distinguish memory accesses that may have overlapping ranges but are nonetheless separated by a sufficient offset in their access strides (e.g., two loops may write to the same buffer but with one writing to the odd indices and the other writing to the even indices).

3.4.3 Cost Model

Our implementation models the cost-saving of vectorization similar to existing SLP vectorizers and only vectorizes when the saving outweighs the cost.

The saving of vectorization comes from replacing the scalar instructions with fewer, more efficient vector instructions, and its cost comes from the overhead of data-movement instructions such as vector shuffles, which are necessary when the vectorizer is unable to directly produce the vector operands for some of the vector instructions. Consider the following example, where the vectorizer is able to vectorize everything except for the two function calls.

```

t1    = call f();
t2    = call g();
add1 = add t1, 1
add2 = add t2, 1
store add1, x[0]
store add2, x[1]
=>
t1     = call f();
t2     = call g();
t      = pack {t1, t2}
vadd0 = vadd t, {1, 1}
vstore vadd0, x

```

Because the vectorizer cannot directly vectorize the calls to `f` and `g`, it then needs to

emit an additional instruction to explicitly pack the result of the calls into a vector using the highlighted instruction.

For each input function, we evaluate the cost saving from vectorization as follows and only vectorize when the saving is greater than zero.

$$c_{scalar} - c_{vector} - c_{shuffle}$$

where the terms c_{scalar} , c_{vector} , and $c_{shuffle}$ are the total cost of the packed scalar instructions, the cost of vector instructions, and the cost of vector shuffling (and other vector data-movement instructions), respectively. For all three terms, we use LLVM’s target-specific cost model [llv, 2022], which estimates the expected cost of individual IR instructions after instruction selection. Our implementation does not currently model the change of cost from restructuring the input CFG (e.g., co-iterating loops).

3.5 Evaluation

We evaluated our framework on three benchmark suits: TSVC [Callahan et al., 1988], PolyBench [Pouchet, 2021], and a collection of sequential C++ programs that Pharr and Mark [2012] used as baselines to evaluate `ispc`, a data-parallel language they designed to target SIMD extensions.

For all the experiments, we used a machine with an Intel® Core™ Platinum 8180 CPU with AVX-512 support and 32 GB of memory and with hyperthreading disabled. Unless otherwise noted, we use the same optimization pipeline as the LLVM -O3 optimizations, except that we run our vectorizer instead of LLVM’s loop and SLP vectorizers (which are disabled in our pipeline). Although the experiment machine comes with AVX-512 support, LLVM 12, by default, sets the maximum vector width to 256-bit (less than the full 512-bit) to avoid performance scaling.

3.5.1 TSVC

Figure 4-19 shows the results on TSVC, a comprehensive benchmark suite for evaluating automatic vectorizers [Callahan et al., 1988]. Our vectorizer is on par with LLVM and yields an average (geomean) speedup of $1.04\times$ over LLVM’s vectorizers on TSVC. Our vectorizer gets significant speedups (more than $1.5\times$ and up to $4.3\times$) on 17 benchmarks and significant slowdowns (more than 20%) on 18 benchmarks. For the 17 benchmarks with significant speedup, the speedups come from partial vectorization and outer-loop vectorization with complex control dependences. Partial vectorization comes naturally for SLP vectorization because the packing heuristic is not required to pack all instructions and can selectively pack any instructions that it finds profitable and legal. In contrast, a traditional vectorizer only vectorizes if all of the loop iterations are independent. Figure 3-16 shows an example benchmark where we obtain a $2.91\times$ speedup by vectorizing an outer loop that also contains data-dependent control flow.

The slowdowns mainly come from benchmarks that have potential data dependences that prevent vectorization. In such a case, LLVM’s vectorizer versions the loop into two duplicates, with one of the duplicated loops vectorized and guarded with a run-time check to ensure that there are no unsafe dependences. We have not yet implemented code versioning and do not vectorize in these cases.

3.5.2 PolyBench

PolyBench [Pouchet, 2021] is a benchmark for polyhedral optimizations, and we use it here as a vectorization benchmark because its applications also benefit from vectorization and because they have data-access patterns that benefit from outer-loop vectorization, and we want to validate that we can effectively perform outer-loop vectorization with unrolling plus SLP.

Figure 4-16 shows the PolyBench results. PolyBench comes with a configurable dataset size; for our experiments, we use the *large* dataset size. Our vectorizer yields a $1.46\times$ average speedup over LLVM’s vectorizers (Loop + SLP) and a $1.80\times$ average speedup over LLVM’s scalar baseline (–O3 without vectorization).

Because PolyBench is a benchmark for polyhedral optimizers, we also compared our approach with Polly [Grosser et al., 2012], a polyhedral optimizer for LLVM IR. Polly performs aggressive cache optimizations and vectorization, achieving significant speedup over LLVM (-O3). To separate the effects of Polly’s cache optimization from its vectorization, we run Polly in three settings: Polly with LLVM vectorization, Polly with its own vectorization, and Polly with our vectorization. Figure 3-15 shows the comparison against Polly. Compared to LLVM’s full optimizations (-O3 with vectorization), Polly’s cache optimizations gets a $1.41\times$ average speedup; Polly’s vectorization gains a $1.49\times$ average speedup ($1.06\times$ over Polly’s cache optimization); and Polly combined with our vectorizer obtains a $1.82\times$ average speedup ($1.22\times$ over Polly’s cache optimization).

3.5.3 ISPC

Pharr and Mark [2012] originally developed their benchmarks to motivate the need for `ispc`, a data-parallel programming model and language that they proposed to target applications with complex data and control dependences (e.g., graphics simulations) unsuitable for auto-vectorization. For all the ISPC benchmarks, we use their baseline, scalar implementations; in other words, the benchmarks have sequential semantics, and the vectorizers must uncover the parallelism from scratch. Figure 3-17 shows some code snippets from Volum Rendering, one of the benchmarks. Volume Rendering consists of five imperfectly nested loops with early exits.

Figure 3-18 shows the results on the `ispc` benchmarks [Pharr and Mark, 2012]. Our vectorizer yields a $1.88\times$ average speedup over LLVM’s vectorizers (Loop + SLP) and a $2.43\times$ average speedup over LLVM’s scalar baseline (-O3 without vectorization).

For the ISPC benchmarks, we extended SuperVectorization to recognize and vectorize the random number generator `drand48` (used by `aobench`). Without the modification, the vectorizer cannot reorder different calls to `drand48` and does not vectorize `aobench`. Nonetheless, `drand48` is not the sole reason that LLVM does not vectorize. In a separate experiment, we modify `aobench` to use a compile-time constant instead of calling `drand48`. Even in this setting, LLVM’s vectorizer does not vectorize,

whereas our vectorizer reproduces the speedup.

The only ISPC benchmarks that our vectorizer does not vectorize is Ray Tracer. Ray Tracer reuses a single array across computations that are otherwise logically independent. Consequently, our dependence analysis concludes that all reads and writes to the array are dependent, thus preventing vectorization.

To analyze the source of the speedups, we collected hardware performance counters. Figure 3-19 shows the reduction of dynamically executed instructions for the ISPC benchmarks. The speedups correlate with the reduction in the number of instructions executed at runtime. We observed similar correlations in TSVC and PolyBench. In some PolyBench benchmarks such as *2mm*, not only did our vectorizer reduce the number of instructions executed but also reduced L1 data cache miss rates from 60% to 40%.

3.6 Related Work

There has been extensive work on compiler auto-vectorization. We list some of the key developments here.

Loop-based vectorization. Allen and Kennedy [1987] pioneered loop vectorization. Their seminal paper also introduced applying if-conversion to vectorize control flow. Nuzman et al. [2006] extended the traditional loop-based vectorization algorithm to support interleaved data accesses. Bagsorkhi et al. [2016] proposed an architectural extension and a joint vectorization for algorithm detecting and speculating over data dependences at runtime. Nuzman and Zaks [2008] proposed an outer-loop vectorization approach based on unroll-and-jam.

SLP vectorization. Larsen and Amarasinghe [2000] developed SLP vectorization as an alternative to traditional loop-based vectorization. SLP vectorization targets short-vector parallelism and requires a much simpler dependence analysis in comparison to traditional loop vectorizers. Almost all subsequent work following Larsen and Amarasinghe [2000] proposed algorithmic improvements to the original algorithm to

uncover more parallelism. Examples include Holistic SLP vectorization [Liu et al., 2012], Super-node SLP [Porpodas et al., 2019], TSLP [Porpodas and Jones, 2015], PSLP [Porpodas et al., 2015], VW-SLP [Porpodas et al., 2018], and applying integer linear programming (ILP) solver to select vector packs [Mendis and Amarasinghe, 2018]. Shin et al. [2005] extended SLP to vectorize control flow with if-conversion.

Compilation for data-parallel languages. There is a related line of work on compiling data-parallel languages for vector architectures. In contrast to auto-vectorization, compilers for such language do not need to extract vector parallelism or prove its safety because their source semantics are already data-parallel. OpenCL provides a data-parallel programming model that maps to SIMD extensions [Group, 2009]. Karrenberg and Hack [2011] proposed a set of techniques for vectorizing arbitrary reducible CFGs in a data-parallel language. Moll and Hack [2018] proposed techniques for recovering control flow from if-conversion, thereby reducing the amount of speculative computation and improving utilization. Pharr and Mark [2012] proposed `ispc`, a C-like data-parallel language that targets SIMD extensions.

Compiler Intermediate Representation. There has been extensive work towards developing IRs to simplify the downstream compiler optimizations. Cytron et al. [1991] developed a seminal algorithm for efficiently computing SSA form, which is now used by production compilers such as GCC and LLVM. Ottenstein et al. [1990] originally proposed gated SSA as an IR for their Fortran compiler that targets dynamic data-flow architectures. Tu and Padua [1995] developed an algorithm for efficiently computing gated SSA. Tristan et al. [2011] used gated SSA for translation validation.

3.7 Summary

We have introduced a new vectorization framework that targets arbitrary superword-level parallelism that spans different basic blocks and loops. With loop unrolling, our vectorizer matches and, in many cases, outperforms LLVM’s vectorization pipeline, which uses both loop and SLP vectorizers. For example, our vectorizer vectorizes

and accelerates several of the benchmarks that Pharr and Mark [2012] argued as being unsuitable for automatic vectorization due to their complex control and data dependences. Our vision for the future is that a single vectorizer based on our framework would be sufficient for a vectorizing compiler.

```

1 def lower(func_or_loop, entry):
2     eliminate_phis(func_or_loop)
3
4     # Utility structure to create basic blocks
5     # from control predicates
6     block_builder = BlockBuilder(entry)
7     if func_or_loop.is_loop():
8         # Allocate dedicated latch and exit blocks
9         latch = BasicBlock()
10        exit = BasicBlock()
11        # the header executes unconditionally
12        header = block_builder.get_block(true)
13
14    for item in func_or_loop.items:
15        pred = func_or_loop.control_pred_of(item)
16        b = block_builder.get_block(pred)
17        if item.is_loop():
18            loop_exit = lower(item.as_loop(), entry=b)
19        else:
20            item.as_inst().move_to(b)
21
22    if func_or_loop.is_loop():
23        loop = func_or_loop.as_loop()
24
25        # finally convert the mu-nodes back to phi-nodes
26        for mu in loop.mus:
27            phi = Phi(
28                incoming_blocks=[entry, latch],
29                incoming_values=[mu.init, mu.rec],
30                insert_at=header)
31            replaceAllUsesWith(mu, phi)
32
33        # Allocate a dedicate, flag variable,
34        # indicating if we should continue
35        should_continue = Alloca(type=bool, insert_at=header)
36        # Set the flag to false
37        assign_false(should_continue, insert_at=header)
38
39        # Create a block that
40        # has the same predicate as the continue predicate
41        temp_block = block_builder.get_block(loop.cont_pred)
42        # and set the continue flag there
43        assign_true(should_continue, insert_at=temp_block)
44
45        # Create a block that executes unconditionally
46        pre_latch = block_builder.get_block(true)
47        # and load the continue flag
48        cont = create_load(should_continue,
49                           insert_at=pre_latch)
50        # Then exit or continue depending on the flag
51        create_branch(cond=cont,
52                      if_true=latch, if_false=exit,
53                      insert_at=pre_latch)
54
55    restore_ssa()
56    return None

```

Figure 3-12: Algorithm for lowering Predicated SSA to a traditional IR with CFG (e.g., LLVM IR)

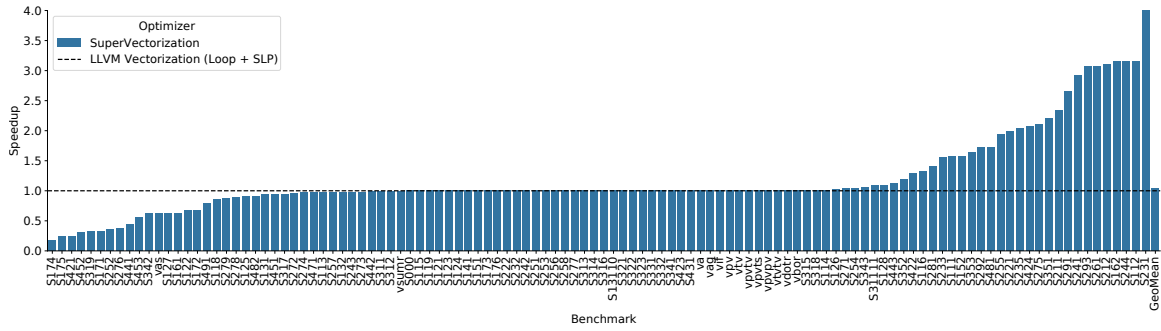


Figure 3-13: Speedup over LLVM scalar optimizations (-O3) on TSVC. The benchmarks are sorted (in increasing order) according to speedup from SuperVectorization over LLVM’s scalar optimizations.

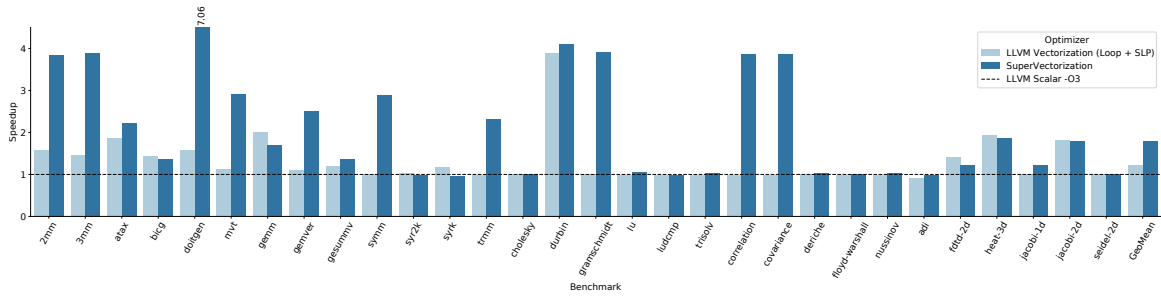


Figure 3-14: Speedup over LLVM scalar optimizations (-O3) on PolyBench

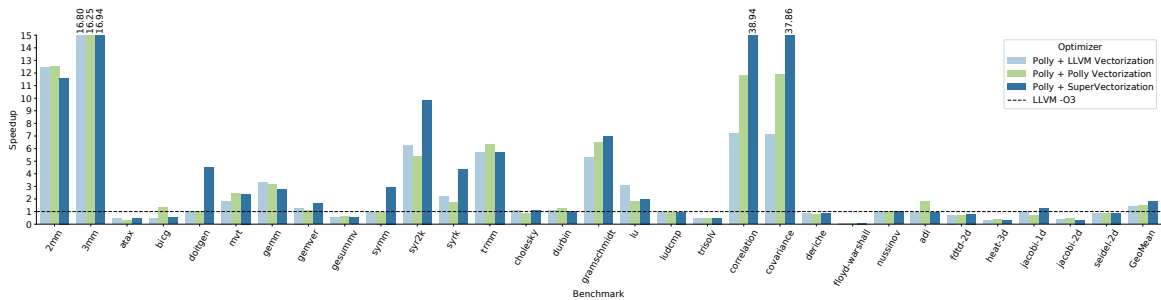


Figure 3-15: Speedup over full LLVM optimizations (-O3 + vectorization) on PolyBench using Polly

```

for (int i = 0; i < LEN2; i++)
    if (aa[0][i] > (float)0.)
        for (int j = 1; j < LEN2; j++)
            aa[j][i] = aa[j-1][i] + bb[j][i] * cc[j][i];

```

Figure 3-16: An example benchmark from TSVC where our vectorizer gains a $2.91\times$ speedup over LLVM's vectorizers by unrolling the outer i -loop and packing instructions across the inner j -loops and their loop guards.

```

...
static float transmittance(...) {
    ...
    if (!IntersectP(ray, pMin, pMax, &rayT0, &rayT1))
        return 1.;
    ...
    while (t < rayT1) {
        tau += ...
        pos = pos + dirStep;
        t += stepT;
    }
    ...
    return expf(-tau);
}

static float raymarch(...) {
    ...
    if (!IntersectP(ray, pMin, pMax, &rayT0, &rayT1))
        return 0.;
    ...
    while (t < rayT1) {
        ...
        // terminate once attenuation is high
        float atten = expf(-tau);
        if (atten < .005f)
            break;

        // direct lighting
        float Li = ... / transmittance(...)
        ...
    }

    // Gamma correction
    return powf(L, 1.f / 2.2f);
}

void volume_serial(...) {
    int offset = 0;
    for (int y = 0; y < height; ++y) {
        for (int x = 0; x < width; ++x, ++offset) {
            Ray ray;
            generateRay(..., ray);
            image[offset] = raymarch(density, nVoxels, ray);
        }
    }
}
}

```

Figure 3-17: Code snippets from Volume Rendering, which contains five imperfectly nested loops with early exits and multiple live-outs. Our vectorizer achieves a 3.28× speedup by restructuring these to a loop. No optimization could be applied to this

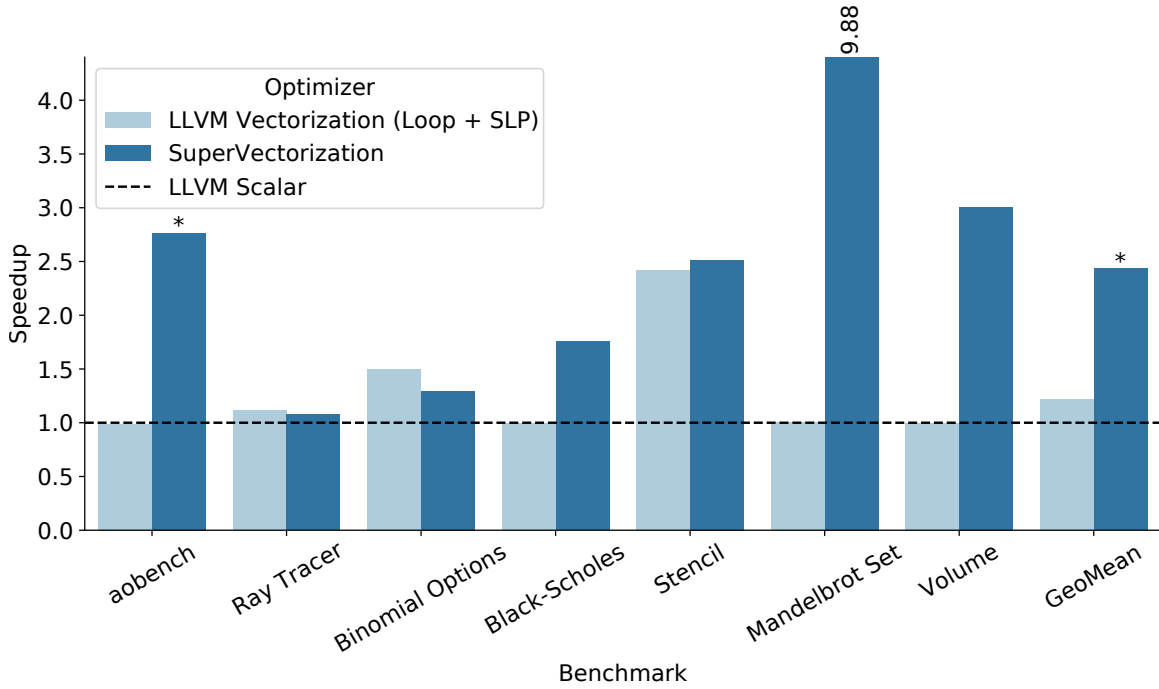


Figure 3-18: Speedup over LLVM vectorization (Loop + SLP) on the ISPC benchmarks. **aobench** (marked with *) uses the random number generator **drand48**; We modified SuperVectorization to recognize **drand48** and to assume that it is safe to reorder different calls to **drand48**. Without this modification, **aobench** is not vectorizable, and the geomean speedup becomes $2.09\times$ (instead of $2.43\times$).

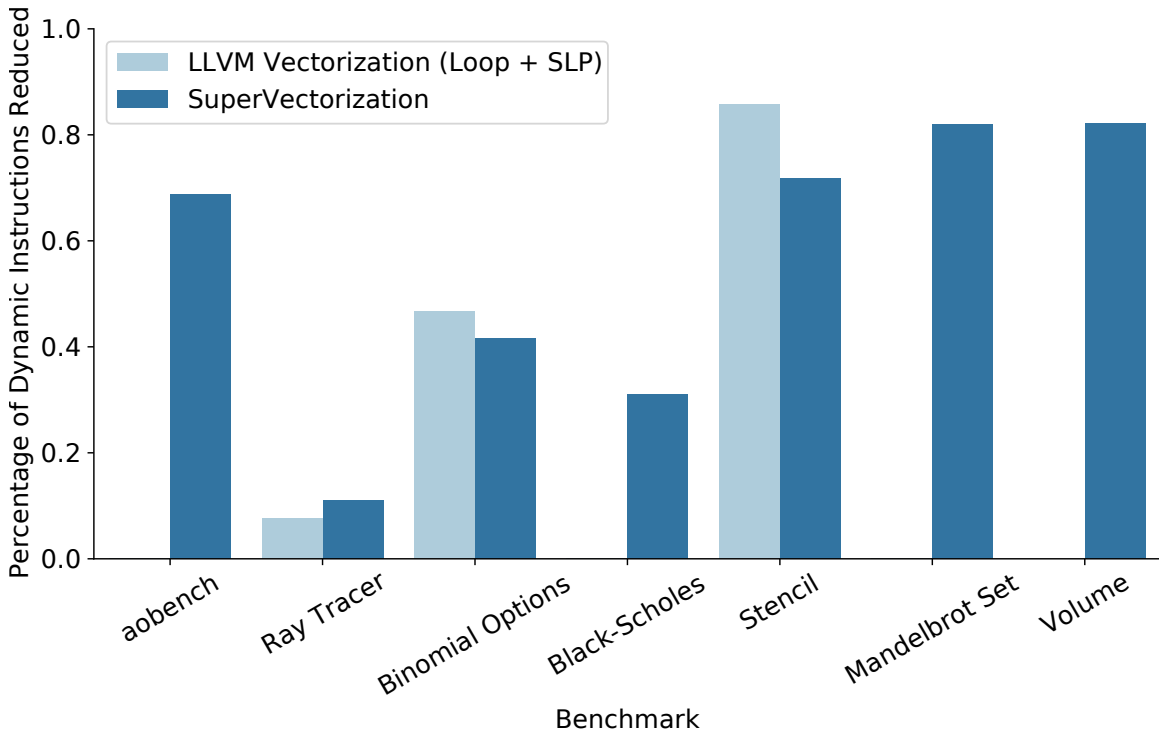


Figure 3-19: Reduction in dynamic instruction counts in the ISPC benchmarks

Chapter 4

Fine-grained program versioning for SLP

4.1 Introduction

Dependence analysis is crucial for many compiler transformations, such as loop-invariant code motion, vectorization, and, more generally, any optimizations that perform code motion. However, performing precise dependence analysis is difficult. For example, in a language such as C, the compiler must assume that arbitrary pointer arguments may alias and that all memory accesses on those pointers are dependent. This scenario can be addressed, to some degree, with interprocedural pointer analyses. However, such analyses are prohibitively expensive and not deployed, by default, in production compilers. Furthermore, dependence analysis is complicated by nontrivial pointer arithmetic and control flow, both of which can induce patterns that are impossible to analyze soundly at compile time, even with state-of-the-art analysis. For example, two indirect pointer accesses could be both dependent and independent, depending on how the input is initialized. All of the aforementioned factors contribute to imprecise dependence analysis, making optimizations less effective as a consequence.

Loop Versioning. When there is insufficient dependence information, production

compilers use a technique called *loop versioning*. With loop versioning, the optimizer makes two copies of a loop—one loop is optimized, assuming there are no dependencies among instructions critical to the optimization, and the other is left unoptimized—and generates run-time checks so that the optimized loop is only entered if the absence of these dependencies is confirmed. Because the run-time checks must be performed upfront before the loop is executed, loop versioning only works if all the requisite information is known before the loop is executed.

Consequently, loop versioning cannot rule out dependences that are loop-variant. For instance, complex pointer arithmetic can lead to some memory accesses that are dependent in some iterations yet independent in others. Similarly, loop versioning cannot rule out rare dependences that are caused by conditionally executed instructions. To deal with such dependences in the context of vectorization, researchers have proposed an approach that performs run-time checks that are more fine-grained [Sujon et al., 2013, Liu et al., 2023]. In this approach, the vectorizer first identifies some specific control-flow paths along which the loop iterations are completely independent; they then optimize the program assuming only those paths are taken, while also generating the necessary recovery code in case the paths are not taken. Unlike loop versioning, this approach is tied specifically to vectorization and only works in the presence of loops.

Our approach. In this paper, we propose a fine-grained versioning framework to rule out program dependences at run time. Our framework is general and not coupled with specific optimizations. To use our system, a client compiler optimization supplies groups of instructions (or loops) whose independence is desired but cannot be proven statically. In response, our system identifies a small—and in many cases optimal—set of conditions to ensure the independence of those instructions and versions accordingly. Our framework is fine-grained because it tailors a versioning strategy for each group of instructions (or loops) individually, without having to version whole loops at a time. This is particularly useful when some dependence relationships change from loop iteration to iteration due to nontrivial uses of pointer arithmetic. Furthermore, our framework supports *nested* versioning, which is necessary when a versioning

transformation itself relies on assumptions that cannot be statically proven, thus requiring a further round of versioning to confirm those assumptions at run time.

Central to our framework is the concept of *dependence condition*, which identifies the necessary condition under which a dependence occurs at run time. While most dependences are unconditional, such as when arithmetic instructions depend on their operands unconditionally, others are conditional. Examples include when a pair of memory accesses being only dependent if the accessed memory locations overlap or when a load only depending on a preceding conditional store if the store actually executes at run time. Our insight is that when dealing with instructions whose independence cannot be statically proven due to *conditional* dependences, we can establish a sufficient condition for their independence by finding a cut over the dependence graph that makes the instructions unreachable from one another. This cut reveals the conditional dependences that we must check at run time.

To demonstrate the effectiveness of our framework, we first applied our versioning framework to an existing implementation of superword-level parallelism (SLP) vectorization. Although production loop vectorizers have used loop versioning, there has been no implementation of SLP vectorization capable of versioning because it is more flexible and not loop-centric. Consequently, SLP vectorization has been, to an even greater degree when compared to loop vectorization, limited by pointer aliasing and other sources of program dependences.

We adapted Chen et al. [Chen et al., 2022]’s SLP vectorizer—an aggressive SLP implementation that can pack independent instructions globally across loops and basic blocks—to use our versioning framework. With versioning, this single implementation of SLP is $1.17\times$ faster than both LLVM’s loop and SLP vectorizers combined on the TSVC benchmark (and $1.07\times$ faster than their previous implementation). On PolyBench with the `restrict` keyword disabled, versioning enabled a $1.51\times$ speedup over both LLVM’s vectorizers (and $1.62\times$ faster than their previous implementation). In both benchmark suites, we found examples of programs that cannot be vectorized with loop versioning alone. We obtained these results without changing *any* of the vectorizer’s heuristic or code-generation algorithm: we only modified the vectorizer to

inform our framework of the groups of instructions that should be independent, and it versions the program transparently.

Finally, we used our framework to implement a variant of redundant load elimination that removes redundant loads that cannot be proven redundant with traditional static analysis. With versioning, our implementation of redundant load elimination achieves a geometric mean speedup of 1.2% on the SPEC 2017 Floating Point benchmark suite, with the maximum speedup being 6.4%.

4.2 Running Example

Our goal is to develop a general framework for using versioning to check program dependences at run time. To this end, we begin by reviewing some background to motivate our approach.

4.2.1 Dependence Analysis

Dependence analysis is an essential component of many classic compiler optimizations. A quintessential example is vectorization, where the vectorizer must prove that the original scalar instructions are independent before replacing them with vector instructions. There are two types of program dependences: (1) Use-def dependence, which occurs when an instruction uses the result of another instruction as an operand, and (2) memory dependence, which happens when two memory instructions access the same memory locations, and at least one of the them is a write. Dependence is a transitive relation, and it is common for two instructions without a direct dependence to still be dependent due to some intermediate dependences. Typically, a dependence analysis models program dependences with a *dependence graph*, where instructions are represented as nodes, and direct dependences between instructions are represented as edges. Two instructions are independent if there is no path connecting them in the dependence graph. Dependence analysis is complicated by pointer aliasing, which happens when two distinct pointers may point to the same memory location. Without precise aliasing information, because without it, the dependence analysis has to be

conservative and assume that memory instructions using different pointers as dependent. Alias analysis is often imprecise due to factors such as function calls, complex control flow, or pointer arithmetic.

Versioning. To deal with imprecise aliasing information, production compilers sometimes use loop versioning to disambiguate pointers at run time. With versioning, the compiler makes two copies of a loop—one loop is optimized assuming some desired but unproven aliasing relationships among some memory locations, and the other left unoptimized—and modify the control flow so that the optimized loop is entered at run time only if the assumed aliasing relationships are confirmed. Loop versioning comes with its own costs; it increases code size and incurs run-time overhead with the extra aliasing checks and branches. Usually, the optimizer uses a cost model to determine whether the benefits of optimization outweigh the costs of versioning.

Speculative Vectorization. In the context of vectorization, a related transformation is called speculative loop vectorization [Sujon et al., 2013, Liu et al., 2023]. Speculative vectorization is useful when the loop iterations are independent only if some paths in the control-flow graph (CFG) are executed. In this case, the speculative vectorizer then identifies these CFG paths and the associated conditions under which these paths are taken. It then changes the control flow within the loop to ensure that the vectorized code only executes when all those conditions are met. Additionally, the vectorizer needs to generate fallback code to handle situations when the conditions are not met.

Unlike versioning, which is a general optimization separate from loop vectorization, speculation in this context is done specifically for vectorization. Nonetheless, there are many similarities between versioning and speculation. Both approaches address, in different ways, the problem of dependences that cannot be ruled out at compile time. They both duplicate code so that vector execution happens only under some predetermined conditions. Despite their similarities, they have typically been viewed as two distinct transformations, and there is no unifying framework that combine both.

```

Y[0] = 0;
if (*X) cold_func();
Y[1] = 0;

```

Figure 4-1: Running example for fine-grained versioning. The highlighted instructions may be dependent and cannot be directly vectorized.

```

noalias = X != Y;
if (noalias) {
    x = *X;
    if (x) Y[0] = 0;
    else {
        Y[0] = 0;
        Y[1] = 0;
    }
} else {
    Y[0] = 0;
    x = *X;
}
if (x) cold_func();
if (!noalias || x) Y[1] = 0;

```

Figure 4-2: Running example transformed so that the stores become independent (and amenable to vectorization).

4.2.2 Motivating Example

Figure 4-1 shows a snippet of a C function that will serve as a running example throughout the paper. Suppose we want to optimize the program by replacing the stores to the array `Y` with a single vector store. This is only possible if the two stores are independent; however, any sound static dependence analysis will conclude that the stores are dependent for the following reasons. First, there is a dependence chain from the last store (`Y[1] = 0`) to the load (`*X`) because the pointers `X` and `Y` may alias, and then to the first store (`Y[0] = 0`). Second, the function call also prevents vectorization because it may read and write arbitrary memory.

Figure 4-2 shows a program equivalent to the running example. We can verify that the programs are equivalent with a case analysis of the two `if` conditions and the resulting traces of operations. Although the two programs have the same semantics, there is one key difference. In the rewritten program, the stores we are interested in are independent. Chapter 4.3 discusses how our framework automatically performs this transformation for general programs to ensure that the specified sets of instructions are independent.

$$\begin{aligned}
fn &::= item_1 : p_1, \dots, item_n : p_n \\
item &::= \mathbf{instruction} \mid loop \\
loop &::= \mathbf{with} \ v_1 = \mu_1, \dots, v_m = \mu_m \ \mathbf{do} \\
&\quad item_1 : p_1, \dots, item_n : p_n \\
&\quad \mathbf{while} \ p_{cont} \\
\mu &::= \mathbf{mu}(v_{init}, v_{rec}) \\
\phi &::= \mathbf{phi}(v_1 : p_1, \dots, v_n : p_n) \\
p &::= \mathbf{true} \mid v \mid \bar{v} \mid p_1 \wedge p_2 \mid p_1 \vee p_2 \\
v &::= \mu \mid \mathbf{instructions} \mid \mathbf{constant} \mid \mathbf{argument}
\end{aligned}$$

Figure 4-3: Definition of predicated SSA.

```

store [Y + 0], 0    ; true
x = load X          ; true
c = cmp ne x, 0     ; true
call cold_func      ; c
store [Y + 1], 1    ; true

```

Figure 4-4: The running example from Figure 4-1 represented in predicated SSA

4.3 Approach

We first present the general workflow of our versioning framework. First, during the planning stage, the client optimization provides us with a set of instructions (or loops) that are possibly dependent. In response, our system infers *a versioning plan* that ensures their independence. Each versioning plan outlines a set of instructions (and loops) that require versioning and specifies the set of conditions under which versioning should take place. The client can make multiple requests to make different sets of instructions (or loops) independent, which can result in multiple versioning plans. Finally, the client invokes a separate routine to *materialize* the inferred versioning plans. Afterward, the optimization may use the instruction dependence guaranteed by the plans to perform its transformations.

Control-flow code motion and predicated SSA. Control-flow code motion (i.e., moving instructions across basic blocks and branches) is a basic operation that our versioning framework relies on. To simplify this operation, we first convert the input program to predicated SSA form [Chen et al., 2022]. Figure 4-3 shows the definition of predicated SSA. In predicated SSA, each instruction (or loop) is annotated with a

predicate indicating whether the instruction should be executed. With these predicates, the IR no longer uses branches or basic blocks and instead models the program as a hierarchy of nested loops, each of which contains a flat list of member instructions and loops. This representation simplifies global code motion because individual instructions and loops are no longer tied to any specific basic blocks and can be moved without affecting correctness (provided no dependences are violated). Figure 4-4 shows the program represented in predicated SSA form.

4.3.1 Dependence Condition

The traditional notion of program dependence can be too restrictive and imprecise for complex, real-world programs. For example, if the alias analysis cannot soundly disambiguate the pointers used by a pair of memory accesses (assuming at least one is a write), a traditional dependence analysis simply concludes that there is a dependence. However, this dependence only exists at run time if the accessed memory locations actually intersect. For this reason, we propose a more refined approach, in which program dependence is labeled with the necessary condition for the dependence to occur at run time. We refer to this condition as the *dependence condition*. Figure 4-5 shows the definition of dependence conditions, which formalizes the intuition that a dependence exists either due to some predicate p (e.g., an instruction i depends on another instruction j only if j is executed) or on the intersection of two memory ranges (e.g., two memory writes are dependent if they alias).

Given two instructions i and j from the dependence graph, we use the notation $c(i, j)$ to represent the dependence condition for i to directly depend on j . Figure 4-6 shows the algorithm calculating $c(i, j)$. We assume the input program is represented in

$$\begin{aligned}
 c &::= p \mid \textit{intersection} \mid c_1 \vee c_2 \\
 \textit{intersection} &::= \mathbf{intersects}([m_1, m_2], [m_3, m_4]) \\
 p &::= \mathbf{control\ predicate} \\
 m &::= \mathbf{memory\ address}
 \end{aligned}$$

Figure 4-5: Definition of dependence condition.

```

def c(i, j):
    # return the condition for when i depends on j directly
    if is_phi(i):
        for op_pred, op in cast_to_phi(i).operands():
            if op == j:
                return op_pred
        return Predicate.false()
    if is_select(i):
        select = cast_to_select(i)
        if select.true_val == j:
            return pred(i).and(select.cond)
        if select.false_val == j:
            return pred(i).and(select.cond, negate=True)
        return Predicate.false()
    if i in j.users():
        return Predicate.true()
    if not i.may_write() and not j.may_write():
        return Predicate.false()
    if i.is_instruction() and j.is_instruction():
        # If j executes at a more specific predicate than i
        if pred(j).implies(pred(i)) and pred(j) != pred(i):
            return pred(j)
        return Intersects(mem_location(i), mem_location(j))
    # deal with cases when either i or j is a loop
    insts1 = [i.mem_instructions()] if i.is_loop() else [i]
    insts2 = [j.mem_instructions()] if j.is_loop() else [j]
    conds = []
    for i1 in insts1:
        for i2 in insts2:
            if i1.may_write() or i2.may_write():
                conds.append(c(i1, i2))
    return Or(conds)

```

Figure 4-6: Algorithm for determining the dependence condition between two dependence nodes.

Predicated SSA and use $pred(i)$ to denote the predicate associated with the instruction i . Our framework identifies dependence conditions as follows:

- A ϕ node $t = \text{phi } (x_1: p_1, \dots, x_n : p_n)$ only depends on the operand x_i if p_i is true at run time; i.e., $c(t, x_i) = p_i$.
- A select instruction $t = \text{select } x, a, b$ depends on a if x is true; i.e., $c(t, a) = pred(a) \wedge x$.
- In other cases where an instruction i uses the result of another instruction j as an operand, then i depends on j unconditionally.
- A necessary condition for an instruction i to depend on j is if j is ever executed. Therefore, when we detect that j executes at a more specific predicate than j , we return j 's predicate as the dependence condition.
- Two memory accesses (assuming at least one is a write) are dependent if their memory locations intersect.
- Assuming there are no use-def dependences, two loops l_1 and l_2 are dependent if any of their member instructions are dependent; i.e., $c(l_1, l_2) = \bigvee_{i \in insts(l_1), j \in insts(l_2)} c(i, j)$. We define the dependences between a loop and an instruction similarly.

For any two instructions i and j , we refer to the dependence $i \rightarrow j$ as an *unconditional dependence* if $c(i, j) = true$; otherwise, we will call it a *conditional dependence*.

For some of the dependence conditions, our system eventually needs to generate run-time checks to verify that those conditions are indeed true. In this sense, a dependence condition implies a computation that, in turn, has its own dependences. For instance, if a dependence condition contains intersection checks, then the dependence condition itself unconditionally depends on the memory addresses involved in the intersections.

Graph cut and versioning. Figure 4-7 shows the dependence graph of the running example, where the conditional dependences are represented as dotted edges and unconditional dependences are represented as solid edges. Note that we can make the two stores independent (i.e., unreachable from one another) by eliminating the

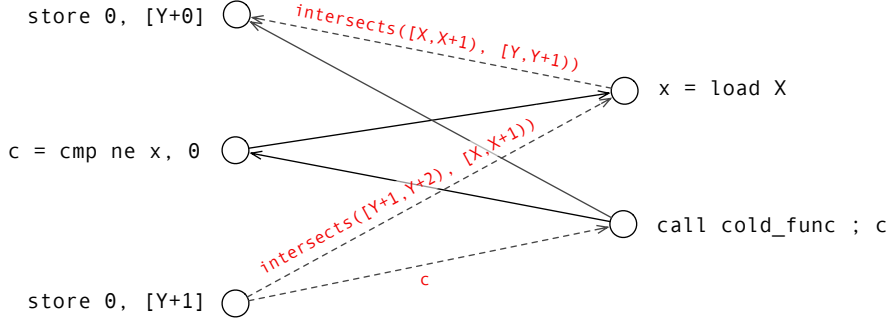


Figure 4-7: Dependence graph of the running example. Conditional dependences are represented as dotted edges and labelled with their dependence conditions. The function call depends unconditionally on the first store because the store has a stronger predicate (i.e., the execution of the call implies that of the store but not vice versa).

following conditional edges: (1) the conditional dependence from the load to the first store and (2) the dependence from the last store to the function call. This is a core operation that we use extensively throughout our framework.

More generally, given two sets of dependence graph nodes S and T (*which may intersect*), we need to identify a set of *conditional dependences* whose removal makes S independent of T . This is equivalent to finding a *cut* over the dependence graph so that nodes in T are not reachable from S^1 , with the *cut-set*—edges crossing the partitioned sets—exclusively containing edges for conditional dependences. To optimize for performance, our algorithm also reduces the cut-set because each edge in the cut-set represents a dependence condition that must be eventually checked at run time. A smaller cut-set typically leads to better performance. If successful, the algorithm returns two sets: (1) the cut-set and (2) the source side of the cut that can reach T . Note that, if the S and T are already independent, the algorithm returns two empty sets.

We find the cut with a reduction to min-cut. The reduction works by constructing an auxiliary flow graph such that there is a flow from the source to sink if and only if any of the nodes in S depends on T . A min-cut of the flow graph reveals the set of dependence edges whose removal would make the input instructions independent. To create this flow graph, we follow these steps:

¹Because S and T may intersect, we ignore trivial reachabilities of the form $s \rightarrow s$, where $s \in S \cap T$.

1. We start with a depth-first search (DFS) to discover the dependences of nodes in S , considering both conditional and unconditional dependences.
2. We split each node discovered by the DFS into an in-node and an out-node. We then duplicate the dependence edges such that each dependence flows into an in-node and out of an out-node, with each pair of in- and out-nodes connected with a single auxiliary edge. Splitting the nodes is important because otherwise the sink will always be reachable from the source even when all the conditional edges are cut.
3. Next, we connect the source node to all of the out-nodes of S and all of the in-nodes of T to the sink node.
4. Finally, we set the capacity for each conditional edge to 1 and all the other edges to $n + 1$, where n is the total number of unconditional edges.

With the capacity thus set, if the min-cut is greater than n , we must disconnect at least one unconditional dependence edge to make the nodes independent; and versioning is infeasible. If the min-cut is less than n , then versioning is feasible, and we set the versioning conditions—which we speculate are all false—as the union of the dependence conditions of the edges in the cut-set. With profile information, we can also set the capacity of each conditional edge as the likelihood of the corresponding dependence occurring at run time, thereby maximizing the pass rate of the run-time checks.

Figure 4-8 shows the flow graph generated for the running example. Note that without splitting the nodes into pairs of in-nodes and out-nodes, the sink is always reachable from the source even if all conditional edges are cut, implying (wrongly) that versioning is infeasible.

4.3.2 Versioning by Example

We now discuss how our framework uses the aforementioned algorithm for finding dependence graph cuts as a subroutine to version the running example (Figure 4-

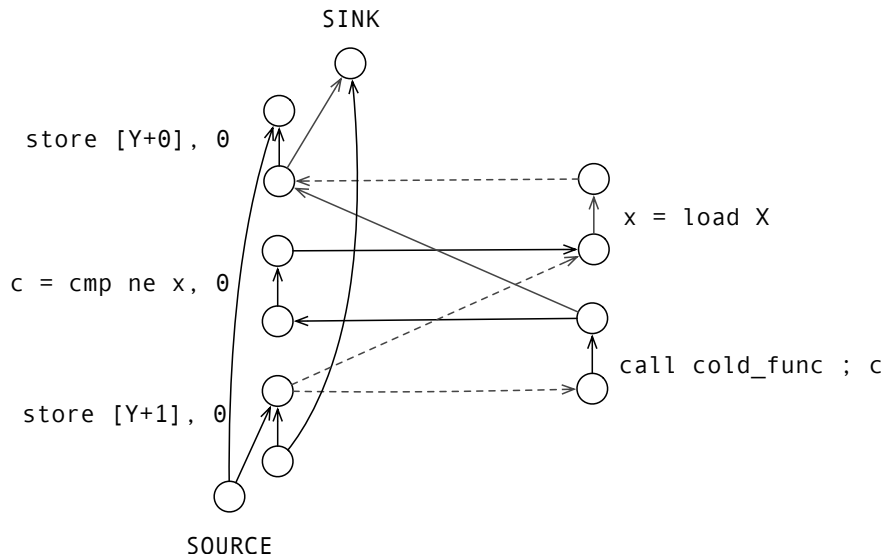


Figure 4-8: The flow graph constructed for the running example to separate the two stores (in the original dependence graph). Dotted edges are conditional and have capacities of 1, while solid edges are unconditional and have capacities of 4.

4), using step-by-step examples. Chapter 4.3.3 discusses the general algorithm for versioning.

Versioning is more intricate than just finding a cut in the dependence graph. It might be tempting to version the program directly given a cut by duplicating nodes on one side of the cut and using the dependence conditions from the cut-set as the versioning conditions. Nevertheless, this approach is generally unsafe because materializing a versioning plan involves generating code to check that none of the dependence conditions are true. These checks themselves are computations that can depend on the nodes being versioned. When these checks cannot be computed because they depend on the code under versioning, we need to identify a *secondary* versioning plan, the goal of which is to ensure that the initial set of versioning conditions is independent of the code that was intended to be versioned in the first place. We will refer to the initial versioning plan as the *primary* versioning plan, and such a collection of plans—primary, secondary, etc—constitute a single *nested* versioning plan.

The running example (Figure 4-4) is one such example that requires a nested versioning plan. To start, we aim to make the two stores independent, which entails

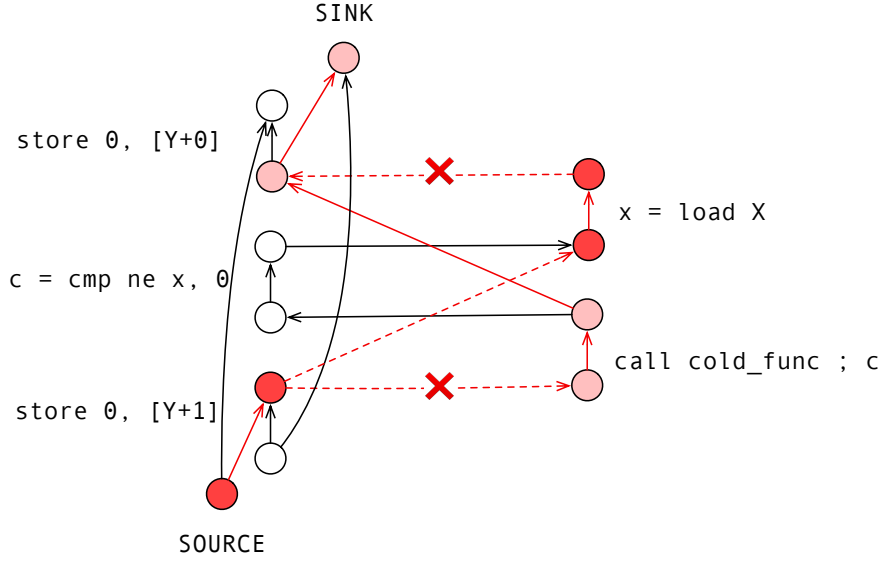


Figure 4-9: A primary cut to separate the two stores in the dependence graph. Nodes and edges with positive flow are highlighted in (different shades of) red. ● represents a node in the source-side cut. ● represents a node in the sink-side cut. Edges in the cut-set are marked with ✗.

finding a cut that separates the two stores. Figure 4-9 shows the flow graph and the min-cut. The cut has the following dependence edges in its cut-set (marked with ✗):

- The dependence from the second store (`store [Y+1], 0`) to the function call, which has the dependence condition c ; i.e., the dependence only exists if c is true.
- The dependence from the load (`x = load X`) to the first store (`store [Y+0], 0`), which has the dependence condition $\text{intersects}([X, X + 1), [Y + 0, Y + 1))$; i.e., the dependence only exists if the addresses X and $Y+1$ alias.

We focus on the conditional dependence involving the conditional function call. This dependence has a dependence condition of c , which itself depends (transitively) on the first store (because X may alias with Y). Because we must be able to evaluate c before both of the stores in order to materialize this primary plan, the plan is not materializable. More generally, a plan is materializable if its versioning conditions

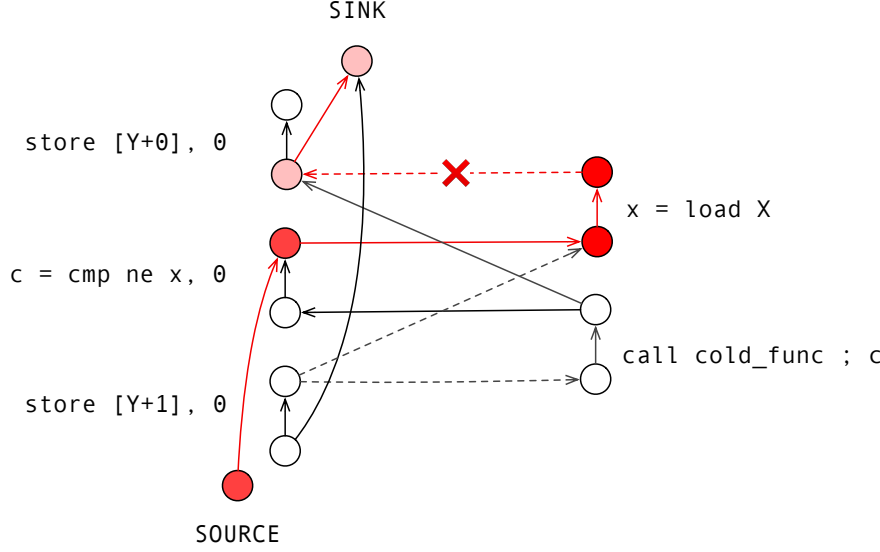


Figure 4-10: A secondary cut to make the comparison (c) independent of the stores.

are independent of its versioned nodes. Therefore, we proceed to infer a secondary versioning plan that makes c independent of the stores.

Figure 4-10 shows the secondary cut. The cut has exactly one edge in the cut-set, the dependence edge from the load to the first store ($x = \text{load } X \rightarrow \text{store } [Y+0], 0$), which has the dependence condition $\text{intersects}([X, X + 1], [Y + 0, Y + 1])$. We do not need a tertiary plan here because this dependence condition only depends on the pointers X and Y but no other instructions.

Given the secondary plan, we finish by updating the initial flow graph constructed for the primary plan. The updated flow graph and (primary) cut are shown in Figure 4-11. Note that we delete the dependence edge from the load to the first store ($\text{store } [Y+0], 0$) because this dependence has been eliminated by the secondary plan. Having considered the removal of this dependence, we update the source-side cut to include only the second store ($\text{store } [Y+1], 0$). Figure 4-12 shows the final versioning plan—constructed based on the cuts we just computed—making the two stores independent. Essentially, for a given cut, we perform versioning on stores as well as any nodes on the source side that can reach the stores.

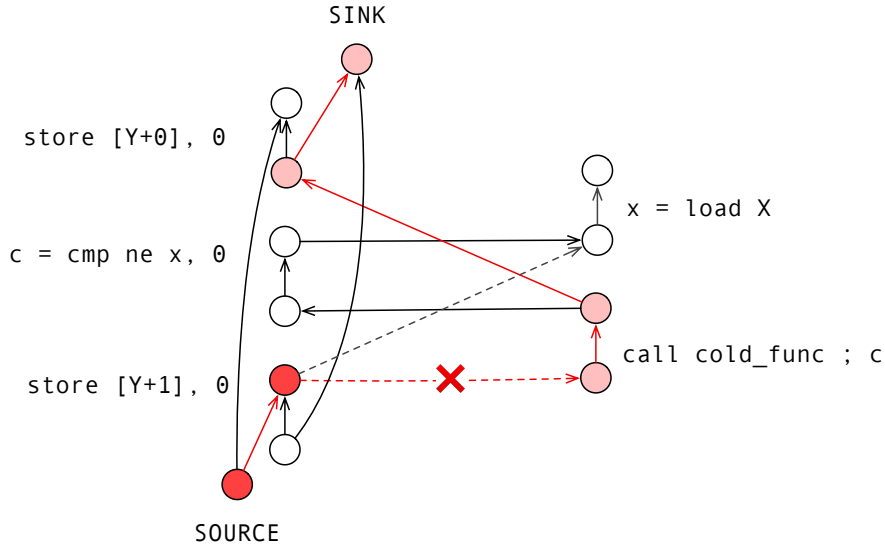


Figure 4-11: The primary cut from Figure 4-9 updated to reflect that the dependence from the load to the first store is removed by the secondary cut (Figure 4-10).

```

N = {
  store [Y+0], 0
  store [Y+1], 0
}
C = {c}
V' = (N' = {
  x = load X
  c = cmp ne x, 0
  store [Y+0], 0
  store [Y+1], 0
}
  C' = {intersects([X,X+1), [Y,Y+1))})

```

Figure 4-12: The final nested versioning plan derived from the cuts shown in Figure 4-10 and Figure 4-11. N (and N') are the nodes that will be versioned and C (and C') are the conditions under which they will be versioned.

4.3.3 Inferring Versioning Plans

We are now ready to formalize the example procedure for inferring a versioning plan given a set of instructions that we wish to make independent. *Versioning plan* is an abstraction that allows us to represent a subprogram that we want to version and the condition under which it should be executed without explicitly transforming the program. More specifically, a versioning plan is a three-tuple $V = (N, C, V')$, where

- N is the set of dependence graph nodes that we want to version, and a node represents either an instruction or a loop.
- C is the set of versioning conditions. These are the conditions that we will assert to be false at run time, and if any of them is true, execution will proceed to the fallback path.
- V' is an optional *secondary versioning plan*, which becomes necessary if some of the dependence conditions (C) transitively depend on a subset of the versioned nodes (N), introducing circular dependencies that make code generation impossible.

Figure 4-13 shows the algorithm for inferring a nested list of versioning plans so that a given list of instructions is independent, with the initial primary plan ensuring that the input instructions are independent and each subsequent secondary plan ensuring that a previous primary plan is materializable (i.e., ensuring that the dependence conditions of the primary version do not depend on the versioned nodes).

The core algorithm (named `infer_versioning_plans`) is recursive and takes two sets of nodes as input: `nodes` and `input_nodes`. Each recursive call returns a versioning plan that, once materialized, ensures that no node in `nodes` depends on `input_nodes`. The algorithm starts by finding a cut of the dependence graph that separates `nodes` from `input_nodes`. The versioning conditions are set as the union of the conditions of the dependences in the cut-set. If the versioning conditions are all false, then it implies that `nodes` are independent of `input_nodes`.

With this set of versioning conditions, at run time, the transformed program must verify that all of them are false before executing the input instructions (`input_nodes`) as vector instructions. Because of this execution strategy, the versioning conditions must not depend on the `input_nodes`. Otherwise, those conditions cannot be evaluated before `input_nodes`. Consequently, instead of directly constructing a primary versioning plan according to the initial cut, we need to recursively run the algorithm to ensure that the versioning conditions are independent of `input_nodes` (lines 10-20). Before we attempt to find a secondary versioning plan recursively, it is crucial that we check that none of the versioning conditions *directly* uses any of the `input_nodes` (at line 15) because otherwise it would be the case that the versioning conditions always depend on the `input_nodes` *unconditionally*.

With the secondary plan in hand, we then update the cut to take into account the dependence edges that no longer exist due to secondary versioning. Alternatively, we could compute this cut from scratch to account for the dependence edges removed by secondary versioning. Finally, the algorithm constructs the primary versioning plan to version only the source side of the cut that can reach `input_nodes` and the `input_nodes` themselves.

Although the recursive call only constructs a secondary plan guaranteeing that the versioning conditions (of a primary plan) are independent of the `input_nodes`, the secondary plan, thus constructed, also implicitly guarantees that the versioning conditions are independent of all versioned nodes, which is necessary for a plan to be materializable. In other words, if a versioning condition c does not depend on any of the input instructions, then c must also be independent of any node n versioned by the primary plan. We can prove this by contradiction. Suppose, for contradiction, that a versioning condition c depends on such a node n . Recall that n , by definition, is either (1) a node on the source side of the cut that can reach input instructions (`input_nodes`) or (2) one of the input instructions. Either case would imply that there is a path from n to some of the input instructions, leading to the contradiction that c depends on n and, thus, an input instruction.

The versioning plan inference algorithm always terminates. We show this by fixing

a topological ordering for all the nodes in the dependence graph. The original program order is one such ordering, which assigns a unique number, $order(.)$, to each dependence graph node so that if a node s depends on another node t , then $order(s) > order(t)$. Observe that if the following condition is true, then the recursion terminates (because if $order(s) \leq order(t)$, then s cannot depend on t):

$$\max_{s \in \mathbf{nodes}} order(s) \leq \min_{t \in \mathbf{input_nodes}} order(t)$$

It suffices to show that the ordering of the **nodes** is always decreasing. This is true because the *operands* of the versioning conditions—i.e., the nodes that we pass to the recursive call (Line 18)—are always ordered before at least one of the **nodes**. To see why this is true, recall that each versioning condition is the dependence condition of one of the edges in the cut-set. Let $s \rightarrow t$ be an edge in the cut-set and x be an operand of the dependence condition $c(s, t)$. Following the definition of dependence conditions, there are two cases here: (1) $c(s, t)$ involves an intersection check and x is some memory address used by s or t ; and (2) $c(s, t)$ involves a predicate and $x = pred(t)$. Either case implies that s depends on x ; consequently, x is ordered before s and at least one of the **nodes**.

4.3.4 Materializing Versioning Plans

Figure 4-14 shows the algorithm for materializing versioning plans. The algorithm recursively materializes any secondary plans before materializing the primary plans. It uses a table that maps each instruction (or loop) to the set of dependence conditions that the instruction (or loop) is versioned under. An instruction (or loop) may be involved in more than one versioning plan; thus, its entry in this table is the union of all the conditions in the involved versioning plans.

Figure 4-15 shows how the versioning plan of the running example (Figure 4-12) is materialized in steps. First, the secondary plan is lowered (Figure 4-15a) by duplicating all of the versioned nodes and guarding them with checks computing the versioning conditions. After the secondary plan is lowered, the primary plan is lowered similarly

```

1 def infer_versioning_plan(nodes, input_nodes):
2     # Find a cut in the dependence graph that makes 'nodes'
3     # independent from 'input_nodes'
4     cut = find_cut(nodes, input_nodes)
5     # Abort if such a cut does not exist
6     if cut is None:
7         return None
8     dep_conds = set_union(c(e.src, e.dst)
9                           for e in cut.cross_edges)
10    # Find a secondary versioning to make sure that
11    # we can evaluate the dependence conditions
12    # before the versioned nodes.
13    cond_operands = set_union(operands(dep_cond)
14                              for dep_cond in dep_conds)
15    if set_intersect(cond_operands, input_nodes):
16        return None
17    secondary_plan =\
18        infer_versioning_plan(cond_operands, input_nodes)
19    if secondary_plan is None:
20        return None
21    # Update the cut to take the secondary plan into account.
22    # This is equivalent to:
23    # 1) updating the dependence graph to remove
24    #    edges that no longer exists because
25    #    of the secondary plan and
26    # 2) rerunning 'find_cut' on the updated graph.
27    cut = update_cut(cut, secondary_plan)
28    return VersioningPlan(nodes=cut.source_nodes + input_nodes,
29                          versioning_conds=dep_conds,
30                          secondary=secondary_plan)
31
32 def infer_version_plans_for_insts(insts):
33     return infer_versioning_plan(nodes=insts, input_nodes=insts)

```

Figure 4-13: Algorithm for inferring a versioning plan that makes a list of instructions independent. Lines 10-20 deal with secondary versioning.

```

1 def materialize_plans(versions):
2     # lower the secondary versions
3     secondaries = [version.secondary
4                     for version in versions
5                     if version.has_secondary()]
6     if len(secondaries) > 0: materialize_plans(secondaries)
7     # Build a table that maps each instruction/loop
8     # to its versioning conditions
9     versioning_table = build_versioning_table(versions)
10    # Mapping each cloned instruction/loop to its clone
11    clones = {}
12    # Mapping each cloned instruction to a phi that
13    # joins the original and clone
14    versioning_phis = {}
15    for inst_or_loop, conditions in versioning_table:
16        # Emit a check to return true iff *none* of
17        # the conditions are true.
18        # Also take care to hoist any dependences that
19        # exists *after* this inst/loop.
20        check = emit_check(conditions,
21                           insert_before=inst_or_loop)
22        clone = inst_or_loop.clone(
23            insert_after=inst_or_loop)
24        clones[inst_or_loop] = clone
25        # Predicates indicating if the check passes/fails
26        success_p = Predicate.make(check)
27        failure_p = Predicate.make(check, negate=True)
28        # The original computation runs if the check passes
29        inst_or_loop.update_predicate(success_p)
30        # The cloned computation runs if the check fails
31        clone.update_predicate(failure_p)
32        if inst_or_loop.is_instruction():
33            inst = inst_or_loop.cast_to_inst()
34            phi = Phi.make({success_p: inst, failure_p: clone},
35                           insert_after=clone)
36            versioning_phis[inst] = phi
37        else:
38            # Process the loop instructions similarly
39            ...
40
41    # Change the cloned instructions to
42    # use the correct operands
43    for inst_or_loop, conditions in versioning_table:
44        if inst_or_loop.is_instruction():
45            inst = inst_or_loop.cast_to_inst()
46            cloned_inst = clones[inst]
47            for user in inst.uses():
48                if conditions.subset_of(versioning_table[user]):
49                    cloned_user.replace_uses_of(inst,
50                                                with=cloned_inst)
51            else:
52                phi = versioning_phis[inst]
53                user.replace_uses_of(inst, with=phi)
54                if user in clones:
55                    clones[user].replace_uses_of(inst, with=phi)
56        else:
57            # Process the uses of the loop live outs similarly
58            ...

```

Figure 4-14: Algorithm for versioning a program according to a set of inferred versioning plans. For simplicity, the pseudocode emits the versioning checks for each instruction (or loop) from scratch without regard to redundancy; our implementation only emits one check for each unique set of conditions.

(Figure 4-15b); it is important to note that the materialization algorithm hoists the comparison as well as the load (of X) before stores (versioned by the primary plan) in order to compute the primary versioning condition; this hoisting enabled by the secondary versioning. Figure 4-15c shows the final versioned program further lowered to SSA (with control flow).

More generally, the algorithm for materializing the versioning plans works as follows. First, we emit a check for each unique set of dependence conditions to ensure they are all false. Before emitting the check, we also hoist the dependences of the check before the versioned nodes, which is feasible because, in the planning phase, we have ensured that the conditions do not depend on the versioned nodes. After emitting the checks, we clone the nodes and strengthen their control predicates so that the original copy executes if its check passes and the clone executes if the check fails.

Versioning may break the invariant that all definitions should dominate their uses. Therefore, we must update the instructions that use the versioned nodes as operands. More specifically, let i_{def} be an instruction that we want to version, i_{user} be one of its users, and i'_{def} be the cloned version of i_{def} . After versioning, i_{def} may no longer dominate i_{user} . This occurs when i_{user} is either not versioned or versioned under a set of conditions not implied by the versioning conditions of i_{def} . In this case, we need to replace all uses of i_{def} with an auxiliary phi node that takes the value of i_{def} if the check passes and i'_{def} if the check fails. Lines 47-55 show the algorithm for restoring the def-use dominance for a versioned instruction. Similarly, we also restore def-use dominance for the live-out instructions of versioned loops.

4.4 Implementation

We implemented the proposed versioning framework as an LLVM [Lattner and Adve, 2004] library with 4,982 lines of C++ code. The main interface of the library is two functions: the first performs versioning plan inference by taking a list of instructions (or loops) and returning a versioning plan that makes those instructions independent; this function may report that it is infeasible to infer such a plan; and the second

```

chk = cmp ne X, Y      ; true
store [Y+0], 0         ; chk
store [Y+0], 0         ; not chk
x = load X             ; chk
x' = load X            ; not chk
c = cmp ne x, 0        ; chk
c' = cmp ne x', 0      ; not chk
c_phi = phi(chk: c, not chk: c')
call cold_func         ; c_phi
store [Y+1], 0         ; chk
store [Y+1], 0         ; not chk

```

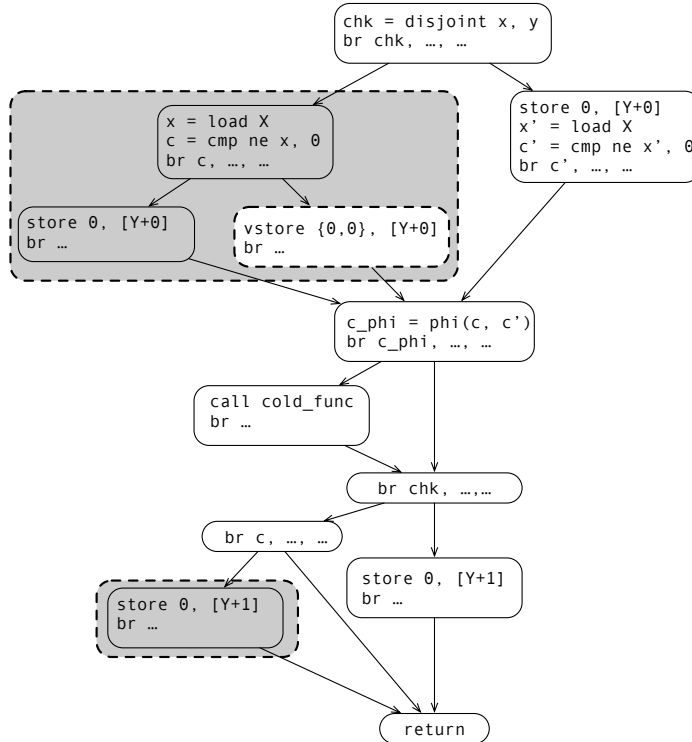
```

chk = cmp ne X, Y      ; true
x = load X             ; chk
c = cmp ne x, 0        ; chk
store [Y+0], 0         ; chk and not c
store [Y+1], 0         ; chk and not c
store [Y+0], 0         ; chk and c
store [Y+0], 0         ; not chk
store [Y+0], 0         ; not chk
x' = load X            ; not chk
c' = cmp ne x', 0      ; not chk
c_phi = phi(chk: c, not chk: c')
call cold_func         ; c_phi
store [Y+1], 0         ; chk and c
store [Y+1], 0         ; not chk

```

(a) The running example with the secondary versioning plan materialized. Because the comparison (c) is versioned and used (as a predicate) by the function call that is not versioned, we must insert a ϕ node to join the two versions of the comparisons for this use.

(b) The running example with the secondary and primary versioning plans materialized.



(c) The running example after versioning and conversion back to SSA with control flow.

Figure 4-15: The running example with the versioning plan (from Figure 4-12) materialized. Instructions versioned by the primary plan are surrounded with $\boxed{}$; instructions versioned by the secondary plan are surrounded with $\boxed{}$. We materialize the secondary plan before the primary plan.

function materializes a given list of versioning plans into actual IR instructions and control flow. In this section, we discuss some important implementation choices for the framework to work in practice, for correctness and performance.

4.4.1 Optimizing Dependence Conditions

Before materializing the versioning plans, we additionally allow the user to run some optimizations on their versioning plans to reduce the cost of run-time checks. More specifically, each memory intersection requires us to emit a sequence of pointer arithmetic and comparisons to perform the check. We have three optimizations to reduce the cost of performing the intersection checks: *redundant condition elimination*, *condition coalescing*, and *condition promotion*.

Redundant Condition Elimination. Some intersection checks are logically equivalent (but syntactically different). For example, we can see with basic algebraic manipulation that if $[a, a + 10]$ intersects with $[b, b + 2]$, then $[a + 100, a + 110]$ also intersects with $[b + 100, b + 102]$. In general, two intersection checks $intersects(r_a, r_b)$ and $intersects(r_x, r_y)$ (where r are memory ranges) are equivalent if $offset(r_x, r_a) = offset(r_y, r_b)$ or if $offset(r_x, r_b) = offset(r_y, r_a)$. where $offset(., .)$ provides the offset between two memory ranges. The offset between two ranges is undefined when their lower bounds and upper bounds differ by two different numbers; when this occurs we simply conclude that the two intersection checks are not equivalent. To perform redundant condition elimination, we first partition the dependence conditions into equivalence classes and then replace each dependence condition with an arbitrarily chosen representative from its equivalence class.

Condition Coalescing. Condition coalescing replaces two different range checks with a single check that overapproximates the original checks. The final coalesced check may be less precise yet cheaper to compute. For example, we may coalesce $intersects([a, a + 10], [b, b + 10])$ and $intersects([a + 20, a + 30], [b + 40, b + 50])$ into a single check $intersects([a, a + 30], [b, b + 50])$. Because condition coalescing results in less precise checks (that fail more often), we perform condition coalescing after

redundant condition elimination.

Condition Promotion. Condition promotion generalizes a range check to be loop-invariant so that the promoted check can be hoisted outside a loop. Consider the intersection check $\text{intersects}([a + i, a + i + 2), [b + i, b + i + 4))$, where i is a loop induction variable; in this case, we can promote the check to the equivalent check $\text{intersects}([a, a + 2), [b, b + 4))$, which no longer involves the induction variable and is therefore loop-invariant. This type of promotion is precise; i.e., a promoted check passes if and only if the original check passes. We also perform a second type of promotion that is imprecise. For example, we can promote the check $\text{intersects}([a + i, a + i + 1), b[b + 4*i, b + 4*i + 4))$ to the check $\text{intersects}([a, a + N + 2), [b, b + 4*N + 4))$, where N is the trip count of the loop.

4.4.2 Aliasing Annotation

Although our versioning plan materialization algorithm generates the requisite checks and control flow to guarantee that the user-supplied instructions (or loops) are independent, the independence of these instructions is implicit, and it is unlikely that LLVM’s built-in alias and dependence analysis can detect their independence. Thus, our implementation additionally annotates the versioned instructions with LLVM’s scoped noalias metadata [llv, 2023].

Using LLVM’s scoped noalias system provides two benefits. First, it allows us to reuse LLVM’s existing alias analyses and be compatible with any client optimization that uses those analyses. Second, LLVM’s instruction cloning utility preserves these noalias annotations, which is useful when an instruction is cloned multiple times due to nested versioning.

4.5 Evaluation

We evaluated the proposed versioning framework by using it for two optimizations: redundant load elimination and SLP vectorization. We show that our framework

makes both optimizations more effective with relatively minimal changes to their implementations.

Experimental Setup. We ran all experiments on a machine with an Intel® Xeon® Platinum 8124M CPU running at 3.00GHz. All benchmarks were single-threaded, run sequentially, and pinned to a specific core with `taskset`. All benchmarks were run 30 times and the median run times were used for reporting. Unless otherwise specified, we used LLVM’s -O3 pipeline as the baseline, which used LLVM’s loop and SLP vectorizers with the former using LLVM’s loop versioning implementation.

4.5.1 Versioning for SLP Vectorization

To demonstrate the effectiveness of our versioning framework, we modified an existing SLP vectorizer to support versioning.

While loop versioning is a natural enabling transformation for loop vectorization, there is no equivalent versioning scheme for SLP vectorization. With the unit of vectorization being individual vector packs and considering the diversity of the different SLP packing heuristics [Liu et al., 2012, Porpodas et al., 2019, Porpodas and Jones, 2015, Porpodas et al., 2015, 2018], a versioning scheme for SLP vectorization must be more fine-grained to fully take advantage of its flexibility.

Versioning for SLP Vectorization

We added support for versioning to SuperVectorization [Chen et al., 2022], an extension of SLP vectorization to handle complex data-dependent control flow. This vectorizer can pack instructions across different basic blocks or even loops that may have divergent run-time trip counts. Because of these complex control-flow patterns, their code-generation algorithm is more involved than a more traditional SLP vectorizer. Nonetheless, we added versioning to their vectorizer without modifying any of their code generation machinery.

We made only two changes to SuperVectorization to enable versioning. First, we modified their implementation to allow packing instructions that are conditionally

independent. Their system originally had a filter pass to reject any packs involving dependent instructions for vectorization. We modified this pass to forward such packs to our versioning plan inference routine and to accept a pack for vectorization if we can infer a versioning plan. At the end of the packing stage (but before vector code generation), our system ends up with a list of versioning plans. The second and last change we made was to materialize the list of versioning plans before performing vector code generation. No other changes were necessary, and their original vector code generator works out of the box.

Results

We evaluated the modified SuperVectorization on two benchmark suites: PolyBench [Pouchet, 2021] and TSVC [Callahan et al., 1988]. We used the same optimization pipeline as the LLVM -O3 optimizations, except that we ran SuperVectorizer instead of LLVM’s loop and SLP vectorizers (which are disabled in our pipeline). Note that LLVM’s loop vectorizer uses loop versioning.

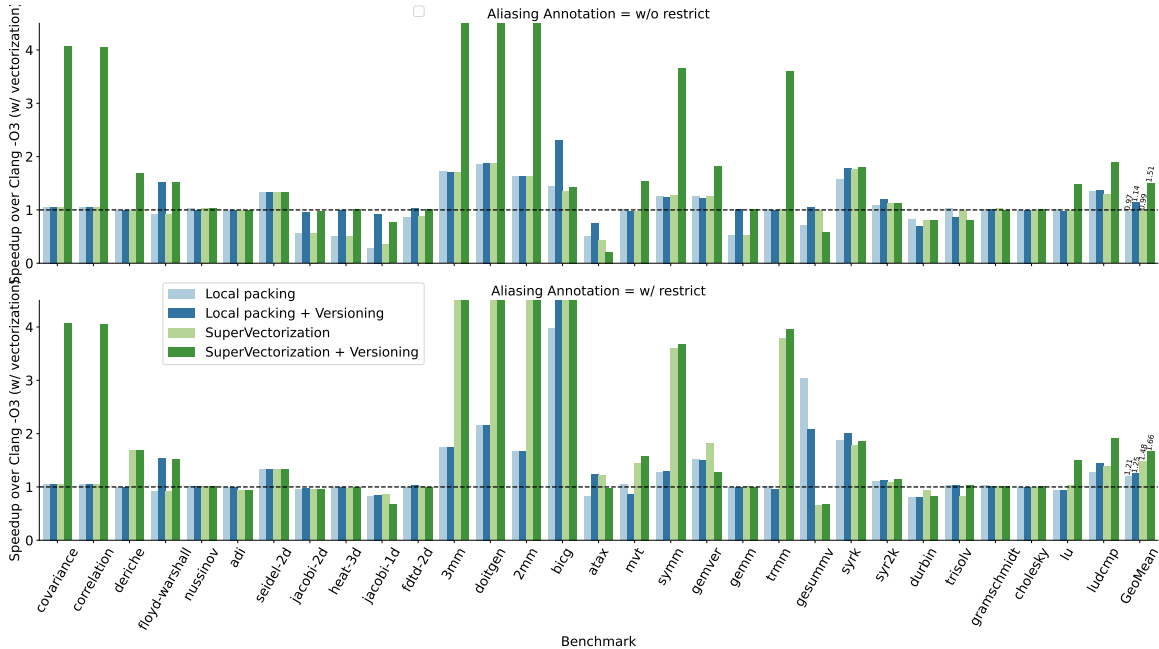


Figure 4-16: Results of fine-grained versioning + SuperVectorization on PolyBench over LLVM -O3 without vectorization (higher is better).

PolyBench. Figure 4-16 shows the results on PolyBench. We ran the experiments in

two settings, with the `restrict` keyword enabled and disabled. Because LLVM does not perform interprocedural alias analyses, without the `restrict` keyword, LLVM’s dependence analysis conservatively assumes that all input arrays in the PolyBench kernels may alias.

With `restrict` annotation turned off, SuperVectorization with versioning obtains a geomean $1.51\times$ over LLVM’s vectorizers, whereas SuperVectorization without versioning obtains a geomean $0.99\times$ speedup. With `restrict` turned on, SuperVectorization together with versioning obtains a $1.66\times$ speedup over LLVM’s vectorization, whereas SuperVectorization without versioning is $1.48\times$ faster.

We additionally ran a version of SuperVectorization that only performs local (i.e., intra-basic-block) packing in both settings (with and without `restrict`). With versioning, local packing is $1.14\times$ ($1.25\times$ with `restrict`) faster and $0.97\times$ ($1.21\times$ with `restrict`) without. So even without global packing, versioning is still beneficial, especially when `restrict` is disabled.

In sum, even with full aliasing information, versioning enables SuperVectorization to vectorize five more benchmarks—correlation, covariance, floyd-warshall, lu, and lu-dcmp, boosting SuperVectorization’s performance by $1.12\times$ on average. Of the five benchmarks, correlation, covariance, lu, and lu-dcmp have loop iterations that are independent but involve triangular iteration space, which is too complex for LLVM’s dependence analysis to reason. With versioning, SuperVectorization effectively performs dependence analysis dynamically.

Figure 4-17 shows a code snippet of the floyd-warshall benchmark. Neither SuperVectorization (without versioning) nor LLVM can vectorize the floyd-warshall benchmark because the benchmark performs in-place updates on the output buffer, leading to a dependence from the read of `path[i][k]` to the write of `path[i][j]`. However, this read-write conflict does not occur on every loop iteration. Our versioning algorithm allows SuperVectorization to safely vectorize by checking for this dependence dynamically and executing the vectorized code in the absence of this dependence.

Figure 4-18 shows floyd-warshall optimized by our versioning algorithm. Notice that the code does not check the dependencies between the write `path[i][j]` and the read `path[k][j]`. Although this dependence blocks traditional auto-parallelization, it does not prevent SLP vectorization by itself and is left out of run-time checking by our versioning algorithm. We illustrate how our algorithm comes to this decision by unrolling the code (by a factor of 2, but another unrolling factor also works) as follows:

```

1    ... = path[k][j]
2    ... = path[i][k]
3    ... = path[i][j]
4    path[i][j] = ...
5    ... = path[k][j+1]
6    ... = path[i][k]
7    ... = path[i][j+1]
8    path[i][j+1] = ...

```

We can see that the write to `path[i][j+1]` may depend on the reads of `path[i][j+1]` and `path[k][j+1]` (we ignore the dependence from `path[i][j+1]` to `path[i][k]` because it's in the cut), neither of which depend on `path[i][j]`. We can therefore conclude that the writes to `path[i][j]` and `path[i][j+1]` are independent and can be packed into a single vector store. One can follow this process and conclude that the other reads are also independent and can be packed: provided the accesses to `path[i][j:...]` and `path[k][j...]` are disjoint (which we check at run time).

```

1  for (k = 0; k < _PB_N; k++)
2    for (i = 0; i < _PB_N; i++)
3      for (j = 0; j < _PB_N; j++)
4        path[i][j] = path[i][j] < path[i][k] + path[k][j]
5        ? path[i][j] : path[i][k] + path[k][j];

```

Figure 4-17: Snippet of the floyd-warshall benchmark from PolyBench. Traditional vectorization algorithms do not apply here (even with full aliasing information) because of the read-write conflict between the write of `path[i][j]` and `path[i][k]`. Our versioning scheme enables vectorization by checking for this conflict dynamically at run time.

We also observed several cases where local packing achieves better performance than global packing (e.g., `gesummv` when `restrict` is disabled), due to the way SuperVectorization performs unrolling. SuperVectorization requires a preparatory

```

1  for (k = 0; k < _PB_N; k++) {
2      for (i = 0; i < _PB_N; i++)
3          for (j = 0; j < _PB_N; j += VL) {
4              if (disjoint(&path[i][j:j+VL], &path[i][k])) {
5                  // vector code
6              } else {
7                  // scalar code
8              }
9          }

```

Figure 4-18: Sketch of the code generated by our versioning algorithm for floyd-warshall. Our implementation generates LLVM IR, but we show an equivalent code snippet in C for readability. VL represents the native vector length.

unrolling pass to expose the available parallelism. When faced with nested loops, our custom unroller performs unrolling trials for outer loops first, and as soon as an outer loop is unrolled and vectorization opportunity detected, the unroller stops any further unrolling of the inner loops to avoid compile-time and code-size blowups. Overall this is a profitable strategy as evidenced by the speedups we observe. However, for programs such as `gesummv`, where inner-loop vectorization is more beneficial (and outer-loop vectorization in fact degrades performance), this strategy leads to suboptimal unrolling.

TSVC. Figure 4-19 shows the results of TSVC, a comprehensive suite of 151 loops for evaluating vectorization algorithms. Without versioning, SuperVectorization is $1.15\times$ (geomean) faster than LLVM; and with versioning, SuperVectorization is $1.22\times$ faster. Versioning enables SuperVectorization to vectorize 13 more benchmarks, improving the overall performance of SuperVectorization by $1.06\times$ on average. We also show the speedups of local packing with (and without) versioning. With versioning, local packing leads to a $1.00\times$ speedup and 0.93 without versioning. The slowdown in this case mainly comes from the fact that LLVM’s vectorizer supports vectorizing inner-loop control flow.

Figure 4-21 shows a code snippet from `s342`, where a loop-recurrent variable is updated conditionally. We enable vectorization by speculating that the update does not take place.

Figure 4-22 shows a snippet of the benchmark. Our versioning framework enables vectorization by speculating that successive iterations of `a[i] > 0` are true at run time. We did not observe speedup (or slowdown) in TSVC’s original setup because,

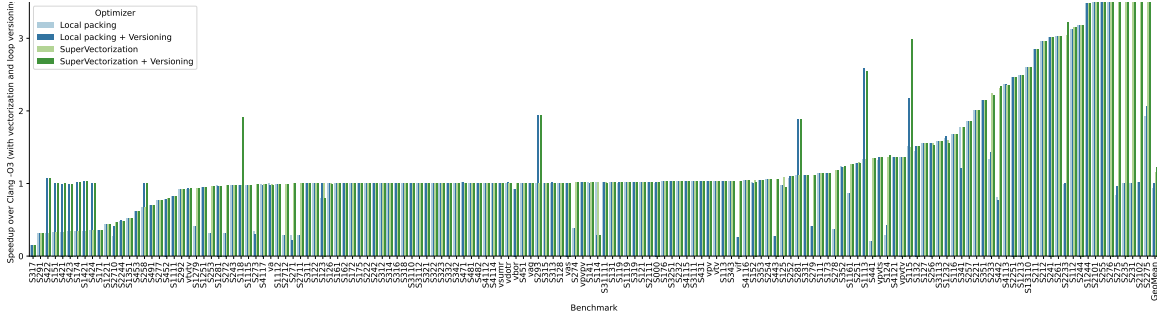


Figure 4-19: Speedup on TSVC over LLVM -O3 (higher is better). Note that LLVM’s loop vectorizer uses loop versioning.

```

1  for (int i = 0; i < LEN; i++) {
2      x = a[LEN-i-1] + b[i] * c[i];
3      a[i] = x-(float)1.0;
4      b[i] = x;
5  }

```

Figure 4-20: s281 from TSVC. There is a read-write conflict on the array *a*, but only in some iterations (around the middle of the array). Our versioning framework generates code that identifies precisely the iterations that are safe to vectorize at run time.

```

1  for (int i = 0; i < LEN; i++) {
2      if (a[i] > (float)0.) {
3          j++;
4          a[i] = b[j]; } }

```

Figure 4-21: s342 from TSVC. The benchmark is not vectorizable by traditional techniques because the conditional update on *j* causes a loop-carried dependence. Our versioning framework enables vectorization by speculating that none of the branches are taken.

the run time is data-dependent with versioning. Consequently, we did another set of experiments where we initialized the array *a* randomly so that more than 99% of the entries are positive. In this setup, the vectorized code is 2.0× faster than the scalar version. In a separate experiment, we further modified the benchmark so that all of the arrays are declared as function parameters (instead of global variables), forcing the compiler to assume the arrays may alias. In this setup, one level of versioning is no longer sufficient; to generate code to speculate that $a[i] > 0$ for successive iterations, we must hoist the comparisons and the load of $a[i]$ past the stores to the arrays *b* and *e*, which is only safe if *a* does not alias either *b* or *e*. We observe similar speedups even with two-levels of versioning for this example because our implementation is able

to hoist the alias checks outside the loop and amortize the overhead of the checks.

```
1  float s = 0.;
2  for (int i = 0; i < N; i++) {
3      if (a[i] > 0.) {
4          s = d[i] * d[i];
5      }
6      b[i] = s * c[i] + d[i];
7      e[i] = (s + (float)1.) * aa[i];
8  }
```

Figure 4-22: s258 from TSVC. Without versioning, the loop-carried dependence around `s` prevents vectorization.

4.5.2 Versioning for redundant load elimination

In a case study to demonstrate the effectiveness of our versioning framework, we used it to perform redundant load elimination, a basic optimization implemented by modern compilers. A load is redundant if (1) it is dominated by a prior load on the same memory address and (2) there are no intervening memory writes between them. In practice, the latter condition often prevents compilers from eliminating many redundant loads because, between two redundant loads, there are often some spurious writes—stores on plausibly aliased addresses or even function calls—that force the compiler to conservatively assume the loads are not redundant.

We can eliminate more redundant loads by viewing redundant loads from a perspective of independence: a set of loads—assuming one load dominates all the others—is redundant if they are all independent. Together with our versioning framework, this view leads to a simple implementation of redundant load elimination that is also more aggressive than those implemented by existing compilers:

1. Collect groups of loads that operate on the same pointer values and the same data types. Each group should have at least one load whose execution is implied by the other loads, and we designate such a load as the *leader* of the group.
2. For each such group, if the loads in the group are not already independent, attempt to infer a versioning plan for the group so that they become independent, or abandon the group for optimization if versioning is infeasible.

	namd_r	parest_r	povray_r	lbm_r	blender_r	imagick_r	nab_r	GeoMean
Speedup	0.5%	-0.5%	-1.7%	6.4%	4.7%	0.0%	0.0%	1.2%
Loads eliminated	1.2%	-0.43%	-0.21%	26%	0.69%	-0.22%	2.7%	4.8%
Branches increase	3.4%	1.1%	1.3%	3.5%	-0.73%	0.0%	2.2%	5.5%
Extra hoists by LICM	50%	0.15%	0.1%	0.0%	1.2%	0.9%	0.0%	6.4%
Extra deletions by GVN	0.59%	9.6%	1.3%	3.3%	19%	7.1%	7.7%	8.5%
Code size increase	1.5%	0.73%	1.2%	12%	0.88%	0.51%	0.00%	2.3%

Figure 4-23: Result of running redundant load elimination (with versioning) on the SPEC 2017 Floating Point benchmark suite.

3. Materialize the versioning plans inferred for the store groups.
4. Hoist the *leader* of each group before all the other loads—such hoisting is safe because we have established that the loads are independent—and replace the uses of the other loads with the leader.

We implemented this optimization as an LLVM pass with 153 lines of C++.

Figure 4-23 shows the result of running the proposed redundant load elimination pass on the SPEC 2017 Floating Point benchmark suite. Our optimization leads to major speedups for `lbm_r` (6.4%) and `blender_r` (4.7%), with the geomean speedup being 1.2%.

We also collected performance counters for the benchmarks. With versioning, redundant load elimination eliminated 4.8% (geomean) of the loads executed dynamically, at the expense of executing 5.5% (geomean) more branches at run time. Our optimization also enabled more optimizations downstream in the pipeline: loop-invariant code motion (LICM) hoisted 6.4% more instructions and global value numbering (GVN) deleted 8.5% more instructions.

4.6 Related Work

Loop Versioning. Existing loop versioning implementations, such as the ones used by GCC and Clang, focus on pointer disambiguation and require predictable memory access patterns to overapproximate the accessed ranges of memory.

To address this limitation, researchers have proposed using specialized memory allocators that can efficiently check whether two pointers can point to the same allocated object [Chitre et al., 2022, 2023, Alves et al., 2015]. With allocators, loop versioning can be applied to loops with unpredictable memory accesses: Two arrays may be accessed randomly; nevertheless, they can establish their disjointness by querying the allocator whether their base pointers point to the same object.

This allocator-based loop versioning scheme is more efficient than our approach because they can disambiguate pointers outside loops with irregular memory accesses; in contrast, to check the dependencies for such loops, our approach must perform the checks within those loops. On the other hand, our approach can rule out intra-object dependences (common to code with in-place updates), whereas theirs cannot. Our approach is also more fine-grained: given a list of instructions that should be independent, our system automatically computes the minimal set of dependencies that need to be checked at run time instead of using hand-designed and optimization-specific rules to decide which dependencies should be checked. Furthermore, our approach works in the absence of loops and applies to non-loop optimizations.

Speculative Optimization. Our framework is also related to Lin et al. [Lin et al., 2003]’s work on performing speculative partial redundancy elimination (PRE) in the presence of rare dependencies. Their work relies on the architectural speculation support provided by IA-64. Our approach does not perform speculation and targets more conventional architectures without speculation capabilities.

Finally, our work is related to Sujon et al. [Sujon et al., 2013]’s work on speculative loop vectorization, which was proposed to deal with loops with biased branches that incur rare loop-carried dependences that prevent vectorization. Their approach works by identifying sets of paths where vectorization is safe, vectorizing the loop, and then generating run-time checks to guard vectorized code in case any other paths are taken. If the checks fail, the loop restarts execution in scalar mode for the next several iterations. Their approach relies on explicitly enumerating all possible CFG paths as candidates for speculation. Because there can be an exponential number of paths within a given control-flow graph, their approach must consider exponentially many

paths or up to a predetermined abort threshold. Our approach does not suffer from exponential blow-ups because it reduces the problem of finding the set of speculation (versioning) conditions to min-cut, which has an efficient solution.

4.7 Summary

We have introduced a general framework for performing fine-grained versioning. To demonstrate its effectiveness, we presented case studies of applying our framework to redundant load elimination and SLP vectorization. Our redundant load elimination is simple and yet effective in removing redundant loads that LLVM fails to detect. With our versioning framework, an existing implementation of SLP vectorization can be extended with minimal changes to perform versioning, achieving speedups that are competitive with, and in many cases outperform, LLVM’s vectorizers that rely on loop versioning by exploiting dependence information that is only available at run time.

Chapter 5

VeGen: vectorization beyond SIMD instructions

5.1 Introduction

Vector instructions are ubiquitous in modern processors. Previous work on auto-vectorization has focused on single instruction multiple data (SIMD) instructions, but there is little research on systematically targeting non-SIMD vector instructions, which has applications in domains such as digital signal processing, image processing, and machine learning (e.g., Intel’s VNNI extension and the dot-product instructions in ARMv8 [Holdings, 2011]). In contrast with the SIMD instruction shown in Figure 5-1a, Figures 5-1b–5-1d show three examples of the non-SIMD instructions from the AVX2 instruction set. Figure 5-1b shows a single instruction, multiple operations, multiple data (SIMOMD) instruction [Bachega et al., 2004] that performs additions and subtractions on alternating lanes (`vaddsubpd`); Figure 5-1c shows a horizontal addition with lane interleaving (`vhaddpd`); and Figure 5-1d shows an instruction computing dot-products (`vpmaddwd`). To date, there is no unified model of parallelism that captures the capabilities of these instructions.

Existing techniques for vectorizations—whether SLP or loop—make two fundamental assumptions about vector instructions: a SIMD instruction performs isomorphic

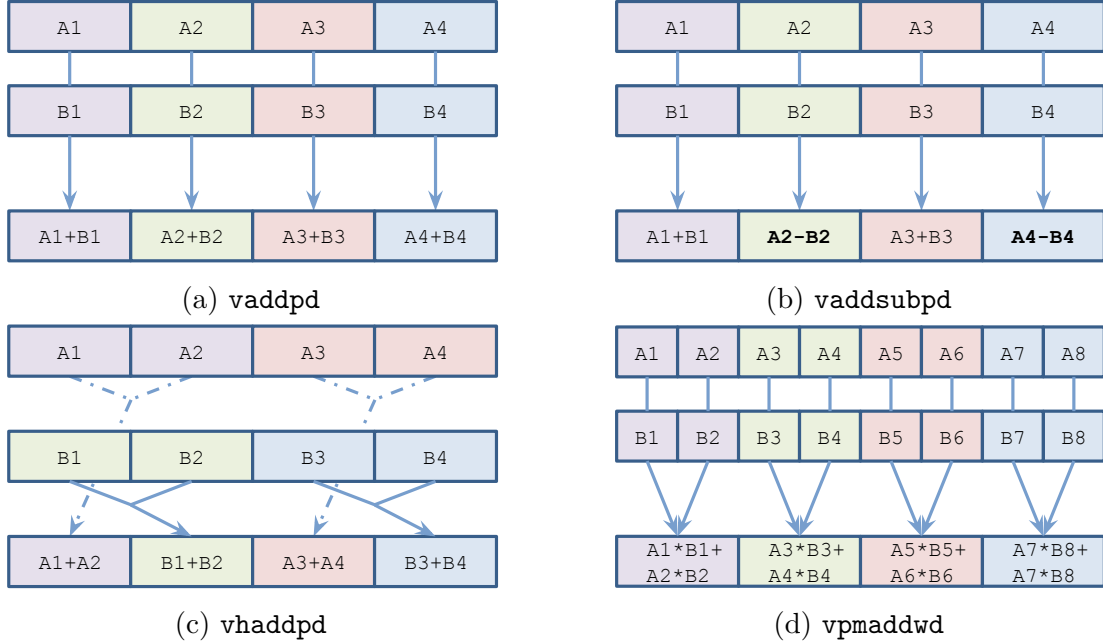


Figure 5-1: Examples of SIMD and non-SIMD instruction in the AVX2 instruction set. We use different colors to indicate how the input values flow to different output lanes.

operations across all lanes, and the instruction applies the operations elementwise (i.e., there is no cross-lane operation). Relying on these two assumptions, these algorithms enable compiler developers to support SIMD instructions across a variety of architectures with relatively little incremental effort.

Existing Support for Non-SIMD Instructions. Because non-SIMD instructions violate the two fundamental assumptions of existing vectorization algorithms, compiler developers support non-SIMD instructions using ad hoc approaches that are cumbersome and often ineffective. For most non-SIMD instructions, compiler developers support them with backend peephole rewrites. However, because these peephole rewrites do not generate vector instructions by themselves—they fuse sequences of SIMD instructions and vector shuffles into more non-SIMD instructions—relying on peephole rewrites alone is ineffective. A relatively more effective but more labor-intensive strategy involves coordinating with the compiler’s vectorizers to generate SIMD vector patterns that are tailored for those rewrite rules. In other words, to support non-SIMD instructions within an existing vectorization algorithm, compiler engineers must modify the algorithm to emit short sequences of SIMD instructions

and vector shuffles that emulate the non-SIMD instructions and in a later pass, rewrite the sequences into the desired non-SIMD instructions. For instance, the initial support in LLVM [Lattner and Adve, 2004] for the `addsub` instruction family (Figure 5-1b) required three coordinated changes to LLVM: refactoring LLVM’s SLP vectorizer to support alternating opcodes, changing LLVM’s cost model to recognize a special case of vector shuffle (blending odd and even lanes), and modifying LLVM’s backend lowering logic to detect the special patterns generated by the SLP vectorizer. As processor vendors continue to add more complex non-SIMD instructions, this methodology is not sustainable. Compilers are falling behind in identifying the complex code sequences that can be mapped to these instructions, and these multibillion-dollar investments by the processor vendors in enhancing the vector instruction sets go underutilized without expert developers manually writing assembly or compiler intrinsics.

VeGen. In this thesis, I describe an extensible framework for systematically targeting non-SIMD vector instructions. We define a new model of vector parallelism more general than SIMD parallelism, and we present a vectorizer generator that can effectively extract this new model of parallelism using non-SIMD instructions.

To broaden the parallelism modeled by existing vectorizers, we introduce *Lane Level Parallelism* (LLP), which generalizes superword level parallelism (SLP) [Larsen and Amarasinghe, 2000] beyond SIMD in two ways: (1) An instruction can execute multiple non-isomorphic operations, and (2) the operation on each output lane can use values from arbitrary input lanes. These two properties of LLP depend on the semantics of a given target vector instruction. Consequently, our framework encapsulates the two LLP properties (i.e., which operation executes on a given lane and which values the operation uses) in a couple of target-dependent vectorization utility functions. By interfacing with these utilities, the core vectorization algorithm in our framework remains target-independent, as traditional vectorization algorithms do.

We realize this framework with VEGEN, a system that automatically generates target-architecture-aware vectorizers to uncover LLP in straight-line code sequences while using only instruction semantics as input. From these instruction semantics, VEGEN automatically generates the implementation of the aforementioned vector-

<pre> void dot_16x16_uint8_int8_int32(uint8_t data[restrict 4], int8_t kernel[restrict 16][4], int32_t output[restrict 16]) { for (int i = 0; i < 16; i++) for (int k = 0; k < 4; k++) output[i] += data[k] * kernel[i][k]; } </pre>	<pre> movzx r11d, [rdi] movsx eax, [rsi] imul r11d, eax ... add r11d, r10d add r11d, ecx mov [rdx], r11d </pre>	<pre> vmovdqa xmm0, [rip] vmovdqu xmm1, [rsi] ... vpmovsxbw xmm7, xmm6 vpbroadcastw xmm5, xmm5 vpmullw xmm7, xmm7, xmm9 vpsrldq xmm2, xmm6, 8 ... </pre>
(a) Reference Implementation	(b) ICC	(c) GCC
Number of Instructions Speedup Relative to ICC Vector Extensions Used	273 1.0× Not Vectorized	106 1.5× SSE4
<pre> vmovdqu xmm6, [rsi + 32] vmovdqu xmm7, [rsi + 48] ... vpmulld zmm1, zmm11, zmm1 vpadd zmm1, zmm1, [rdx] vpmovsxbd zmm3, xmm3 vpmulld zmm3, zmm10, zmm3 ... </pre>	<pre> vmovdqu64 zmm0, [rdx] vpbroadcastd zmm1, [rdi] vpdpbusd zmm0, zmm0, [rsi] vmovdqu64 [rdx], zmm0 </pre>	
	(d) LLVM	(e) VEGEN
Number of Instructions Speedup Relative to ICC Vector Extensions Used	61 2.2× SSE4 & AVX-512	4 11.0× AVX512-VNNI

Figure 5-2: One of the dot-product kernels used by TVM’s 2D convolution layer (Figure 5-2a). Compiler generated assembly and statistics for Intel’s compiler ICC (Figure 5-2b), GCC (Figure 5-2c), LLVM (Figure 5-2d), and VEGEN (Figure 5-2e)

ization utilities as a compiler library to describe the specific kind of LLP supported by the target architecture. With this automatically generated target-description library, VEGEN’s vectorizer can automatically use non-SIMD vector instructions. We added support for newer classes of non-SIMD vector instructions (e.g., those found in AVX512-VNNI, which are not fully supported by LLVM) by providing only their semantics.

5.2 Teaser: Using a neural network instruction

In Figure 5-2, we compare VEGEN with three production compilers on a kernel used by TVM’s [Chen et al., 2018] 2D convolutional layers. Figure 5-2a shows the naive scalar implementation of this kernel. Figures 5-2b–5-2e show the assembly output of ICC 19.0.1, GCC 10.2, LLVM 10.0, and the VEGEN-generated vectorizer, respectively. All code generators were configured to target AVX512-VNNI.

VEGEN’s vectorizer generates by far the shortest assembly code sequence, 15.25×

shorter than the next shortest code generator, LLVM, and the generated code runs $5\times$ faster than LLVM's. VEGEN's vectorizer uses a new AVX512-VNNI instruction (`vpdpbusd`); GCC uses some of the integer vector instructions introduced in SSE4 (`vpaddq` and `vpmullw`); LLVM uses a mix of SSE and AVX512 instructions (`vpaddq` and `vpmulld` operating on the 512-bit `zmm` registers); and ICC, Intel's own compiler, does not vectorize the code. This is in spite of many man-hours spent on these compilers to support Intel's multibillion-dollar investment in these vector extensions. In contrast to these manual engineering efforts to target new vector extensions, the target-specific components of VEGEN are automatically generated from semantics.

In this example, VEGEN's vectorizer uses a new dot-product instruction (`vpdpbusd`) introduced in the AVX512-VNNI instruction set. No other evaluated compilers were able to use this instruction. It is important to note that VEGEN's output (Figure 5-2e) cannot be generated simply by pattern matching because of the extra data movement using the instruction `vbroadcastw`, which reorders the inputs of `vpdpbusd`.

VEGEN allows compilers to target new vector instructions with less development effort. Thus, we believe this new capability will enable the creation of more robust vectorizers in production compilers.

5.3 Lane Level Parallelism

Lane Level Parallelism (LLP) is our relaxation of superword level parallelism (SLP) [Larsen and Amarasinghe, 2000], which models short-vector parallelism (in which an instruction executes multiple scalar *operations* in parallel) with the following restrictions:

- The operations execute in lock-step.
- The inputs and outputs of the operations reside in packed storage (usually implemented as vector registers). We refer to an element of such packed storage as a *lane*.
- The operations are isomorphic.

- The operations are applied elementwise (i.e., there is no cross-lane communication).

LLP relaxes SLP by removing the last two restrictions: (1) The operations can be non-isomorphic, and (2) an operation executing on one lane can use values from other input lanes.

Non-isomorphism. LLP allows different operations to execute in parallel, whereas SLP applies only one operation across all vector lanes. An example of an instruction that uses such a parallel pattern is the x86 instruction `vaddsubpd` (Figure 5-1b), which does addition on the odd lanes and subtraction on the even lanes.

Cross-lane communication. LLP allows an operation executing on one lane to access values from another input lane (as long as the lane is selected statically). In contrast, SLP restricts an operation to use values from its own input lane. This flexibility is useful for computations that require communication between lanes (e.g., parallel reduction). For example, `vhaddpd` horizontally combines pairs of lanes using addition and then interleaves the results (Figure 5-1c).

These properties of LLP depend on the semantics of individual instructions. Different instructions can use different combinations of operations or apply different cross-lane communication patterns.

5.4 VeGen’s Workflow

The key idea of VEGEN is to encapsulate the details of the two LLP properties (non-isomorphism and cross-lane communication) behind two interfaces. VEGEN views a given vector instruction as a list of operations, each of which associated with a pattern matcher (interface 1). Each vector instruction has a lane-binding function that tells VEGEN how the input lanes bind to the operations (interface 2). VEGEN generates the implementations of these two interfaces offline. At compile time, VEGEN’s target-independent vectorization algorithm works by first using the pattern matcher to find independent IR fragments that can be packed into the available vector instructions,

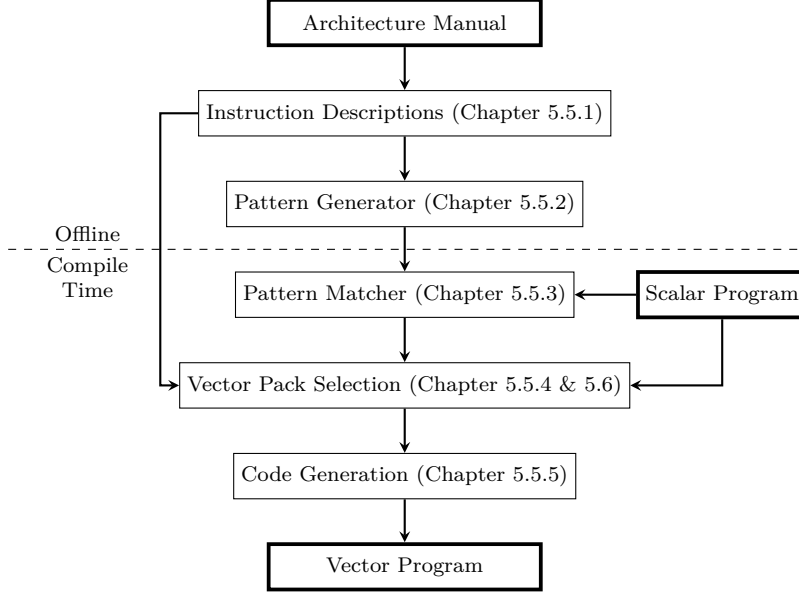


Figure 5-3: VEGEN’s workflow. Bolded boxes represent artifacts such as manuals and programs.

then using the lane binding rule to identify the vector operands used by the packed vector instructions, and then recursively finding other IR fragments that can be packed to produce those vector operands.

Figure 5-3 shows the workflow of VEGEN. VEGEN targets non-SIMD (and SIMD) vector instructions in two phases. In the offline phase, VEGEN takes instruction semantics (encoded in its vector instruction description language) as input and generates the target-dependent utility functions, such as the pattern matchers. At compile time, VEGEN’s target-independent heuristic uses the generated utility functions to combine independent streams of scalar instructions into vector instructions.

To target a new vector instruction set, VEGEN only requires the compiler writers to describe the semantics of each instruction in VEGEN’s vector instruction description language. If the vendor has provided instruction semantics in a machine-readable format such as Intel’s Intrinsics Guide [Intel Corporation, 2012], this process can be automated. In Chapter 5.7, we describe how VEGEN automatically translates semantics from the Intrinsics Guide.

Terminology & Notation. We use two related but distinct terms: *instructions* and *operations*. *Instructions* can refer to either IR instructions such as LLVM IR or target

instructions such as x86 instructions. *Operations* refer to (side-effect free) bit-vector functions that can be implemented both by IR and target instructions.

For brevity, we overload common set operations for vectors. While doing so, we implicitly convert a vector to a set before applying the set operator. For example, let x be a vector and i a scalar; when we say $i \in x$ we mean that x contains i .

5.5 An end-to-end example of VeGen using a non-SIMD instruction

Before I discuss in more details the different components of VEGEN, I will give a short walkthrough of how VEGEN uses `pmaddwd`, a complex non-SIMD instruction supported by x86. This walkthrough should illustrate on a high level what these components of VEGEN do and how they fit together.

We first start with the semantic description of the instructions. The following is Intel’s pseudocode documentation of `pmaddwd`:

```
FOR j := 0 to 3
  i := j*32
  dst[i+31:i] :=
    SignExtend32(a[i+31:i+16]*b[i+31:i+16]) +
    SignExtend32(a[i+15:i]*b[i+15:i])
ENDFOR
```

In its offline phase, VEGEN translates this semantic description into the representation shown in Figure 5-4. We call this internal representation *Vector Instruction Description Language (VIDL)*. The main purpose of VIDL is to abstract away the syntactic details of instruction documentation that is irrelevant for code generation.

From this formal representation, VEGEN then generates two utility functions that VEGEN’s vectorization heuristic will eventually use at compile time. The following is the generated pattern-matching function that recognizes the type of operations that `pmaddwd` applies for each of its output vector elements.

VEGEN also generates a lane-binding function. The lane-binding function informs

$$\begin{aligned} \text{op}_{\text{madd}} &= (x_1 : 16, x_2 : 16, x_3 : 16, x_4 : 16) \mapsto \\ &\quad \text{add}(\text{mul}(\text{sext32}(x_1), \text{sext32}(x_2)), \text{mul}(\text{sext32}(x_3), \text{sext32}(x_4))) \\ \text{pmaddwd} &= (a : 4 \times 16, b : 4 \times 16) \mapsto \\ &\quad [\text{op}_{\text{madd}}(a[0], b[0], a[1], b[1]), \text{op}_{\text{madd}}(a[2], b[2], a[3], b[3])] \end{aligned}$$

Figure 5-4: Semantics of `pmaddwd` formalized in VEGEN’s vector instruction description language

```
bool match_MADD_Op(llvm::Value *V, Match &M) {
    llvm::Value *t0, *t1, *t2, *t3;
    if (m_c_Add(m_c_Mul(m_SExt(t0), m_SExt(t1)),
                 m_c_Mul(m_SExt(t2), m_SExt(t3))).match(V)) {
        M.LiveIns = { t0, t1, t2, t3 };
        return true;
    }
    return false;
}
```

Figure 5-5: Pattern matcher generated for the vector instruction `pmaddwd`

VEGEN’s vectorization heuristic that, if it combines multiple such operations (as recognized by the above pattern-matching function) into a single instruction using `pmaddwd`, how the live-ins of matched operations should bind to the input vector lanes.

```
std::vector<llvm::Value *>
operand_1_pmaddwd(const std::vector<Match> &Matches) {
    return { Matches[0].LiveIns[0], Matches[0].LiveIns[2],
             Matches[1].LiveIns[0], Matches[1].LiveIns[2] };
}
```

At compile time, VEGEN then uses these automatically generated utilities to vectorize its input program. Suppose we have the following input scalar program.

```
int16_t A[4], B[4];
int32_t C[2];
void dot_prod() {
    C[0] = A[0] * B[0] + A[1] * B[1];
    C[1] = A[2] * B[2] + A[3] * B[3];
}
```

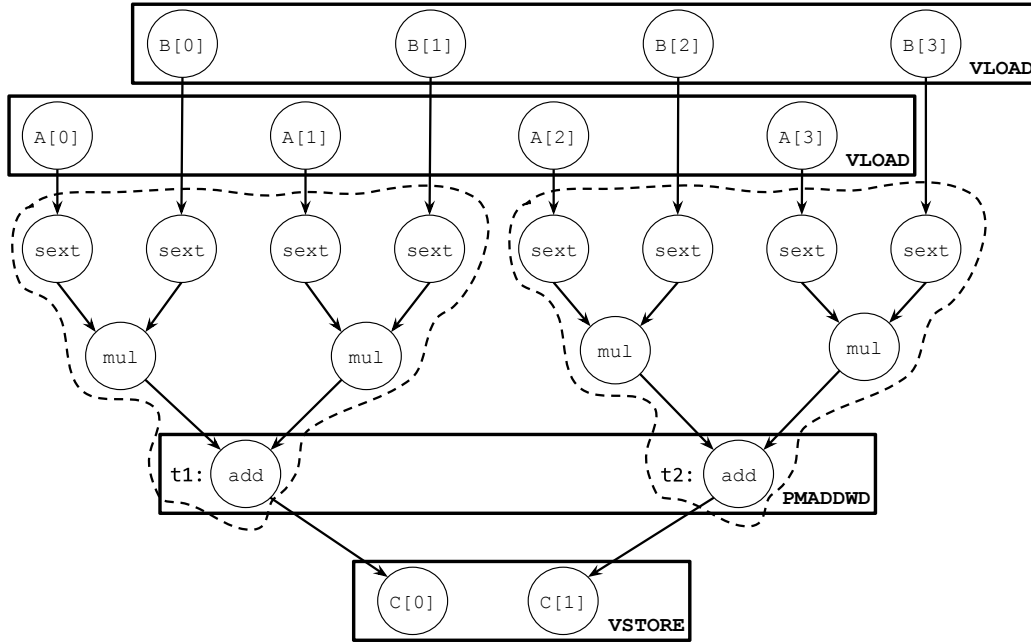


Figure 5-6: The instruction DAG corresponding to the example scalar program. The regions enclosed by the dotted curves represent matched integer multiply-add operations. The rectangles represent vector packs.

}

At compile time, VEGEN vectorizes its input program in three steps:

1. Apply the auto-generated pattern matchers for the target-specific operations.
2. Select a profitable subset of the matched scalar instructions that can be combined into (potentially non-SIMD) vector instructions.
3. Combine the selected scalar instructions into vector instructions.

Figure 5-6 shows VEGEN matching (step 1) and combining (step 2) the operations of `pmaddwd` that occurs in the input program. Figure 5-6 shows the result of code generation (step 3).

5.5.1 Vector Instruction Description Language

VEGEN uses its vector instruction description language (VIDL) to model the semantics of each target vector instruction as a list of scalar operations, with lane-binding rules

```

vmovd xmm0, [A]
vmovd xmm1, [B]
pmaddwd xmm0, xmm1, xmm0
vmovd [C], xmm0

```

Figure 5-7: Generated vector code

$$\begin{aligned}
& x \in \mathbf{variables} \quad i \in \mathbf{integers} \\
& sz \in \mathbf{bit-widths} \quad vl \in \mathbf{vector-lengths} \\
& lane ::= x[i] \\
& expr ::= x \mid lane \mid binop(expr_1, expr_2) \mid \\
& \quad unop(expr) \mid \mathbf{select}(expr_1, expr_2, expr_3) \\
& opn ::= (x_1 : sz_1, \dots, x_n : sz_n) \mapsto expr \\
& res ::= opn(lane_1, \dots, lane_n) \\
& inst ::= (x_1 : vl_1 \times sz_1, \dots, x_n : vl_n \times sz_n) \mapsto [res_1, \dots, res_m]
\end{aligned}$$

Figure 5-8: Syntax of the Vector Instruction Description Language (VIDL). $\mapsto$ denotes function abstraction.

indicating how the input lanes bind to the operations. Figure 5-8 shows the syntax of VIDL. VIDL assumes that target instructions read and write to registers but have no other side-effects. VEGEN models memory instructions such as vector load as a separate, special class of instructions. VIDL only allows selecting the input lanes using constant indices: This restriction allows VEGEN to statically determine the vector operands used by each vector instruction.

Figure 5-4 shows the semantics of the SSE instruction `pmaddwd` specified in VIDL. The instruction `pmaddwd` takes two vector registers as input, sign-extends the values from 16-bit to 32-bit temporaries, multiplies the sign-extended values element-wise, and finally adds together every adjacent pair of the multiplication results.

5.5.2 Generating Pattern Matchers

In the offline phase, VEGEN collects the set of operations used by the target vector instructions, and for each operation, VEGEN generates pattern matching rules to recognize IR sequences that implement the operation. Figure 5-5 shows an example of the pattern matching code generated by VEGEN.

We designed VIDL to mirror the scalar IR that its vectorizer takes as input. Thus, generating pattern matching code from VIDL is generally straightforward. In Chapter 5.7 we discuss how to generate pattern matchers that are more robust.

5.5.3 Pattern Matching

At compile time, VEGEN applies the generated pattern matchers on the input scalar program. We call the result of pattern matching a *match*, an IR instruction DAG with (possibly) multiple live-ins and a single live-out. VEGEN represents each match as a tuple consisting of its live-ins, live-out, and operation. In the running example (Figure 5-6), the integer multiply-add operation has two matches (the sub-graphs enclosed in dotted curves): one rooted at the instruction t_1 , and another rooted at t_2 .

Unlike other common applications of pattern matching such as term rewriting, VEGEN does not directly use the result of pattern matching to rewrite the program. Instead, VEGEN records the matched patterns in a match table, which records the mapping $\langle \text{live-out}(m), \text{operation}(m) \rangle \mapsto m$, for each match m . The match table allows VEGEN’s target-independent vectorization algorithm (Chapter 5.5.4) to efficiently enumerate the set of candidate vector instructions that can produce a given vector (Algorithm 1).

5.5.4 Vectorization

After running the generated pattern matchers (at compile time), VEGEN (1) uses a target-independent heuristic to find profitable groups of matched IR instructions that can be packed into (possibly non-SIMD) vector instructions—we call such a group of instructions a *vector pack*—and then (2) lowers the vector packs into target vector instructions.

Vector Pack. A *pack* is a tuple $\langle v, [m_1, \dots, m_k] \rangle$, where v is a vector instruction with k output lanes, and $m_1, \dots, m_k$ are a list of matches whose live-outs are independent. For example, let m_1 and m_2 be the two matched integer multiply-add operations rooted at the instructions t_1 and t_2 in Figure 5-6, we can use the instruction `pmaddwd`

to combine them into a single vector pack:

$$p_{ex} = \langle pmaddwd, [m_1, m_2] \rangle$$

VEGEN models vector loads and stores as two special kinds of packs, whose memory addresses must be contiguous.

We define two notations for vector packs. Let $p = \langle v, [m_1, \dots, m_k] \rangle$ be a vector pack, then $values(p)$ is the list of IR values produced by pack p (i.e., $values(p)_i = live-out(m_i)$) and $opcode(p) = v$. In the running example,

$$values(p_{ex}) = [t_1, t_2]$$

$$opcode(p_{ex}) = pmaddwd$$

Vector Operand. Vector packs have vector operands, represented as lists of IR values. In the running example, p_{ex} has two vector operands (We overload the $[.]$ operator here; e.g., $A[0]$ denotes a load of the first element of A):

$$operand_1(p_{ex}) = [A[0], A[1], A[2], A[3]]$$

$$operand_2(p_{ex}) = [B[0], B[1], B[2], B[3]]$$

More specifically, let $p = \langle v, [m_1, \dots, m_k] \rangle$ be a vector pack, then $operand_i(p) = [x_1, \dots, x_n]$; where $x_j \in \bigcup_k live-ins(m_k)$ is one of the live-ins of the matches that should bind to the j 'th lane of the i 'th operand of the vector instruction v . VEGEN generates the implementation of $operand_i(.)$ automatically from instruction semantics; $operand_i(.)$ is known statically because the VIDL only allows selecting input vector lanes using constant indices.

Don't-Care Lanes. Some instructions don't use all of their input lanes. For example, the SSE4 instruction `vpmuldq` (Figure 5-9) sign-extends and multiplies *only* the odd input lanes. To handle a case, we introduce a special *don't-care* value. Each element of a vector operand (i.e., $operand_i(.)$) therefore takes the value of either a scalar IR

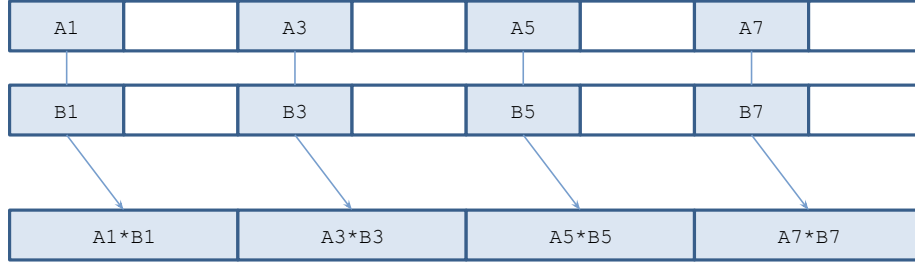


Figure 5-9: Semantics of `vpmuldq` (sign-extended integer multiplication). White cells represent lanes unused by the instruction.

value (from the input program) or *don't-care*.

Producing a Vector Operand. A pack p produces a vector operand x if they have the same size (i.e., $|values(p)| = |x|$) and, for every lane i , x_i is either $values(p)_i$ or *don't-care*. Algorithm 1 shows the algorithm for finding the set of feasible producer packs for a given vector operand x . VEGEN uses a separate routine to enumerate producer packs that are vector loads, which can be done efficiently because only contiguous loads can be packed together.

Dependence and Legality. A pack p_1 depends on another pack p_2 if there exists an instruction $i \in values(p_1)$ that depends on another instruction $j \in values(p_2)$. We define the dependencies among scalar IR instructions and vector packs similarly. A set of packs are legal when there are no cycles in the dependence graph.

Vector Pack Selection. Because lowering a given set of vector packs to target vector instructions is relatively straightforward, vectorization reduces to finding a subset of the matches and combining them into *legal* vector packs. The choice of packs determines the performance of the generated code by affecting the level of parallelism and the level of data-movement overhead (e.g., if a vector operand is not produced directly, VEGEN needs to use vector shuffles to gather the elements of the operand). Given a scalar program, VEGEN selects a set of profitable vector packs using two alternative heuristics that we will discuss in Chapter 5.6.

Algorithm 1: Algorithm for finding the set of (non-load) packs that produce a given vector operand x . Load packs are found separately by enumeration.

Input :

x : The vector operand that we need to produce
 M : The match table, which contains the mapping
 $\langle \text{live-out}(m), \text{operation}(m) \rangle \mapsto m$ for each match m .
 I : A list of instruction descriptions.

Output : A (potentially empty) set of producer packs of x .

```

1 if there are dependent values in  $x$  then
2   | return {}
3 end
4  $producers \leftarrow \{\}$ 
5 for  $vinst \in I$  do
6   |  $matches \leftarrow []$ 
7   | for  $i \leftarrow 1$  to number of lanes of  $vinst$  do
8   |   |  $f \leftarrow$  the  $i$ 'th operation of  $vinst$ 
9   |   |  $m \leftarrow M[\langle x_i, f \rangle]$ 
10  |   | if  $x_i$  is don't-care or  $m$  is not null then
11  |   |   | append  $m$  to  $matches$ 
12  |   | end
13  | end
14  | if  $|matches| =$  number of lanes of  $vinst$  then
15  |   |  $producers \leftarrow producers \cup \text{pack}(vinst, matches)$ 
16  | end
17 end
18 return  $producers$ 

```

5.5.5 Code Generation

Given a set of vector packs (and the input program), VEGEN’s code generator emits a vector program as a combination of (1) the scalar instructions not covered by the packs, (2) the *compute* vector instructions corresponding to the packs, and (3) the *data-movement* vector instructions that follow from the dependence among the packs and scalars.

Given a pack set P , we generate vector code as follows. The code generation algorithm uses the target-specific functions $operand_i(.)$ generated from instruction semantics.

Scheduling. The code generator first schedules the *scalar* instructions (regardless of whether an instruction is replaced by vector instructions) according to their dependencies and the following constraint: For any pack $p \in P$, all instructions in $values(p)$ are grouped together in the final schedule. Such a schedule exists when the set of packs are legal.

Lowering. After scheduling, the code generator lowers the packs in P in topological order. The previous scheduling step ensures that all of the values in $operand_i(p)$ are ready by the time we lower any $p \in P$. The code generator also emits any required swizzle instructions to gather a vector operand if the operand is not produced directly by another pack and to extract an element of a vector pack if the pack has a scalar user.

5.6 Vector Pack Selection

VEGEN uses a target-independent heuristic to select a set of profitable vector packs. The goal of the heuristic is to select a set of packs to maximize the total saving from vectorization while minimizing the overhead of explicit data-movement that is necessary when an instruction (whether vector or scalar) operand is not produced exactly by any other instruction—such as when a scalar instruction uses a vector element and therefore requires a vector extraction.

$$cost_{SLP}(v) = \min \begin{cases} \min_{p \in producers(v)} cost_{op}(opcode(p)) & \text{Produce a vector directly.} \\ \quad + \sum_i cost_{SLP}(operand_i(p)) \\ C_{broadcast} + cost_{scalar}(v_0) & \text{If } \forall i. v_i = v_0, \text{ try broadcast} \\ C_{insert} \cdot |v| + \sum_{x \in v} cost_{scalar}(x) & \text{Produce each element as scalar} \end{cases}$$

Figure 5-10: The SLP heuristic uses this recurrence to decide whether to produce a vector operand v directly via a vector pack or by vector insertions. $cost_{scalar}(x)$ is the cost of producing the (scalar) value x and its dependences using only scalar instructions.

Optimization Objective and Cost Model. Let P be the set of selected vector packs, and let us focus on one of the packs $p \in P$. If the results of p are used by some scalar instructions, we need to extract those values and pay the following cost:

$$C_{extract} \cdot |values(p) \cap scalarUses|$$

Let v be a vector operand of p . When a subset of v is produced by some other pack $p' \neq p$, we need to use vector shuffles to move those values into v and pay the following cost:

$$C_{shuffle} \cdot |\{p' \in P \setminus \{p\} \mid v \cap values(p') \neq \emptyset\}|$$

When some elements of v are produced by scalar instructions, we need to use vector insertions to insert those values into v and pay the following cost:

$$C_{insert} \cdot |v \setminus [\bigcup_{p' \in P} values(p')]|$$

$C_{extract}$, C , and C_{insert} are cost-model parameters.

Recall that VIDL doesn't model vector shuffles (Chapter 5.5.1). VEGEN's code generator therefore emits a mix of target vector instructions and virtual (target-independent) vector shuffles and relies on LLVM's backend to lower the shuffles.

5.6.1 Pack Selection Using the SLP heuristic

The SLP heuristic builds a set of vector packs by traversing the instruction DAG bottom-up (uses before definitions). Initially, the set of packs are seeded with *seed* packs such as chains of contiguous stores. The heuristic then recursively introduces vector packs to produce the vector operands—VEGEN uses Algorithm 1 to find such producers—in the current set of packs.

There are often multiple vector packs that can produce a given operand. For a given operand, VEGEN uses the dynamic programming algorithm shown in Figure 5-10 to choose a producer. This is the main modification we added to the original SLP algorithm—in SLP-based vectorization, there is at most one pack that can produce any given operand.

Limitations. A major limitation of the SLP heuristic is that it only considers a vector pack p if p directly produces the operands of another pack. And there are real-world cases where it's more profitable to include a vector pack even when its result is not directly used by any other vector packs. Consider the following code that swaps the first two elements of an array.

```
b[0] = a[1];  
b[1] = a[0];
```

Notice that we can vectorize this program (manually) by first loading the a elements with two-wide vector load, swap the first and second elements with a vector shuffle, and finally store the shuffled two-wide vector to b . However, we could never generate this code sequence with the SLP heuristic because the vector load of a (which produces the vector $[a_0, a_1]$) doesn't produce the vector operand used by the vector store to a (which requires the vector $[a_1, a_0]$).

Another limitation of the SLP heuristic it assumes that each program value only has one user. Consequently, it is optimistic when there are external scalar users of a vector pack and fails to account for the vector extraction cost. On the other hand, the SLP heuristic is also pessimistic when there are multiple uses of non-vectorizable

vector operands and fails to recognize that the multiple uses lower the cost of vector shuffle/insertion (by amortization).

Consider the following code snippet, where there are two seed packs: the two pairs of stores to the arrays **a** and **b**.

```
a[0] = x[0] + t1; a[1] = x[1] + t2;
b[0] = y[0] + t1; b[1] = y[0] + t2;
```

Suppose the temporaries **t1** and **t2** are not vectorizable. To vectorize the rest of the code snippet, the vectorizer would need to emit extra vector insertion instructions to create the vector $[t_1, t_2]$. On a machine where vector insertions are expensive, it is plausible that this code is profitable to vectorize *only* when the instruction (sub-)DAG rooted at both seed packs are vectorized to amortize the cost of creating $[t_1, t_2]$. Unfortunately, because the SLP heuristic processes each seed pack separately, it would (correctly) conclude that none of the seed packs are individually profitable and (incorrectly) decide that the whole basic block is not worth vectorizing.

Improving the SLP heuristic. Let’s rephrase the problem—that the SLP heuristic only consider vector packs that are direct producers of existing vector operands—in another way helps with understanding our new heuristic. Let v be some vector operand that we need to produce. We want to find some profitable vector pack p so that p produces v partially ($values(p) \cap v \neq \emptyset$). Note that in the original SLP heuristic, we insist that $values(p) = v$, which is a stricter condition. For such vector pack p , we can break it down to three cases that we will address separately.

Here are the three possible (disjoint) cases that can happen when we have a vector pack p that produces a subset of v ’s elements.

1. p produces a proper subset of v .
2. p produces exactly the v ’s elements but with the elements permuted.
3. p produces some values that are not in v . One might wonder why is such pack p desirable: why would anyone want to produce more than that’s required? The

answer is that sometimes it's more efficient produce multiple vectors jointly as a single vector and then use vector shuffles to extract the sub-vectors out.

The first case requires us to decompose v into smaller vectors and find a producer packs for those vectors separately. Because there are combinatorially many ways to decompose a given vector, we instead use a limited family of vector decomposition operator that we call *deinterleaving*.

The second case requires us to find a way to permute v and then a producer pack for the permuted vector. Similarly, because there are factorially many ways to permute any given vector, we instead use a limited set of permutations that we call *transposition*.

For the third case, we apply a heuristic that we call *combination*, in which we look at all possible pairs of vector operands in an existing set of packs and check whether it's more profitable to produce the elements of the two vectors simultaneously with a single instruction (note that we still have much freedom in choosing how to combine those vectors, and different *combination* methods can give different performance).

Deinterleaving and *transposition* are non-intrusive in the sense that they still follow the high-level idea of the original SLP heuristic; that is, at each step, we inspect the vector operand x of some existing vector pack p , and we try to find another vector pack p' to produce x . This allows us to reuse the general struture of the dynamic programming algorithm shown in Figure 5-10 with some modifications.

The *combination* heuristic is more intrusive and requires a separate, more expensive optimization algorithm.

5.6.2 Improve the SLP heuristic with Deinterleaving and Transposition

The idea of *deinterleaving* is that for a given vector x , we decompose x into multiple sub-vectors by a constant stride, and then we attempt to find a producer pack p' that produces one of the sub-vectors exactly. For example, for a four-wide vector x , we can

deinterleave its even and odd elements as follows.

$$\begin{aligned}x' &= [x_0, x_2, x_4, x_6] \\x'' &= [x_1, x_3, x_5, x_7]\end{aligned}$$

And then we recursively find vector packs that can produce x' and x'' . In general, we define *deinterleave*(m, n, x) as a function that takes integers m (the stride) and n (the start position and $1 \leq n \leq m$), an input x , and returns another vector y whose individual elements are $y_i = x_{m+i \cdot n}$.

With *transposition*, we first transform the an input vector x by interpreting it as an $m \times \frac{|x|}{m}$ matrix (assuming $|x|$ is divisible by m), transposing that matrix, and re-interpreting it as the transformed vector x' . We then attempt to find a producer pack p' that produces x' .

Figure 5-11 shows the improved version of the SLP heuristic once we apply both *deinterleaving* and *transposition*.

5.6.3 Fine-tuning with Vector Combination

Combination is an improvement heuristic that we can apply to improve an existing set of vector packs (but doesn't directly give us a set of vector packs directly in the first case). It can give better performance in some cases (by trading off more compile time). The idea is to look at all of the vector operands used in an existing set of vector packs, and find some pair of vector operands that are more profitable to be produced jointly. This leaves two problem: 1) Which pair of vector operands? 2) And how to produce them jointly?

For the first problem, the *combination* heuristic simply checks all pairs of vector operands by following steps iteratively until it reaches an fixed-point:

1. Enumerate all possible pairs of vector operands (v, v') s.t. elements of v and v' are independent and $|v| = |v'|$, in some arbitrary order
2. Combine each pair of vectors v and v' into a larger vector *combine*(v, v') (I will discuss what the *combine* function is).

$$\text{cost}_{\text{SLP}}(v) = \min \left\{ \begin{array}{ll}
\min_{p \in \text{producers}(v)} \text{cost}_{op}(\text{opcode}(p)) & \text{Produce a vector directly.} \\
+ \sum_i \text{cost}_{\text{SLP}}(\text{operand}_i(p)) & \\
\\
\min_{m \in \{2,4,8,\dots\}} \min_{p \in \text{producers}(\text{transpose}_m(v))} \text{cost}_{op}(\text{opcode}(p)) & \text{Transposition} \\
+ \sum_i \text{cost}_{\text{SLP}}(\text{operand}_i(p)) & \\
+ C_{\text{shuffle}} & \\
\\
\min_{m \in \{2,4,8,\dots\}} \sum_{n \leq m} \text{cost}_{\text{SLP}}(\text{deinterleave}(m, n, v)) & \text{Deinterleaving} \\
+ (m - 1) \cdot C_{\text{shuffle}} & \\
\\
C_{\text{broadcast}} + \text{cost}_{\text{scalar}}(v_0) & \text{If } \forall i. v_i = v_0, \text{ try broadcast} \\
\\
C_{\text{insert}} \cdot |v| + \sum_{x \in v} \text{cost}_{\text{scalar}}(x) & \text{Scalar insertions}
\end{array} \right.$$

Figure 5-11: Improved recurrence for choosing a vector producer for a given vector operand. This is essentially of the algorithm in Figure 5-10 with extra rules for deinterleaving and transposing vectors before looking for the vectors' producers. Note that deinterleaving and transposing are not free. For deinterleaving, we need to use $m - 1$ vector shuffles to combine the deinterleaved vectors to produce the final vector. For transposition, we need to use a vector shuffle to permute the transposed vector to produce the final vector.

3. Apply the SLP heuristic to find a new set of vector packs that produces the vector $combine(v, v')$ and its transitive dependences; if any of the new vector packs overlap with an existing one, discard the existing one.
4. If the new set of vector packs are more profitable, accept and go back to step 1.

We need some way to combine two vectors into a larger vectors. For this, we use two alternative combination functions—*concat* and *interleave*—and pick whichever that gives better performance. The function *concat*($\cdot, \cdot$) concatenates its two input vectors. The function *interleave*($\cdot, \cdot$) combines into two input vectors so that the odd elements of the final vector come from the first vector and the even elements come from the second.

5.7 Implementation

5.7.1 Supporting x86 Vector Instructions

We implemented the offline part of VEGEN (the part involved with semantics and pattern generation) in Python. We implemented the rest of VEGEN, the part that performs compile time vectorization, as an LLVM pass in C++. The LLVM pass takes scalar LLVM IR as input and emits a mix of scalar IR and target-specific intrinsics¹ that in most cases, gets lowered to their corresponding instructions (e.g., the LLVM intrinsic `@llvm.x86.sse2.pmadd.wd` maps to the instruction `pmaddwd`).

5.7.2 Target Instruction Specification

VEGEN generates SMT formulas from the XML file that Intel uses to render the Intrinsics Guide [Intel Corporation, 2012], which contains pseudocode documentations of the intrinsics. VEGEN then lifts the SMT formulas to VIDL (vector instruction description language). Lifting the SMT formulas to VIDL is straightforward because

¹There is a straightforward mapping from Intel intrinsics to small sequences of LLVM intrinsics. We find out the mapping from Intel intrinsics to the equivalent LLVM intrinsics by wrapping an intel intrinsic in a standalone function whose signature matches that of the intrinsic. We run Clang on this function, and record the instructions produced by Clang

we designed VIDL to closely match the semantics of SMT bit-vector operations (which are also closely related to LLVM’s integer instructions).

Translating Semantics from the Intrinsic Guide. To document instruction semantics, Intel uses an imperative language that operates on fixed-length bit-vectors. All values in the language are bit-vectors and have one of four types: signed integer, unsigned integer, float, and double. There are no implicit integer overflows in this language; instead, if an operation can overflow its result (such addition and multiplication), the operation first converts its input bit-vectors to a wider width—using zero- or sign-extensions, depending on the signedness—before execution.

We implemented a symbolic evaluator for the language using z3 [De Moura and Bjørner, 2008] and translated Intel’s pseudocode documentation into formal SMT formulas. We chose z3 mostly for its expression simplifier. The evaluator maps expression-level constructs such as ALU operators and bit-vector slicing to their SMT equivalents; for instance, additions become SMT bit-vector additions. We treat the following high-level program constructs specially:

- *Assignment.* We model each assignment to (sub-)bit-vector as a pure expression that takes the original bit-vector value and outputs the post-update value. The output of the expression is a concatenation of the unaffected sub-vector(s) and the updated sub-vector.

Consider, for example, the statement $x[7:0] = 0$, which zeros the lower eight bits of a 32-bit variable x , we emit the following formula:

```
Concat(Extract(31,8,x), 0b00000000)
```

- *Function calls.* We inline all function calls.
- *Loops.* We unroll all for-loop (All for-loops have constant trip-counts in the documentation language).
- *If-statements.* We apply if-conversion to the sub-vector being mutated—bit-vector assignment is the only construct with side-effects. In the if-converted

expression, we set the predicate to the condition of the original if-statement, the true-branch to the right-hand side of the assignment, and the false-branch to the original value of the sub-vector.

For example, for the following statement, which conditionally zeros the lower eight bits of a 32-bit variable `x`,

```
IF ctrl[1:0]
    x[7:0] = 0
FI
```

we emit the following formula:

```
Concat(Extract(31,8,x),
       If(Extract(0,0,ctrl) == 1,
          Extract(7,0,x),
          0b00000000))
```

Our symbolic evaluator returns SMT formulas that are unnecessarily complicated in some cases because of the naive implementation of partial bit-vector updates and predicated updates. We use `z3`'s simplifier to reduce the formula complexity. For most instructions, `z3`'s simplifier simplifies their symbolic results into representations that reflect the high-level intent of the original documentation.

We validated the SMT formulas by random testing. Testing revealed incorrect semantics resulting from ambiguous or simply incorrect documentation. For instance, the signedness of saturation arithmetic is particularly ambiguously documented for instructions from the `psubus` family (subtract packed unsigned integers with saturation). It turns out the result of an *unsigned* subtraction should be saturated as a *signed* integer.

Pattern Generation. We use LLVM's pattern-matching library to implement VEGEN's pattern matching logic. VEGEN canonicalizes the patterns before emitting the pattern matchers. The canonicalizer takes a pattern and generates an LLVM function that has the same signature as the operation. We then run LLVM's `instcombine`

pass on this function and generate pattern matching code according to the final canonicalized IR sequences. This canonicalization biases the patterns toward patterns that LLVM prefers. The most notable rewrite is canonicalizing all comparisons to strict inequalities (such as rewriting $x \leq 1$ to $x < 2$) and is crucial for recognizing integer saturations. Additionally, for (sub-)patterns of the form **select(cmp(a, b), x, y)**, we generate additional code to also match the inverted case of the comparison.

5.7.3 Cost Model

For C_{insert} and $C_{extract}$, we use LLVM’s cost model. We set $C_{shuffle} = 2$. VEGEN additionally detects several special-case vector shuffle and insertion patterns, such as vector broadcast and permutation, and overrides the default cost model.

To estimate the cost of vector instructions, we use the instruction throughput statistics from Intrinsics Guide. To remain compatible with the rest of LLVM’s cost model, we set the cost of each intrinsic to be its inverse throughput scaled by a factor of two.

One of VEGEN’s goals is reducing developer effort. It is important for us to account for the engineering effort as well as which LLVM components VEGEN depends on. We implemented the semantics components of VEGEN — parsing, symbolic execution, pattern generation, etc. — in 5.5k lines of Python. We implemented the pack selection heuristic with 489 lines of C++. We implemented the rest of the infrastructure with 2.6k lines of C++.

For reference, the closest analog of our vectorizer is LLVM’s SLP vectorizer, which takes 7.5k lines of C++. The hand-written backend peephole rule of the SSE4 instruction **pmaddwd** alone takes 129 lines of C++.

VEGEN uses the LLVM’s target-specific selection DAG builder to lower virtual vector shuffles into hardware instructions, the backend to lower vector intrinsics into hardware instructions, and its cost model to determine the cost of scalar arithmetic instructions and vector shuffles.

x86 Specific Engineering. Most of the aforementioned implementation is target-

independent. The only target-specific component is the semantics translator from Intel’s documentation to our description language. This translator automatically translated the semantics of 1164 vector intrinsics and is written in 1.1k lines of Python. To target a new vector instruction set, one would only need to supply the semantic description of the instruction set. The main dependence is LLVM’s shuffle lowering machinery, since (in general) there’s a straightforward mapping from vector intrinsic to hardware instructions.²

5.7.4 Supporting Arm Neon Instructions

VEGEN also has support for a limited subset of Arm’s Neon instructions. Rather than using an automatic translator of Arm’s instructions (i.e., following what we did for x86), we translated the semantics of these Neon instructions with a semi-automatic approach. Most Neon instructions come from a small family of instructions (e.g., vector add) but parametrized with different vector-length and bit-width (e.g., 4-wide 32-bit vector add), we implemented a semantics generator for various families of vector instructions and instantiating them with the vector-lengths and bit-widths that Arm supports.

We emit SMT formulas with this semantics generator. We then reused the same code-generation pipeline as we do for x86 vector instructions to process the generated SMT formulas.

All told, we added support for 152 Neon instructions. Table 5.1 lists the non-SIMD instructions in this subset of supported instructions. There are more Neon instructions whose semantics we could generate but not included here because we could not find available Arm machines that can execute those instructions (e.g., matrix multiplications).

Because Arm doesn’t have a publicly available, machine-readable file detailing the throughput and latencies of their instructions (unlike Intel), VEGEN assigns a uniform

²Our implementation actually emits target-independent virtual instructions for common arithmetic operations such as addition, which do not have dedicated LLVM intrinsic. For instance, Clang directly emits virtual vector addition for the compiler intrinsic `_mm_add_epi32`.

Table 5.1: Summary of the non-SIMD Neon instructions that VEGEN supports

Category	Semantics Summary	Number of Instructions
Pairwise addition	Adding every pair of lanes	41
Folding min/max	Take the maximum of every pair of lanes	14
Dot product	-	4

cost of two for all the Neon instructions that it supports.

5.8 Experimental Results

We evaluated VEGEN on a subset of LLVM’s vector instruction selection tests, some reference DSP kernels chosen from FFmpeg and x265, and fixed-size dot-product kernels from OpenCV. We also evaluated VEGEN’s generated Arm Neon backend on the aforementioned DSP kernels. We show that in most cases, VEGEN outperforms LLVM’s vectorizer, and we explain how VEGEN fails to vectorize in the other cases. Additionally, we present a case study of VEGEN vectorizing the scalar complex-multiplication kernel.

Experimental Platforms. For experiments requiring only AVX2, we run the benchmarks on a server with the Intel®Xeon®CPU E5-2680 v3 CPU and 128 GB of memory. For experiments requiring AVX512-VNNI, we use a server with the Intel®Xeon®Platinum 8275CL CPU and 4 GB of memory. For experiments requiring Arm Neon instructions, we used an AWS Graviton2 processor (based on the Arm Neoverse N1 microarchitecture). We use LLVM 12.0.0. In all cases, we invoke clang with `-O3 -ffast-math -march=native`.

Note on cost model. During development, we discovered that LLVM’s backend code-generator for LLVM `vectorshuffle` (LLVM’s target-independent construct to represent arbitrary vector shuffles) are incomplete and crashes lowering the vector shuffles emitted by VEGEN while targeting Icelake (the microarchitecture of our AVX512 machine). We got around the bug by configuring LLVM to target Haswell instead but with extra flags informing LLVM that the (purely hypothetical) Haswell is

Test	Speedup	Test	Speedup
max_pd	1.0	hadd_pd	1.4
min_pd	1.0	hadd_ps	1.2
max_ps	1.0	hsub_pd	1.4
min_ps	1.0	hsub_ps	1.2
mul_addsub_pd	1.0	hadd_i16	2.9
mul_addsub_ps	1.0	hsub_i16	4.9
abs_pd	0.8	hadd_i32	1.3
abs_ps	0.4	hsub_i32	1.3
abs_i8	1.0	pmaddubs	16.8
abs_i16	1.0	pmaddwd	4.2
abs_i32	1.0		

(a) Tests LLVM able to vectorize (b) Tests LLVM unable to vectorize

Figure 5-12: Speedup (over LLVM, higher is better) on instruction selection tests ported from LLVM’s x86 backend. These tests were originally written to exercise the pass that lowers LLVM’s vector IR into their desired target instructions. We ported the tests by manually transforming them into their scalar equivalents.

also equipped with AVX512 features. This allows us to get emit AVX512 instructions but has unfortunately also caused LLVM to use its Haswell cost model even while targeting the Icelake machine. As we analyze the experimental results, we can see evidence that this has an adverse performance effect.

5.8.1 Synthetic Benchmarks

For our first set of experiments, we ported some of LLVM’s backend instruction selection tests for non-SIMD instructions and SIMD instructions with complex semantics (e.g., `min`). These tests were originally written to exercise the pass that lowers LLVM vector IR into target vector instructions. Because LLVM’s vector IR only models isomorphic vector instructions, the tests for non-SIMD instructions (e.g., `haddpd`) are written as combinations of LLVM vector instructions and vector shuffles. We translated the test cases (written in LLVM IR) to their equivalent scalar version by expanding IR vector instructions into multiple scalar instructions and by converting vector function arguments to non-aliased pointer arguments. Clang/LLVM is not able to vectorize 10 out of 21 of the tests. Aside from the tests `abs_ps` and `abs_pd`, our system performs

Table 5.2: Evaluation Benchmarks

Benchmark	Description
g722-qmf	Quadrature Mirror Filters for G.722 codec
fft4	4-point Fast Fourier Transform
fft8	8-point Fast Fourier Transform
sbc	Audio subband codec
chroma	Chroma subsampling
idct4	4-point Inverse Discrete Cosine Transform
idct8	8-point Inverse Discrete Cosine Transform
imdct36	36-point Modified Discrete Cosine Transform

equally or better to Clang.

Figure 5-12 shows the test results. VEGEN vectorizes 19 out of 21 of the tests. LLVM fails to vectorize 10 out of 21 of the tests, all of which are non-SIMD instructions and are vectorized by VEGEN. Interestingly, the only non-SIMD tests that LLVM can vectorize are `mul_addsub_pd` and `mul_addsub_ps`, for which LLVM does have special-case support.

Both of the two tests that VEGEN failed to vectorize compute floating-point absolute values, and for which LLVM uses the fact that the absolute value of a floating-point can be computed by masking-off the sign-bit (i.e., the most significant bit) to vectorize; VEGEN does not have this knowledge and does not vectorize in these two cases.

5.8.2 Optimizing Image and Signal Processing Kernels

Table 5.2 shows our benchmarks. To demonstrate that VEGEN can effectively use non-SIMD instructions on real-world kernels, we evaluated VEGEN’s pack selection heuristic on six kernels from x265. We chose these kernels because DSP and image processing are the motivating domains for non-SIMD instructions such as `pmaddwd`. These benchmarks are challenging to vectorize because they require intermediate shuffles and partial reductions. We ported the `idct4` and `idct8` kernels from x265’s reference implementation. The rest are from FFmpeg.

Figure 5-13 shows the benchmarking results on AVX2, AVX512, and (Arm) Neon.

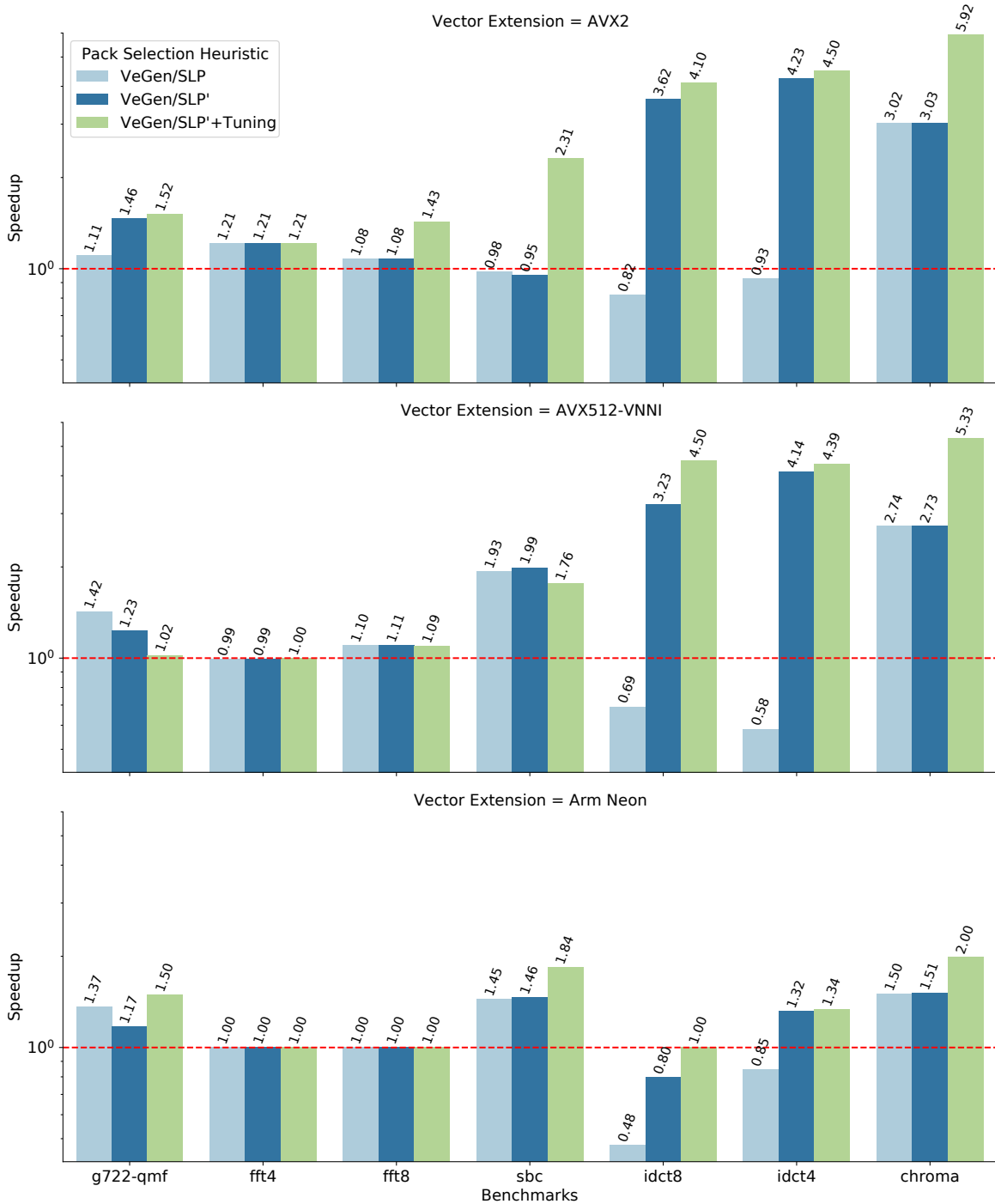


Figure 5-13: Speedup (over LLVM, higher is better) on kernels we selected from x265 (idct4 and idct8) and FFmpeg. **VEGEN/SLP** is the baseline SLP heuristic used by **VEGEN**. **VEGEN/SLP'** is the SLP heuristic augmented with the *deinterleaving* and *transposition* heuristics. **VEGEN/SLP'+Tuning** refers to running the augmented SLP heuristic and then fine-tuning with the *combination* heuristic, which iteratively finds pairs of vector operands that are more profitable to be produced jointly.

```

vmovdqu ymm0, ymmword ptr [rdi]
vpermq ymm1, ymm0, 119
vmovdqa ymm2, ymmword ptr [rip + LCPI7_0]
vpshufb ymm1, ymm1, ymm2
vpmaddwd ymm1, ymm1, ymmword ptr [rip + LCPI7_1]
vpermq ymm0, ymm0, 34
vpshufb ymm0, ymm0, ymm2
vpmaddwd ymm0, ymm0, ymmword ptr [rip + LCPI7_2]
vpaddb ymm2, ymm0, ymm1
vpbroadcastd ymm3, dword ptr [rip + LCPI7_3]
vpaddb ymm2, ymm2, ymm3
vpsrad ymm2, ymm2, 7
vpshufd ymm2, ymm2, 78
vpsubd ymm0, ymm0, ymm1
vpaddb ymm0, ymm0, ymm3
vpsrad ymm0, ymm0, 7
vpckssdw ymm0, ymm2, ymm0
vpshufb ymm0, ymm0, ymmword ptr [rip + LCPI7_4]
vpermq ymm0, ymm0, 216
vmovdqu ymmword ptr [rsi], ymm0

```

Figure 5-14: idct4 vectorized by VEGEN. VEGEN fully vectorizes this kernel with 20 vector instructions. In contrast, Clang partially vectorizes this kernel using 84 instructions (not shown here).

VEGEN outperforms LLVM in all cases.

5.8.3 Discussion

When considering the best-performing heuristics used by VEGEN, VEGEN outperforms Clang in all cases; in the best case, VEGEN is able to reach more than $5\times$ speedup.

The baseline SLP heuristic is in general inferior to the SLP heuristic augmented with the *deinterleaving* and *transposition* heuristic, incurring slowdowns on idct4 and idct8 on all three (micro)architectures.

Aside from the g722-qmf benchmark, the SLP heuristic augmented with *deinterleaving* and *transposition* outperforms the baseline SLP heuristic. For the g722-qmf benchmark, the augmented SLP heuristic and finetuning found vectorization strategies that are actually more profitable according to LLVM’s cost model, but the actual

generated to runs slower, indicating inaccuracy in the cost model. The main benefit *deinterleaving* and *transposition* show up in the idct4 and idct8 benchmarks, both of which require shuffling the memory inputs before vector computations and require shuffling the computed value before storing back to the memory.

Fine-tuning with the *combination* heuristic (**VEGEN/SLP'+Tuning** in Figure 5-13) improves the initial vectorization strategy discovered by the augmented SLP heuristic on idct4, idct8, and the chroma benchmarks. It improves augmented heuristic by more than 20%.

Vectorizing idct4. We highlight some instructions that VEGEN generated for the idct4 kernel (targeting AVX512-VNNI). Figure 5-14 shows the generated code, which is more than 4 $\times$ faster than LLVM’s code. VEGEN uses the non-IMSD instructions `vpaddwd` (the motivating dot-product instruction) and `vpackssdw` (saturate two vectors of 32-bit integers to 16-bit integers and concatenate the result). Of note are the `vpshufb` and `permq` instructions preceding the vector stores. VEGEN uses these shuffle instructions—without which it is not profitable to vectorize this kernel—to form vector operands that are not directly produced by *compute* instructions such as `vpaddwd`.

5.8.4 Optimizing OpenCV’s Dot-Product Kernels

For our next set of experiments, we evaluated VEGEN on OpenCV’s reference dot-product kernel implementations. OpenCV’s reference implementation is a C++ template parameterized with different data types and kernel sizes. These kernels are challenging to auto-vectorize because they have interleaved memory accesses as well as reduction.

Figure 5-15 shows the benchmarking results. VEGEN found non-trivial vectorization schemes for three of the four kernels. VEGEN vectorizes the first benchmark naively—essentially vectorizing across the unrolled iterations and paying the shuffle cost for the interleaved accesses—and only yielded a 10% speedup. We investigated the slowdown VEGEN incurred on AVX512 (VNNI). It turned out that for the first kernel, VEGEN

Kernel Size	Speedup	Kernel Size	Speedup
$int8 \times 32$	1.1	$int8 \times 32$	0.7
$uint8 \times 32$	2.0	$uint8 \times 32$	2.2
$int32 \times 8$	1.5	$int32 \times 8$	1.7
$int16 \times 16$	1.6	$int16 \times 16$	2.5

(a) Results on AVX2

(b) Results on AVX-512 (VNNI)

Figure 5-15: OpenCV’s dot-product kernels specialized for AVX2 and AVX-512 (VNNI) and different kernel sizes.

actually emitted identical vector IR/intrinsics for both AVX2 and AVX-512. The performance difference comes down to how LLVM’s backend lowered the shuffles emitted by VEGEN. For the AVX2, LLVM emitted the `vpshufb` instruction, whose latency and inverse throughput are both one cycle. For the AVX-512, LLVM instead emitted the `vpmovdb` instruction, whose inverse throughput is two cycles (and latency four cycles) and slower than `vpshufb`.

Of note is the vector code VEGEN generated for the $int32 \times 8$ kernel (Figure 5-16), which matches OpenCV’s expert-optimized code. We inspected the machine code and confirmed that VEGEN used the same high-level algorithm used by OpenCV’s expert developer. The reference (naive) implementation of the $int32 \times 8$ kernel sign-extends the input elements from 32-bit to 64-bit, multiplies the two input arrays elementwise, and then reduces every adjacent pair of elements by addition. There is no single instruction that can implement this kernel by itself, and the high-level strategy of VEGEN (and OpenCV) is to perform the odd multiplications separate from the even ones and finally add the odd and even entries together. To multiply the odd (and even) entries, VEGEN uses the instruction `vpmulldq`, which is deceptively complicated and performs sign-extended multiplications only on the *odd* input elements (Figure 5-9). The multiplications of the odd elements therefore map naturally to `vpmulldq`.

5.8.5 Optimizing Complex Multiplication

Complex arithmetic is a motivating application for SIMOMD instructions. In fact, (to the best knowledge of our knowledge) the first SIMOMD instructions were designed

```

vmovdqu ymm0, [rdi]
vmovdqu ymm1, [rsi]
vpmuldq ymm2, ymm1, ymm0
vpshufd ymm0, ymm0, 245    ## ymm0 = ymm0[1,1,3,3,5,5,7,7]
vpshufd ymm1, ymm1, 245    ## ymm1 = ymm1[1,1,3,3,5,5,7,7]
vpmuldq ymm0, ymm1, ymm0
vpaddq ymm0, ymm0, ymm2
vmovdqu [rdx], ymm0

```

Figure 5-16: Vector code that VEGEN generated for the $int32 \times 8$ dot-product kernel in OpenCV. `vpmuldq` multiplies (with sign-extension) the *odd* elements of its two vector operands.

<code>vmovupd</code>	<code>xmm0, rsi</code>	<code>vmovsd</code>	<code>xmm0, [rdi]</code>
<code>vpermilpd</code>	<code>xmm1, xmm0, 1</code>	<code>vmovsd</code>	<code>xmm1, [rdi + 8]</code>
<code>vmovddup</code>	<code>xmm2, [rdi+8]</code>	<code>vmovsd</code>	<code>xmm2, [rsi]</code>
<code>vmulpd</code>	<code>xmm1, xmm1, xmm2</code>	<code>vmovsd</code>	<code>xmm3, [rsi + 8]</code>
<code>vmovddup</code>	<code>xmm2, [rdi]</code>	<code>vmulsd</code>	<code>xmm4, xmm2, xmm1</code>
<code>vfmaddsub213pd</code>	<code>xmm2, xmm0, xmm1</code>	<code>vfnadd231sd</code>	<code>xmm4, xmm3, xmm0</code>
<code>vmovupd</code>	<code>[rdx], xmm2</code>	<code>vmulsd</code>	<code>xmm1, xmm3, xmm1</code>
		<code>vfmsub231sd</code>	<code>xmm1, xmm2, xmm0</code>
		<code>vmovsd</code>	<code>[rdx], xmm1</code>
		<code>vmovsd</code>	<code>[rdx + 8], xmm4</code>

(a) Instructions generated by VEGEN (`vfmaddsub213pd` does multiply-add on the odd lanes and multiply-sub on the even lanes)

(b) Instructions generated by LLVM

Figure 5-17: Complex multiplication kernel, generated by VEGEN (Figure 5-17a) and LLVM (Figure 5-17b). VEGEN's version is $1.27\times$ faster.

for complex arithmetic [Bachega et al., 2004].

Figure 5-17 shows the complex multiplication kernel compiled by VEGEN and by LLVM. VEGEN uses the instruction `vmaddsub213pd` (which performs fused multiply-add on the odd lanes and multiply-sub on the even lanes). LLVM does not vectorize in this case, even though (as noted earlier) LLVM’s SLP vectorizer has been specifically modified to support such a pattern. We stepped through the LLVM’s optimization decisions and discovered that the root cause is an error in its cost-benefit analysis. Since LLVM’s SLP vectorizer is target-independent, it models such an alternating pattern as two vector arithmetic instructions followed by a vector blending instruction that combines the results. The error occurs when the LLVM’s vectorizer includes the cost of the blending instruction into its analysis and overestimates the total vectorization overhead. VEGEN does not suffer from such issues because VEGEN has direct knowledge of which target instructions are available.

5.9 Related Work

VEGEN is related two long lines of work on compiler research—automatic vectorization and automatic backend-generation from ISA semantics. VEGEN is also more broadly related to recent work on program superoptimization and synthesis, both of which seek to automatically generate target programs while relying only on a declarative description of instruction semantics.

5.9.1 Auto-vectorization

Loop vectorization and SLP vectorization are the two dominant vectorization techniques used by modern compilers. Both types of vectorization techniques do not model non-SIMD vector instruction in principle, but their implementations in mainstream compilers such as LLVM have some special case non-SIMD support.

Nuzman and Zaks [Nuzman et al., 2006] proposed a technique for vectorizing interleaved memory accesses within a loop-based vectorizer. Eichenberger et al. proposed a technique for vectorizing misaligned memory accesses [Eichenberger et al., 2004],

and FlexVec [Baghsorkhi et al., 2016] extends loop vectorizers to support vectorizing irregular programs with manually written rules. In contrast, VEGEN systematically adds support to generate non-SIMD instructions automatically and is not limited to a particular class of non-SIMD instructions.

The vectorizer generated by VEGEN is more similar to SLP vectorization introduced by Larsen and Amarasinghe [Larsen and Amarasinghe, 2000]. However, VEGEN supports a more general type of parallelism (LLP) and can therefore target non-SIMD instructions. Almost all published SLP vectorization techniques propose algorithmic improvements to capture more parallelism within the SLP framework. Some examples are Holistic SLP vectorization [Liu et al., 2012], Super-node SLP [Porpodas et al., 2019], TSLP [Porpodas and Jones, 2015], PSLP [Porpodas et al., 2015], VW-SLP [Porpodas et al., 2018], and ILP solver-aided goSLP [Mendis and Amarasinghe, 2018].

There are domain-specific vectorizers that exploit architecture-specific vector instructions as well as application-specific patterns. The SPIRAL project [Puschel et al., 2005] proposes several auto-vectorization schemes specific to DSP algorithms. More specifically, they propose a target-independent search-based vectorizing compiler targeting DSP algorithms [Franchetti and Püschel, 2002] and show how to use the vector swizzle instructions supported by the AVX and Larrabee ISAs to implement the matrix transpositions found in FFTs [McFarlin et al., 2011]. Compared to SPIRAL and its extensions, VEGEN is a general-purpose vectorizer and not designed to target any specific vector instruction sets.

5.9.2 Instruction Selection

VEGEN closely related to the research on building retargetable compilers. VEGEN is different from this line of work in that it focuses on extracting fine-grained parallelism (as a vectorizer) while simultaneously being aware of the detailed operations supported by these target instructions (similar to an instruction selector). Instruction selection—regardless of the quality of the code generator—alone is insufficient for automatically targeting non-SIMD vector instructions because traditional instruction selectors only lowers IR vector instructions—thus requiring cooperation with the vectorizer.

Ganapathi et al. [1982] presented a survey on retargetable code generation. Cattell [1980] investigated automatically generating code generators from machine descriptions. Ramsey and Fernández [1997] proposed a specification language for describing instruction encoding. Buchwald et al. [2018] synthesized instruction selection rules for 32-bit x86 integer instructions from their bit-vector specification.

5.9.3 Superoptimization

VEGEN is more broadly related to superoptimization, which uses search techniques to directly generate optimized programs based on instruction semantics. In principle, a superoptimizer can accomplish what VEGEN does, but in practice, existing superoptimizers are orders of magnitude slower than auto-vectorizers such as VEGEN.

Bansal and Aiken [2006] constructed a peephole superoptimizer by exhaustively enumerating short sequences of x86 instructions. Schkufza et al. [2013] proposed a stochastic superoptimizer that trades completeness for scalability via a Markov Chain Monte Carlo sampler. Barthe et al. [2013] proposed a synthesizing vectorizer that works by first unrolling the scalar code and then using an enumerative synthesizer to find more an efficient vector program that implements the unrolled loop body. Phothilimthana et al. [2016] build on previous work on enumerative [Barthe et al., 2013], stochastic [Schkufza et al., 2013], and solver-based synthesis to scale up superoptimization. Sasnauskas et al. [2017] described a superoptimizer for straight-line scalar LLVM IR.

5.10 Summary

I have described a framework for building target-aware vectorizers that can use non-SIMD instructions. We introduce Lane Level Parallelism, a new model of short vector parallelism that captures the kind of parallelism implemented by non-SIMD instructions. We realize this framework with VEGEN, a system that takes vector instruction semantics as input and generates a target-aware vectorizer that uncovers LLP found in straight-line code sequences. VEGEN is flexible: to target a new vector

instruction set, the developers only need to describe the semantics of the new vector instructions. VEGEN allows compilers to target new vector instructions with less development effort and thus enable the creation of more robust vectorizers in future compilers.

Chapter 6

Conclusion and Future Work

This thesis has demonstrated how to generalize the traditional SLP framework to vectorize a larger class of programs. In particular, I have shown how to generalize SLP vectorization to vectorize across basic blocks by using predicated SSA a new IR that explicitly represents control dependencies as symbolic predicates. I have also shown how to perform demand-driven versioning for SLP while still preserving its flexibility; together with the proposed versioning framework, SLP vectorization can now support many traditionally disparate optimizations—such as speculative vectorization and iteration space partitioning— automatically without having the concepts underlying these optimization being hardcoded in the vectorizer. This versioning framework, while motivated by SLP vectorization, is agnostic to specific optimizations; for example, I have used it to implement redundant load elimination while getting non-trivial speedups with relatively low implementation effort. Finally, I have shown how to generalize SLP vectorization to target more irregular and complex that do not conform to our traditional notion of SIMD parallelism; this allows us, for example, to automatically use Intel’s neural network instructions even on non-machine learning kernels (e.g., DSP kernels).

While my work has made SLP more effective, there is more work to be done.

Better Packing Heuristics. There is on-going work both in academia and industry to design better SLP packing heuristics. When I first started my PhD, LLVM’s SLP

implementation (LLVM 7.0.0) was just under 8000 lines of C++. Seven years and 13 major releases later, the implementation (in LLVM 20.0.0) has ballooned to over 20,000 lines, and this accounting only includes the code in the main transformation, excluding many changes in the transformation and analysis utilities. The majority of the series of changes over the years have been on its packing heuristic. The heuristic has been made more sophisticated. Unfortunately, all the changes still assume that the target vector instruction set is SIMD in nature, still leaving more complex vector instructions such as many of the neural network instructions supported vendors like ARM and Intel underutilized. While I have proposed some new packing heuristics to use these instructions, I believe more work can be done to improve the effectiveness of pack selection for non-SIMD vector instructions.

Cost Model for Control-flow Vectorization. So far, my work on control-flow vectorization ignores the performance effect of code motion across different control-flow boundaries. While predicated SSA dramatically simplifies code motion, it can have non-negligible effects on the performance on the final machine code in away belied by the ease with which the transformation is performed. One can easily reorder an instructions in predicated SSA without affecting correctness (assuming no dependencies are violated) but the resulting control-flow graph can be dramatically different, with major implications in the number of branches executed and the number of branches mispredicated. To make my approach more useful for real-world programs, more work is required to 1) understand the impact of such code motion in predicated SSA and 2) building a performance model that can be integrated in both the SLP framework and other transformations (such as the proposed versioning framework) that uses predicated SSA.

Towards a unified framework of vectorization. My work has demonstrated that, on many benchmarks, a single implementation of my SLP vectorizer can out-perform both of LLVM’s loop and SLP vectorizers combined. In the future, for both capability and maintainability, I envision that production compilers will employ only as single vectorizer that blurs the distinction between loop and SLP vectorization. This

vectorizer will have the flexibility of SLP vectorization combined with the scalability of loop vectorization; as Rocha et al. [2020] demonstrated in their research one approach to realize this vision is to integrate the SLP packing heuristics into the unroller so that the unrolling is done driven by the heuristic while also informing it of the original looping structure so the heuristic does not need to waste cycles discovering which instructions are isomorphic. This vision is also in line with the direction that LLVM’s vectorizer developers are undertaking with VPlan [?], which rearchitects the traditional loop vectorization pipeline by inserting an additional *planning* stage where the vectorizer can choose between several vectorization plans before code generation. This rearchitecting is similar to how SLP vectorization decouples the process of vector pack selection from code generation, except VPlan is loop-centric and much of the planning centers around coarse-grained decisions such as which unrolling factor should be applied. (In some cases it is beneficial to unroll more than the native vector length to increase instruction-level parallelism.)

Vectorization as way to target domain-specific accelerators. The work presented in Chapter 5 is motivated in part to utilize the instructions that are increasingly being introduced to accelerate machine learning inferences. Of course, much of the gain in inference is being made in specialized accelerators such as Nvidia’s tensor cores and Google’s Tensor Processing Unit (TPU). Unlike CPU vector extensions, these accelerators have their own address space and data movement optimization is even more important. I argue that compilation for these accelerators should be treated as a vectorization problem thus relieving programmers from the burden from intrinsic programming. (For example, tensor core intrinsics is still the mainstream way of programming tensor cores.) For performance portability it is likely that the best way forward is to use a high-level, data parallel languages so that the compiler can focus on *mapping* parallelism to the hardware instead of *rediscovering* parallelism which is abundant in deep learning workload and often clear to the programmer.

Bibliography

llvm::targettransforminfo class reference. https://llvm.org/doxygen/classllvm_1_1TargetTransformInfo.html, 2022.

Llvm language reference manual. <https://llvm.org/docs/LangRef.html>, 2023.

Neil Adit and Adrian Sampson. Performance left on the table: an evaluation of compiler autovectorization for risc-v. *International Symposium on Microarchitecture*, 2022.

Randy Allen and Ken Kennedy. Automatic translation of FORTRAN programs to vector form. *ACM Transactions on Programming Languages and Systems*, 1987.

Randy Allen, Ken Kennedy, Carrie Porterfield, and Joe Warren. Conversion of control dependence to data dependence. In *Symposium on Principles of Programming Languages*, 1983.

Péricles Alves, Fabian Gruber, Johannes Doerfert, Alexandros Lamprineas, Tobias Grosser, Fabrice Rastello, and Fernando Magno Quintão Pereira. Runtime pointer disambiguation. In *International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, 2015.

Apple. Accelerate simd library. <https://developer.apple.com/documentation/accelerate/simd-library>, 2025.

Leonardo Bachega, Siddhartha Chatterjee, Kenneth A. Dockser, John A. Gunnels, Manish Gupta, Fred G. Gustavson, Christopher A. Lapkowski, Gary K. Liu, Mark P. Mendell, Charles D. Wait, and T. J. Chris Ward. A high-performance SIMD floating point unit for BlueGene/L: Architecture, compilation, and algorithm design. In *International Conference on Parallel Architecture and Compilation Techniques*, 2004.

Sara S. Baghsorkhi, Nalini Vasudevan, and Youfeng Wu. Flexvec: Auto-vectorization for irregular loops. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2016.

Sorav Bansal and Alex Aiken. Automatic generation of peephole superoptimizers. In *International Conference on Architectural Support for Programming Languages and Operating Systems*, 2006.

- Adrian Barredo, Juan M Cebrian, Mateo Valero, Marc Casas, and Miquel Moreto. Efficiency analysis of modern vector architectures: vector alu sizes, core counts and clock frequencies. *The Journal of Supercomputing*, 2020.
- Gilles Barthe, Juan Manuel Crespo, Sumit Gulwani, Cesar Kunz, and Mark Marron. From relational verification to SIMD loop synthesis. In *Symposium on Principles and Practice of Parallel Programming*, 2013.
- Bob Blainey, Christopher Barton, and José Nelson Amaral. Removing impediments to loop fusion through code transformations. In *International Workshop on Languages and Compilers for Parallel Computing*, 2002.
- Sebastian Buchwald, Andreas Fried, and Sebastian Hack. Synthesizing an instruction selection rule library from semantic specifications. In *International Symposium on Code Generation and Optimization*, 2018.
- David Callahan, Jack J Dongarra, and David Levine. Vectorizing compilers: A test suite and results. In *ACM/IEEE Conference on Supercomputing*, 1988.
- R. G. Cattell. Automatic derivation of code generators from machine descriptions. *ACM Transaction on Programming Languages and Systems*, 1980.
- Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *Symposium on Operating Systems Design and Implementation*, 2018.
- Yishen Chen, Charith Mendis, Michael Carbin, and Saman Amarasinghe. VeGen: A vectorizer generator for SIMD and beyond. In *International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021.
- Yishen Chen, Charith Mendis, and Saman Amarasinghe. All you need is superword-level parallelism: Systematic control-flow vectorization with SLP. In *Programming Language Design and Implementation*, 2022.
- Khushboo Chitre, Piyus Kedia, and Rahul Purandare. The road not taken: Exploring alias analysis based optimizations missed by the compiler. *Proceedings of the ACM on Programming Languages*, 2022.
- Khushboo Chitre, Piyus Kedia, and Rahul Purandare. Rapid: Region-based pointer disambiguation. *Proceedings of the ACM on Programming Languages*, 2023.
- Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 1991.

- Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, 2008.
- Alexandre E. Eichenberger, Peng Wu, and Kevin O’Brien. Vectorization for SIMD architectures with alignment constraints. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2004.
- Franz Franchetti and Markus Püschel. A SIMD vectorizing compiler for digital signal processing algorithms. In *International Parallel and Distributed Processing Symposium*, 2002.
- Mahadevan Ganapathi, Charles N. Fischer, and John L. Hennessy. Retargetable compiler code generation. *ACM Computing Surveys*, 1982.
- Tobias Grosser, Armin Größlinger, and Christian Lengauer. Polly – performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters*, 2012.
- Khronos Group. OpenCL 1.0 Specification. <http://khronos.org/registry/cl/specs/openc1-1.0.pdf>, 2009.
- ARM Holdings. Arm architecture reference manual Armv8, 2011. URL <https://developer.arm.com/documentation/ddi0487/latest/>.
- Susan Horwitz. Precise flow-insensitive may-alias analysis is np-hard. *ACM Transaction on Programming Languages and Systems*, 1997.
- Intel Corporation. Intel Intrinsics Guide, 2012. URL <https://software.intel.com/sites/landingpage/IntrinsicsGuide/>.
- Intel Corporation. Intel 64 and IA-32 Architectures Software Developer’s Manual, 2025. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- Ralf Karrenberg and Sebastian Hack. Whole Function Vectorization. In *International Symposium on Code Generation and Optimization*, 2011.
- Ken Kennedy and Kathryn S McKinley. Maximizing loop parallelism and improving data locality via loop fusion and distribution. In *International Workshop on Languages and Compilers for Parallel Computing*, pages 301–320. Springer, 1993.
- Christoforos E. Kozyrakis and David A. Patterson. Scalable vector processors for embedded systems. *IEEE Micro*, 2003.
- Samuel Larsen and Saman Amarasinghe. Exploiting superword level parallelism with multimedia instruction sets. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2000.

- Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization: Feedback-directed and Runtime Optimization*, 2004.
- Jin Lin, Tong Chen, Wei-Chung Hsu, Pen-Chung Yew, Roy Dz-Ching Ju, Tin-Fook Ngai, and Sun Chan. A compiler framework for speculative analysis and optimizations. In *Programming Language Design and Implementation*, 2003.
- Bangtian Liu, Avery Laird, Wai Hung Tsang, Bardia Mahjour, and Maryam Mehri Dehnavi. Combining run-time checks and compile-time analysis to improve control flow auto-vectorization. In *International Conference on Parallel Architectures and Compilation Techniques*, 2023.
- Jun Liu, Yuanrui Zhang, Ohyoung Jang, Wei Ding, and Mahmut Kandemir. A compiler framework for extracting superword level parallelism. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2012.
- LLVM. Llvm project. <https://github.com/llvm/llvm-project>, 2025.
- Saeed Maleki, Yaoqing Gao, Maria J Garzarán, Tommy Wong, and David A Padua. An evaluation of vectorizing compilers. In *International Conference on Parallel Architectures and Compilation Techniques*, 2011.
- Daniel McFarlin, Volodymyr Arbatov, Franz Franchetti, and Markus Püschel. Automatic SIMD vectorization of fast Fourier transforms for the Larrabee and AVX instruction sets. In *International Conference on Supercomputing*, 2011.
- Charith Mendis and Saman Amarasinghe. goSLP: Globally optimized superword level parallelism framework. *Proceedings of the ACM on Programming Languages*, 2018.
- Simon Moll and Sebastian Hack. Partial control-flow linearization. In *Programming Language Design and Implementation*, 2018.
- Dorit Nuzman and Ayal Zaks. Outer-loop vectorization: Revisited for short SIMD architectures. In *International Conference on Parallel Architectures and Compilation Techniques*, 2008.
- Dorit Nuzman, Ira Rosen, and Ayal Zaks. Auto-vectorization of interleaved data for SIMD. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2006.
- Karl J. Ottenstein, Robert A. Ballance, and Arthur B. MacCabe. The program dependence web: A representation supporting control-, data-, and demand-driven interpretation of imperative languages. In *Programming Language Design and Implementation*, 1990.
- Joseph CH Park and Mike Schlansker. *On predicated execution*. 1991.

- Matt Pharr and William R. Mark. ispc: A SPMD Compiler for High-Performance CPU Programming. In *Innovative Parallel Computing*, 2012.
- Phitchaya Mangpo Phothilimthana, Aditya Thakur, Rastislav Bodik, and Dinakar Dhurjati. Scaling up superoptimization. In *International Conference on Architectural Support for Programming Languages and Operating Systems*, 2016.
- Angela Pohl, Biagio Cosenza, Mauricio Alvarez Mesa, Chi Ching Chi, and Ben Juurlink. An evaluation of current simd programming models for c++. In *Proceedings of the 3rd Workshop on Programming Models for SIMD/Vector Processing*, 2016.
- Vasileios Porpodas and Timothy M. Jones. Throttling automatic vectorization: When less is more. In *Conference on Parallel Architecture and Compilation*, 2015.
- Vasileios Porpodas, Alberto Magni, and Timothy M. Jones. PSLP: Padded SLP automatic vectorization. In *International Symposium on Code Generation and Optimization*, 2015.
- Vasileios Porpodas, Rodrigo CO Rocha, and Luís FW Góes. VW-SLP: auto-vectorization with adaptive vector width. In *International Conference on Parallel Architectures and Compilation Techniques*, 2018.
- Vasileios Porpodas, Rodrigo C. O. Rocha, Evgueni Brevnov, Luís F. W. Góes, and Timothy Mattson. Super-Node SLP: Optimized vectorization for code sequences containing operators and their inverse elements. In *International Symposium on Code Generation and Optimization*, 2019.
- Matthew A. Postiff, David A. Greene, Gary S. Tyson, and Trevor N. Mudge. The limits of instruction level parallelism in spec95 applications. *SIGARCH Computer Architecture News*, 1999.
- Louis-Noël Pouchet. PolyBench/C: the polyhedral benchmark suite. <https://web.cse.ohio-state.edu/~pouchet.2/software/polybench/>, 2021.
- Markus Puschel, José MF Moura, Jeremy R Johnson, David Padua, Manuela M Veloso, Bryan W Singer, Jianxin Xiong, Franz Franchetti, Aca Gacic, Yevgen Voronenko, Robert W. Johnson, and Nicholas Rizzolo. SPIRAL: Code generation for DSP transforms. *Proceedings of the IEEE*, 2005.
- Qualcomm. Qualcomm hexagon v73 programmer’s reference manual. <https://docs.qualcomm.com/bundle/publicresource/80-N2040-53.pdf>, 2024a.
- Qualcomm. Arm neoverse v1 cpu specifications. <https://www.arm.com/products/silicon-ip-cpu/neoverse/neoverse-v1>, 2024b.
- Nikola Rajovic, Nikola Puzovic, Lluís Vilanova, Carlos Villavieja, and Alex Ramirez. The low-power architecture approach towards exascale computing. In *Proceedings of the second workshop on Scalable algorithms for large-scale systems*, 2011.

- G. Ramalingam. The undecidability of aliasing. *ACM Transaction on Programming Languages and Systems*, 1994.
- Norman Ramsey and Mary F. Fernández. Specifying representations of machine instructions. *ACM Transaction on Programming Languages and Systems*, 1997.
- Rodrigo C. O. Rocha, Vasileios Porpodas, Pavlos Petoumenos, Luís F. W. Góes, Zheng Wang, Murray Cole, and Hugh Leather. Vectorization-aware loop unrolling with seed forwarding. In *International Conference on Compiler Construction*, 2020.
- Ira Rosen, Dorit Nuzman, and Ayal Zaks. Loop-aware SLP in GCC. In *GCC Developers Summit*, 2007.
- Raimondas Sasnauskas, Yang Chen, Peter Collingbourne, Jeroen Ketema, Jubi Taneja, and John Regehr. Souper: A synthesizing superoptimizer. *arXiv preprint arXiv:1711.04422*, 2017.
- Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *International Conference on Architectural Support for Programming Languages and Operating Systems*, 2013.
- Jaewook Shin, Mary Hall, and Jacqueline Chame. Superword-level parallelism in the presence of control flow. In *International Symposium on Code Generation and Optimization*, 2005.
- Majedul Haque Sujon, R. Clint Whaley, and Qing Yi. Vectorization past dependent branches through speculation. In *International Conference on Parallel Architectures and Compilation Techniques*, 2013.
- Jean-Baptiste Tristan, Paul Govereau, and Greg Morrisett. Evaluating value-graph translation validation for llvm. In *Programming Language Design and Implementation*, 2011.
- Peng Tu and David Padua. Efficient building and placing of gating functions. In *Programming Language Design and Implementation*, 1995.